실용 파이썬 프로그래밍(Practical Python Programming)

https://wikidocs.net/book/4673

지은이 : 최용

최종 편집일시 : 2021년 3월 18일 1:07 오후

저작권 :

31 명이 추천

David Beazley의 Practical Python Programming 번역

용어 선택에 도움 주신 분들께 감사드립니다('리스트 컴프리헨션', '제너레이터').

환영한다!

내가 파이썬(Python)을 거의 25년 전에 처음 배웠을 때, 온갖 지저분한 프로젝트의 생산성을 높일 수 있다는 것에 충격을 받았다. 그로부터 십년이 흐른 뒤에는 다른 사람들에게 그러한 재미를 가르치게 됐다. 2007년부터 지금까지 400개가 넘는 그룹에 이 코스를 가르쳤다. 트레이더, 시스템 관리자, 천문학자, 팅커러, 심지어 화성에 로버를 착륙시키는 데 참여한 수백명의 로켓 과학자가 이 코스를 들었다. 이제 이 코스에 크리에이티브 커먼즈 라이선스를 적용하게 되어 기쁘다. 함께 즐기자!

1. 0. 코스 준비와 개요

Practical Python Programming 코스에 온 것을 환영한다! 이 페이지는 코스 환경 구성과 준비물에 대한 중요 정보를 담고 있다.

가. 소요 시간

이 코스는 강사가 3 ~ 4일 동안 가르치는 것으로 설계했다. 이 코스를 완전히 마치는 데 최소 25 ~ 35 시간을 들일 것으로 계획해야 한다. 참가자 대부분에 있어, 해답 코드(아래를 참조)를 보지 않고 연습 문제를 푸는 것이 만만치 않았다.

나. 준비 및 파이썬 설치

기본적인 Python 3.6 또는 상위 버전만 있으면 된다. 운영 체제, 편집기, IDE, 기타 파이

썬 관련 도구는 필요하지 않다. 서드 파티에 의존하지 않는다.

따라서, 파일에서 데이터를 읽는 데 필요한 스크립트와 간단한 프로그램을 작성하는 법이 이 코스에 포함된다. 그러므로 파일을 다루기 쉬운 환경을 준비해야 한다. 편집기에서 파이썬 프로그램을 작성하고 셸/터미널에서 프로그램을 실행할 수 있어야 한다.

주피터 노트북(Jupyter Notebook)과 같이 좀 더 상호작용적인 환경을 사용하기를 원할 수도 있다. 주피터 노트북 같은 것은 실험에 매우 유용하지만, 이 코스에서 가르치는 개념 중 많은 부분이 프로그램의 조직화(organization)와 관련된다. 여러 파일에 걸쳐 함수, 모듈, 임포트 문을 사용하며 프로그램 리팩터링 작업을 하게 된다. 내 경험상, 주피터 노트북 등에서는 그런 작업을 재현하기 어렵다.

저장소를 복제하기(Forking/Cloning)
이 코스를 수강할 준비로, 깃허브 https://github.com/dabeaz-course/practical-python 저장소를 포크할 것을 권장한다. 그런 다음, 다음 명령으로 로컬 머신에 클론한다.

```
bash % git clone https://github.com/yourname/practical-python
bash % cd practical-python
bash %
```
모든 작업을 practical-python/ 디렉터리에서 수행하라. 포크한 저장소에 스스로 작성한 코드를 커밋하면, 코드를 한 곳에 모아둘 수 있고, 코스를 마쳤을 때 멋진 기록으로 남을 것이다.

포크하는 것이 내키지 않거나 깃허브 계정이 없다면, 다음 명령으로 이 코스의 디렉터리를 여러분의 머신에 클론해도 된다.

```
bash % git clone https://github.com/dabeaz-course/practical-python
bash % cd practical-python
bash %
```
단, 이 경우는 여러분이 작성한 코드를 로컬 머신 외에는 커밋할 수 없다.

디렉터리 구조
모든 코딩 작업은 Work/ 디렉터리에서 수행하라. 이 디렉터리에는 Data/ 디렉터리가 있다. Data/ 디렉터리에는 이 코스에서 사용하는 여러 가지 데이터 파일과 스크립트가 있다. Data/에 있는 파일을 빈번하게 사용하게 된다. 코스의 연습 문제는 Work/ 디렉터리에 프로그램이 있다고 가정한다.

다. 학습 순서

교재는 섹션 1부터 차례대로 완료해야 한다. 연습 문제는 이전 섹션의 코드를 기반으로 한다. 기존 코드를 조금씩 리팩터링하는 연습이 많다.

라. 해답 코드

연습 문제의 완전한 해답 코드가 디렉터리에 있다. 힌트가 필요할 때는 자유롭게 살펴보라. 그러나 최대의 학습효과를 위해서는 스스로 코드를 작성하려고 노력하라.

마지막 편집일시 : 2020년 7월 30일 5:52

1. 파이썬 소개

첫 섹션의 목표는 파이썬 기초를 소개하는 것이다. 작은 프로그램을 편집하고, 실행하고, 디버그하는 법을 배운다. 끝으로, CSV 데이터 파일을 읽어 간단한 계산을 하는 짧은 스크립트를 작성한다.

1.1 파이썬

파이썬은 무엇인가?

파이썬은 인터프리트 방식의 고수준 프로그래밍 언어다. 파이썬은 펄(Perl), 티클(Tcl), 루비(Ruby) 같은 "스크립팅 언어"로 분류된다. 파이썬의 구문은 C 프로그래밍의 요소에 어느 정도 영향을 받았다.

파이썬은 1990년 무렵 Guido van Rossum이 창시했으며 영국의 코미디 그룹 Monty Python에서 이름을 땄다.

파이썬을 어디서 구할 수 있나?

Python.org에서 파이썬을 구할 수 있다. 이 코스를 수강하는 데는 기본 설치로 충분하다. 파이썬 3.6 또는 그 이상의 버전을 설치할 것을 권장한다. 강의 노트와 연습 문제 해답은 파이썬 3.6을 사용했다.

파이썬을 왜 만들었나?

파이썬의 창시자 가라사대,

Amoeba [운영 체제] 프로젝트에 사용할 고수준 언어가 필요하다고 생각한 것이 파이썬을 처음 만들게 된 동기다. 시스템 관리 유틸리티를 C로 작성하는 것은 시간이 너무 많이 걸린다는 것을 깨달았다. 더구나 본 셸(Bourne shell)로 작업하는 것도 몇 가지 난점이 있었다. ... 그래서 C와 셸의 중간쯤 되는 언어를 만들기로 했다.

Guido van Rossum

내 컴퓨터의 어디에 파이썬이 있나?

실행하는 환경에 따라 다르지만, 파이썬은 일반적으로 터미널 또는 명령 셸에서 실행되는 프로그램으로서 설치된다. 터미널에서 python을 실행하면 다음과 같이 작동한다.

```
bash $ python
Python 3.8.1 (default, Feb 20 2020, 09:29:22)
[Clang 10.0.0 (clang-1000.10.44.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello world")
hello world
>>>
```

셸이나 터미널을 처음 사용한다면 일단 멈추고, 그에 대한 간단한 자습서를 본 다음에 돌아오기 바란다.

셸이 아닌 환경에서 파이썬 코딩을 할 수 있는 것이 많이 있지만, 터미널에서 파이썬을 실행, 디버그, 상호작용할 수 있어야 더 나은 파이썬 프로그래머가 될 수 있다. 이것이 파이썬의 기본 환경이기 때문이다. 셸이나 터미널에서 파이썬을 사용할 줄 알면 다른 환경에서도 사용할 수 있다.

연습 문제
연습 문제 1.1: 파이썬을 계산기로 사용하기
컴퓨터에서 파이썬을 실행해 다음 문제를 풀어보자.

행운아 래리는 구글 주식을 주당 235.14 달러에 75주 샀다. 오늘 구글 주가는 711.25 달러다. 파이썬의 상호작용 모드를 계산기처럼 사용해, 래리가 주식을 모두 팔면 얼마를 벌 수 있는지 계산해보자.

```
>>> (711.25 - 235.14) * 75
35708.25
>>>
```

전문가의 팁: 마지막 계산 결과를 사용하려면 밑줄(_) 변수를 사용한다. 예를 들어, 사악한 주식 중개인이 20%를 수수료로 가져가는 경우 래리는 얼마의 수익을 얻는가?

```
>>> _ * 0.80
28566.600000000002
>>>
```

연습 문제 1.2: 도움말
help() 명령을 사용해 abs() 함수에 대한 도움말을 출력해보자. 그런 다음 help()를 사용해 round() 함수의 도움말도 출력하자. 그냥 아무 값 없이 help()라고 입력하면 상호작용적인 도움말이 나타난다.

한 가지 주의할 점으로, 파이썬의 for, if, while 같은 기본 문장에 대해 help()를 사용할 때 help(for)와 같이 입력하면 구문 오류가 발생한다. 그 대신 따옴표를 써서 help("for")를 실행해 보라. 그렇게 해도 도움말을 볼 수 없으면 인터넷에서 검색하라.

심화: http://docs.python.org에서 abs() 함수의 문서를 찾아보라(힌트: 빌트인 함수 관련 라이브러리 레퍼런스에 있다).

연습 문제 1.3: 잘라 붙이기
이 코스는 전통적인 웹 페이지로 구성했으며,상호작용적인 파이썬 코드 예제를 직접 손으로 타이핑해볼 것을 권장한다. 파이썬을 처음으로 배운다면 이와 같이 "천천히" 학습하는 것을 권장한다. 속도를 늦추고

직접 입력하면서 언어를 음미하고, 무엇을 하고 있는지 스스로 생각할 시간을 갖는 것이 좋다.

예제 코드를 "복사해서 붙여넣기"해야 한다면, >>> 프롬프트 뒤부터 선택하되 공백 행이나 다음 번 >>> 프롬프트(두 가지 중 먼저 나오는 것)를 넘어가지 않게 한다. 브라우저에서 "복사"를 선택하고, 파이썬 창으로 가서 "붙여넣기"를 선택해 파이썬 셸에 복사한다. 붙여넣은 후에 "엔터" 키를 눌러야 실행된다.

이 세션에서 파이썬 구문을 복사해서 붙여넣어 실행해 보라.

```
>>> 12 + 20
32
>>> (3 + 4
         + 5 + 6)
18
>>> for i in range(5):
        print(i)

0
1
2
3
4
>>>
```

경고: 기본 파이썬 셸에 파이썬 명령(>>> 이후의 문장)을 동시에 여러 개 붙여넣는 것은 불가능하다. 각 명령을 한 번에 하나씩 붙여넣어라.

이 방법으로 해봤으니, 앞으로 이 수업에서는 코드를 복사해서 붙여넣지 말고 천천히 타이핑해야 함을 명심하자.

연습 문제 1.4: 버스가 언제 오려나?
좀 더 수준 높은 것을 시도해보자. 시카고의 한 버스 정류장(Clark & Balmoral)에서 북쪽으로 가는 CTA 22번 버스를 기다리는 사람들이 얼마나 더 기다려야 하는지 알아보자.

```
>>> import urllib.request
>>> u = urllib.request.urlopen('http://ctabustracker.com/bustime/map/getStopPredictions.jsp?stop=14791&route=22')
>>> from xml.etree.ElementTree import parse
>>> doc = parse(u)
>>> for pt in doc.findall('.//pt'):
        print(pt.text)

6 MIN
18 MIN
28 MIN
>>>
```

그렇다. 방금 단 6줄의 코드로 웹페이지 하나를 다운로드해 XML 문서를 파싱하고 유용한 정보를 추출

했다. 이 데이터는 실제 웹사이트 http://ctabustracker.com/bustime/home.jsp에 제공된다. 다시 시도해 예상 시간이 바뀌는지 지켜보라.

참고: 이 서비스는 30분 내에 오는 버스의 도착 시간만 제공한다. 만약 당신이 다른 시간대에 살고 있어 시카고가 밤늦은 시간이라면 버스가 다니지 않아 출력이 없을 수도 있다. 위의 링크를 확인하라.

첫 번째 문장인 import urllib.request가 실패했다면, 아마 파이썬 2를 사용해서 그럴 것이다. 이 강좌는 파이썬 3.6 이후 버전을 사용한다. 필요하다면 https://www.python.org에서 다운로드하라.

HTTP 프락시 서버가 필요한 작업 환경이라면 이 연습 문제를 풀기 위해 HTTP_PROXY 환경변수를 설정해야 할 수도 있다. 예:

```
>>> import os
>>> os.environ['HTTP_PROXY'] = 'http://yourproxy.server.com'
>>>
```

잘 안 되더라도 걱정할 필요는 없다. 이 코스의 나머지 부분은 XML 파싱을 다루지 않는다.

마지막 편집일시 : 2020년 6월 30일 9:49

1.2 첫 번째 프로그램

이 섹션에서는 처음으로 파이썬 프로그램을 작성해 인터프리터로 실행하고, 기본적인 디버깅을 해본다.

파이썬 실행하기

파이썬 프로그램은 항상 인터프리터에서 실행된다.

인터프리터는 "콘솔 기반" 애플리케이션으로, 명령 셸에서 실행된다.

```
python3
Python 3.6.1 (v3.6.1:69c0db5050, Mar 21 2017, 01:21:04)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

전문 프로그래머는 이런 방식으로 인터프리터를 사용하는 데 전혀 어려움이 없겠지만, 초보자에게는 그리 친화적이지 않다. 다른 인터페이스를 통해 파이썬을 사용할 수 있는 환경도 있다. 그것도 좋지만, 파이썬 터미널을 실행하는 법을 배워두면 쓸모가 있다.

상호작용 모드

파이썬을 시작할 때 상호작용(interactive) 모드를 선택해 실험할 수 있다.

문장을 타이핑해서 즉시 실행해볼 수 있다. 편집/컴파일/실행/디버그 사이클이 존재하지 않는다.

```
>>> print('hello world')
hello world
>>> 37*42
1554
>>> for i in range(5):
```

```
...     print(i)
...
0
1
2
3
4
>>>
```
이것을 읽기-평가-프린트-루프(read-eval-print-loop, REPL)라 한다. REPL은 디버그와 탐색에 매우 유용하다.

잠깐! 파이썬을 상호작용 모드로 사용하는 법을 모르겠으면 진도를 나가지 말고 사용법부터 익혀라. IDE를 사용한다면 메뉴 옵션이나 다른 창에 숨겨져 있을 것이다. 이 코스는 학습자가 인터프리터와 상호작용할 수 있을 것으로 가정한다.

REPL을 좀 더 살펴보자.

>>>은 문장을 새로 시작하는 인터프리터 프롬프트다.
...은 문장을 이어가는 인터프리터 프롬프트다. 타이핑을 마치려면 공백 행을 입력한다. 그러면 입력한 것이 실행된다.
... 프롬프트는 환경에 따라 보일 수도 있고 그렇지 않을 수도 있다. 이 코스는 코드 예제를 쉽게 복사/붙여넣기 할 수 있게 이것을 공백으로 처리한다.

밑줄(_)은 최종 결과를 저장한다.

```
>>> 37 * 42
1554
>>> _ * 2
3108
>>> _ + 50
3158
>>>
```
단, 상호작용 모드에서만 이렇게 작동한다. 프로그램에 _를 사용하면 안 된다.

프로그램 작성하기
프로그램은 .py 파일에 저장한다.

```
# hello.py
print('hello world')
```
선호하는 텍스트 편집기를 사용해 이러한 파일을 만들 수 있다.

프로그램 실행하기
프로그램을 작성했으면, 터미널에서 python 명령을 함께 사용해 실행한다. 예를 들어, 유닉스(Unix) 명령행에서는 다음과 같이 실행한다.

```
bash % python hello.py
```

hello world
bash %
윈도우(Windows) 셸에서는 다음과 같이 실행한다.

```
C:\SomeFolder>hello.py
hello world

C:\SomeFolder>c:\python36\python hello.py
hello world
```

참고: 윈도우에서는 파이썬 인터프리터의 전체 경로를 지정해야 할 수도 있다(예: c:\python36\python). 그렇지만, 파이썬을 일반적인 방식으로 설치했다면 hello.py와 같이 프로그램 이름만 타이핑해도 실행할 수 있을 것이다.

예제 프로그램
다음 문제를 풀어 보자.

어느 날 아침, 당신은 시카고의 시어스 타워(Sears tower) 근처를 거닐다가 보도에 1 달러 지폐를 한 장 올려뒀다. 그 후 매일 외출할 때마다 그 위에 지폐를 얹어 탑을 쌓으며, 높이는 매일 두 배로 불어난다. 돈으로 쌓은 탑의 높이가 시어스 타워의 높이와 같아지려면 시간이 얼마나 걸릴까?

풀이:

```
# sears.py
bill_thickness = 0.11 * 0.001 # 지폐의 두께(0.11 mm)를 미터로 환산
sears_height = 442 # 높이(미터)
num_bills = 1
day = 1

while num_bills * bill_thickness < sears_height:
    print(day, num_bills, num_bills * bill_thickness)
    day = day + 1
    num_bills = num_bills * 2

print('Number of days', day)
print('Number of bills', num_bills)
print('Final height', num_bills * bill_thickness)
```

실행하면 다음과 같이 출력된다.

```
bash % python3 sears.py
1 1 0.00011
2 2 0.00022
3 4 0.00044
4 8 0.00088
5 16 0.00176
6 32 0.00352
...
```

21 1048576 115.34336
22 2097152 230.68672
Number of days 23
Number of bills 4194304
Final height 461.37344
이 프로그램에서 파이썬의 핵심 개념을 배울 수 있다.

문장(Statement)
파이썬 프로그램은 일련의 문장으로 이뤄진다.

```
a = 3 + 4
b = a * 2
print(b)
```
각 문장은 개행(newline)으로 끝난다. 파일의 끝에 도달할 때까지 한 문장씩 실행된다.

부연 설명
주석은 실행되지 않는 텍스트다.

```
a = 3 + 4
# 이것이 주석이다
b = a * 2
print(b)
```
주석은 #으로 표시하며, 해당 행의 끝까지다.

변수(Variable)
변수는 값(value)의 이름(name)이다. a에서 z까지(소문자 및 대문자)를 사용할 수 있다. 밑줄(_) 문자도 사용할 수 있다. 숫자도 변수명의 일부로 사용할 수 있지만, 변수명의 첫 글자에는 숫자를 사용할 수 없다.

```
height = 442 # 가능
_height = 442 # 가능
height2 = 442 # 가능
2height = 442 # 불가
```
타입(Type)
변숫값의 타입을 선언할 필요가 없다. 변수와 관련된 타입은 변수명이 아니라 오른쪽에 무엇이 오느냐에 따라 결정된다.

```
height = 442            # 정수
height = 442.0          # 부동소수점
height = 'Really tall' # 문자열
```
파이썬은 동적으로 타입이 결정(dynamically typed)된다. 프로그램이 실행됨에 따라 변수 "타입"이 무엇인지에 대한 인식이 달라질 수 있다. 타입은 현재 할당된 값이 무엇인지에 달렸다.

대소문자 구분
파이썬은 대문자와 소문자를 구별한다. 대문자와 소문자를 다른 글자로 간주한다. 다음 세 변수는 모두 서로 다르다.

```
name = 'Jake'
Name = 'Elwood'
NAME = 'Guido'
```
언어의 키워드는 모두 소문자다.

```
while x < 0:    # OK
WHILE x < 0:    # 오류
```
루핑(Looping)
while 문은 루프(loop)를 실행한다.

```
while num_bills * bill_thickness < sears_height:
    print(day, num_bills, num_bills * bill_thickness)
    day = day + 1
    num_bills = num_bills * 2

print('Number of days', day)
```
while 아래에 들여쓴 문장은 while 뒤에 오는 표현식이 true인 한 계속 실행된다.

들여쓰기(Indentation)
들여쓰기는 함께 있는 문장들의 그룹을 나타낸다. 다음 코드에서,

```
while num_bills * bill_thickness < sears_height:
    print(day, num_bills, num_bills * bill_thickness)
    day = day + 1
    num_bills = num_bills * 2

print('Number of days', day)
```
다음 문장들은 들여쓰기가 같으므로 함께 묶여 반복적으로 수행된다.

```
    print(day, num_bills, num_bills * bill_thickness)
    day = day + 1
    num_bills = num_bills * 2
```
그렇지만 마지막 행의 print() 문은 들여쓰기하지 않았으므로 루프에 속하지 않는다. 빈 행은 가독성을 위한 것으로, 실행에는 영향을 주지 않는다.

들여쓰기 모범 사례(best practice)
탭(tab) 대신 공백(space)을 사용한다.
수준 당 4개의 공백을 사용한다.
파이썬 구문을 인식하는 편집기를 사용한다.
파이썬은 같은 블록 내의 들여쓰기에 대해서만 일관적인 들여쓰기를 요구한다. 예를 들어, 다음 코드는 오류를 일으킨다.

```
while num_bills * bill_thickness < sears_height:
    print(day, num_bills, num_bills * bill_thickness)
        day = day + 1 # 오류
```

```
    num_bills = num_bills * 2
```

조건(Conditional)

if 문은 조건부 실행에 사용한다.

```
if a > b:
    print('Computer says no')
else:
    print('Computer says yes')
```

조건이 여러 개 있을 때는elif를 사용해 추가로 검사할 수 있다.

```
if a > b:
    print('Computer says no')
elif a == b:
    print('Computer says yes')
else:
    print('Computer says maybe')
```

프린팅(Printing)

print 함수는 전달한 값을 가지고 단일 행의 텍스트를 생성한다.

```
print('Hello world!') # 텍스트 'Hello world!'를 프린트
```

변수를 사용해 프린트할 수 있다. 변수명이 아니라 변숫값이 텍스트로 프린트된다.

```
x = 100
print(x) # 텍스트 '100'을 프린트
```

복수의 값을 print에 전달하면 공백으로 구분되어 프린트된다.

```
name = 'Jake'
print('My name is', name) # 'My name is Jake'를 프린트
```

print()는 항상 끝에 개행을 넣는다.

```
print('Hello')
print('My name is', 'Jake')
```

위 코드는 다음과 같이 프린트한다.

```
Hello
My name is Jake
```

개행 문자를 프린트하고 싶지 않으면 다음과 같이 한다.

```
print('Hello', end=' ')
print('My name is', 'Jake')
```

위의 코드는 다음과 같이 프린트한다.

```
Hello My name is Jake
```

사용자 입력

사용자가 타이핑한 입력의 행을 읽으려면 input() 함수를 사용한다.

```
name = input('Enter your name:')
print('Your name is', name)
```
input은 사용자에게 프롬프트를 프린트하고 사용자의 응답을 반환한다. 이것은 작은 프로그램, 연습 문제, 간단한 디버깅 등에 적합하다. 실제 프로그램에서는 그리 많이 사용하지 않는다.

pass 문
때로는 빈 코드 블록을 지정하고 싶을 때가 있다. 이때 사용할 수 있는 키워드가 pass다.

```
if a > b:
    pass
else:
    print('Computer says false')
```
이것을 "no-op" 문이라고도 한다. 아무 일도 하지 않는다. 나중에 문장을 작성할 지도 모르는 자리를 표시하는 역할이다.

연습 문제
파이썬 파일을 작성해 실행하는 연습 문제를 풀어보자. 지금부터 모든 연습 문제는 practical-python/Work/ 디렉터리에 파일을 편집하는 것으로 가정한다. 적절한 위치를 찾는 것을 돕기 위해, 적당한 파일명으로 빈 파일을 만들어두었다. Work/bounce.py 파일을 찾아보라.

연습 문제 1.5: 얌체공
고무 공을 100 미터 높이에서 떨어뜨린다. 이 공은 땅에 닿을 때마다 원래 높이의 3/5 만큼 튀어오른다. 공이 열 번 튈 동안, 그때마다 높이를 나타내는 테이블을 프린팅하는 프로그램 bounce.py를 작성하라.

프로그램은 다음과 같은 테이블을 만들어야 한다.

```
1 60.0
2 36.0
3 21.599999999999998
4 12.959999999999999
5 7.775999999999999
6 4.6655999999999995
7 2.7993599999999996
8 1.6796159999999998
9 1.0077695999999998
10 0.6046617599999998
```
참고: round() 함수를 사용해 출력을 정돈할 수 있다. 소수점 아래 네 자리까지 출력해 보라.

```
1 60.0
2 36.0
3 21.6
4 12.96
5 7.776
6 4.6656
7 2.7994
```

8 1.6796
9 1.0078
10 0.6047
연습 문제 1.6: 디버깅
다음 코드는 시어스 타워 문제에서 가져왔다. 이 코드에는 버그가 있다.

```
# sears.py

bill_thickness = 0.11 * 0.001     # 미터(0.11 mm)
sears_height   = 442              # 높이(미터)
num_bills      = 1
day            = 1

while num_bills * bill_thickness < sears_height:
    print(day, num_bills, num_bills * bill_thickness)
    day = days + 1
    num_bills = num_bills * 2

print('Number of days', day)
print('Number of bills', num_bills)
print('Final height', num_bills * bill_thickness)
```

위의 코드를 복사해 sears.py라는 이름의 새 프로그램에 붙여넣어라. 코드를 실행하면 다음과 같은 오류 메시지가 뜨며 프로그램이 충돌한다(crash).

```
Traceback (most recent call last):
  File "sears.py", line 10, in <module>
    day = days + 1
NameError: name 'days' is not defined
```

오류 메시지를 읽는 것은 파이썬 코드에서 중요한 부분이다. 프로그램이 충돌한다면 트레이스백(traceback) 메시지에서 마지막 행을 읽으면 프로그램이 충돌한 진짜 이유를 알 수 있다. 그 위에는 소스 코드의 일부가 있으며 파일명과 행 번호가 있다.

코드의 몇 번째 행에 오류가 있는가?
무엇이 잘못되었는가?
오류를 수정하라
프로그램을 성공적으로 실행하라
마지막 편집일시 : 2020년 10월 3일 5:42

1.3 숫자
이 섹션은 수학 계산을 논의한다.

숫자 타입
파이썬에는 네 가지 숫자 타입이 있다.

불린
정수

부동소수점
복소수(허수)
불린(bool)
불린 값은 True와 False 중 하나다.

```
a = True
b = False
```

수학 계산에서는 각각 1과 0으로 평가한다.

```
c = 4 + True # 5
d = False
if d == 0:
    print('d is False')
```

그렇지만, 코드를 이런 식으로 작성하지 말자. 이상하니까.

정수(int)
임의의 크기에 밑수(base)가 있는 부호 있는(signed) 값이다.

```
a = 37
b = -29939299372771662737712848181224123 1
c = 0x7fa8       # 16진수
d = 0o253        # 8진수
e = 0b10001111   # 2진수
```

일반적인 연산:

```
x + y       덧셈
x - y       뺄셈
x * y       곱셈
x / y       나눗셈(부동소수점수를 생성)
x // y      Floor 나눗셈(정수를 생성)
x % y       모듈로(나머지)
x ** y      제곱
x << n      왼쪽으로 비트 시프트(shift)
x >> n      오른쪽으로 비트 시프트
x & y       AND 비트 연산
x | y       OR 비트 연산
x ^ y       XOR 비트 연산
~x          NOT 비트 연산
abs(x)      절댓값
```

부동소수점(float)
십진수 또는 지수 표기법을 사용해 부동소수점수를 지정한다.

```
a = 37.45
b = 4e5 # 4 x 10**5 또는 400,000
c = -1.345e-10
```

부동소수점은 네이티브 CPU 표현 IEEE 754를 사용해 배정밀도(double precision)로 나타낸다. C 프로

그래밍 언어의 double 타입과 같다.

정밀도는 17 자리
-308에서 308까지

부동소수점수를 십진수로 나타낼 경우 부정확함에 유의하라.

```
>>> a = 2.1 + 4.2
>>> a == 6.3
False
>>> a
6.300000000000001
>>>
```

이것은 파이썬 이슈가 아니라, CPU의 부동소수점 하드웨어 때문이다.

일반적인 연산:

```
x + y       덧셈
x - y       뺄셈
x * y       곱셈
x / y       나눗셈
x // y      Floor 나눗셈
x % y       모듈로
x ** y      제곱
abs(x)      절댓값
```

비트 연산을 제외하면, 이 연산들은 정수 연산과 같다. 추가적인 수학 함수가 math 모듈에 있다.

```
import math
a = math.sqrt(x)
b = math.sin(x)
c = math.cos(x)
d = math.tan(x)
e = math.log(x)
```

비교
숫자에 대해 다음과 같은 비교/관계 연산을 할 수 있다.

```
x < y       작음
x <= y      작거나 같음
x > y       큼
x >= y      크거나 같음
x == y      같음
x != y      같지 않음
```

다음을 사용해 복잡한 불린 표현식을 만들 수 있다.

and, or, not

예:

```
if b >= a and b <= c:
    print('b is between a and c')

if not (b < a or b > c):
    print('b is still between a and c')
```

숫자 변환하기
타입명을 사용해 숫자를 변환할 수 있다.

```
a = int(x)     # x를 int로 변환
b = float(x)   # x를 float로 변환
```

한번 해보자.

```
>>> a = 3.14159
>>> int(a)
3
>>> b = '3.14159' # 숫자를 포함하는 문자열을 가지고도 가능
>>> float(b)
3.14159
>>>
```

연습 문제
참고: 연습 문제는 practical-python/Work 디렉터리에서 작업하는 것으로 가정한다. mortgage.py 파일을 찾아보라.

연습 문제 1.7: 데이브의 주택 담보 대출
데이브는 500,000 달러의 30년 고정 이율 주택 담보 대출(mortgage)을 받기로 결정했다. 이율은 5%이고 매달 납부할 금액은 2684.11 달러다.

다음은 대출 기간 동안 지불할 총액을 계산하는 프로그램이다.

```
# mortgage.py

principal = 500000.0
rate = 0.05
payment = 2684.11
total_paid = 0.0

while principal > 0:
    principal = principal * (1+rate/12) - payment
    total_paid = total_paid + payment

print('Total paid', total_paid)
```

이 프로그램을 입력해 실행하라. 답이 966,279.6로 나와야 한다.

연습 문제 1.8: 추가 납입

데이브가 처음 12개월 동안 매달 1000 달러를 추가로 납입한다면 어떻게 될까?

이 추가 납입금 계산을 포함하도록 프로그램을 수정해 총 납입금액과 소요 월수를 프린트해보자.

프로그램을 실행했을 때 총 납입금이 929,965.62, 소요 월수는 342로 나와야 한다.

연습 문제 1.9: 추가 납입금 계산기
추가 납입금을 일반적으로 다룰 수 있게 프로그램을 수정하자. 사용자가 다음 변수를 설정할 수 있게 한다.

```
extra_payment_start_month = 61
extra_payment_end_month = 108
extra_payment = 1000
```

프로그램이 이 변숫값을 읽고 총 납입액을 계산하게 해 보자.

데이브가 대출 시작 5년 후부터 4년간 매달 1000 달러를 추가로 지불할 경우 총 납입액은 얼마인가?

연습 문제 1.10: 테이블 만들기
월수, 현재까지의 납부액, 남은 원금을 나타내는 테이블을 프린트하게 프로그램을 수정해 보라. 다음과 같이 출력한다.

```
1 2684.11 499399.22
2 5368.22 498795.94
3 8052.33 498190.15
4 10736.44 497581.83
5 13420.55 496970.98
...
308 874705.88 3478.83
309 877389.99 809.21
310 880074.1 -1871.53
Total paid 880074.1
Months 310
```

연습 문제 1.11: 보너스
마지막 달에 초과 납부하는 금액이 생기지 않게 프로그램을 수정해 보라.

연습 문제 1.12: 미스터리
int()와 float()를 사용해 숫자를 변환할 수 있다. 예:

```
>>> int("123")
123
>>> float("1.23")
1.23
>>>
```

그렇다면, 왜 다음과 같이 작동하는지 설명할 수 있는가?

```
>>> bool("False")
```

True
>>>
마지막 편집일시 : 2020년 7월 31일 4:14

1.4 문자열
이 섹션은 텍스트를 가지고 작업하는 법을 소개한다.

리터럴 텍스트(Literal Text) 표현
프로그램에서 문자열 리터럴은 따옴표를 사용한다.

```
# 작은따옴표
a = 'Yeah but no but yeah but...'

# 큰따옴표
b = "computer says no"

# 따옴표 세 개
c = '''
Look into my eyes, look into my eyes, the eyes, the eyes, the eyes,
not around the eyes,
don't look around the eyes,
look into my eyes, you're under.
'''
```

일반적으로 문자열은 한 줄을 넘어가지 않는다. 따옴표 세 개를 사용하면 모든 포매팅을 포함해 여러 행에 걸친 텍스트가 문자열이 된다.

작은따옴표(')와 큰따옴표(")사이에 차이는 없다. 하지만, 문자열의 시작과 끝에 같은 종류의 따옴표를 사용해야 한다.

문자열 이스케이프 코드(escape code)
이스케이프 코드는 제어 문자나 키보드에서 타이핑하기 힘든 문자를 나타내는 데 사용한다. 이스케이프 코드의 예:

```
'\n'      라인 피드(Line feed)
'\r'      캐리지 리턴(Carriage return)
'\t'      탭(Tab)
'\''      작은따옴표(Literal single quote)
'\"'      큰따옴표(Literal double quote)
'\\'      백슬래시(Literal backslash)
```

문자열 표현
문자열의 각 문자는 내부적으로 유니코드(Unicode) "코드 포인트(code-point)"로 저장된다. 다음과 같이 이스케이프 시퀀스(escape sequence)를 사용해 정확한 코드 포인트를 지정할 수 있다.

```
a = '\xf1'          # a = 'ñ'
b = '\u2200'        # b = '∀'
c = '\U0001D122'    # c = '𝄢'
```

```
d = '\N{FOR ALL}'   # d = '∀'
```
사용가능한 전체 캐릭터 코드는 유니코드 문자 데이터베이스(Unicode Character Database)를 참조한다.

문자열 인덱싱(String Indexing)
문자열은 문자에 액세스함에 있어서 배열(array)과 비슷하게 작동한다. 0부터 시작하는 정수 인덱스(index)를 사용한다. 음수 인덱스는 문자열의 끝에서부터 상대적 위치를 가리킨다.

```
a = 'Hello world'
b = a[0]          # 'H'
c = a[4]          # 'o'
d = a[-1]         # 'd'(문자열의 끝)
```
또한 콜론(:)으로 인덱스 범위를 지정함으로써 슬라이스하거나 부분 문자열을 선택할 수 있다.

```
d = a[:5]     # 'Hello'
e = a[6:]     # 'world'
f = a[3:8]    # 'lo wo'
g = a[-5:]    # 'world'
```
끝 인덱스의 문자는 포함되지 않는다. 인덱스를 생략하면 문자열의 시작 또는 끝으로 가정한다.

문자열 연산
이어붙이기(concatenation), 길이(length), 멤버십(membership), 복제(replication).

```
# 이어붙이기(+)
a = 'Hello' + 'World'   # 'HelloWorld'
b = 'Say ' + a          # 'Say HelloWorld'

# 길이(len)
s = 'Hello'
len(s)                  # 5

# 멤버십 테스트(`in`, `not in`)
t = 'e' in s            # True
f = 'x' in s            # False
g = 'hi' not in s       # True

# 복제(s * n)
rep = s * 5             # 'HelloHelloHelloHelloHello'
```
문자열 메서드
문자열에는 문자열 데이터에 대해 다양한 연산을 수행하는 메서드가 있다.

예: 맨 앞이나 맨 뒤의 화이트 스페이스 제거.

```
s = '  Hello '
t = s.strip()     # 'Hello'
```
예: 대소문자 변환

```
s = 'Hello'
l = s.lower()       # 'hello'
u = s.upper()       # 'HELLO'
```
예: 텍스트 교체.

```
s = 'Hello world'
t = s.replace('Hello' , 'Hallo')    # 'Hallo world'
```
그외의 문자열 메서드

문자열에는 텍스트 데이터를 테스트하고 조작하는 다양한 메서드가 있다. 그중 일부를 소개한다.

```
s.endswith(suffix)      # 문자열이 suffix로 끝나는지 확인
s.find(t)               # s에서 t가 처음 나타나는 곳
s.index(t)              # s에서 t가 처음 나타나는 곳
s.isalpha()             # 문자가 영문자인지 여부
s.isdigit()             # 문자가 숫자인지 여부
s.islower()             # 문자가 소문자인지 여부
s.isupper()             # 문자가 대문자인지 여부
s.join(slist)           # s를 구분자(delimiter)로 삼아 문자열의 리스트를 붙이기(join)
s.lower()               # 소문자로 변환
s.replace(old,new)      # 텍스트 교체
s.rfind(t)              # 문자열의 끝에서부터 t를 검색
s.rindex(t)             # 문자열의 끝에서부터 t를 검색
s.split([구분자])       # 문자열을 분할해 부분 문자열의 리스트를 만듦
s.startswith(prefix)    # 문자열이 prefix로 시작하는지 확인
s.strip()               # 앞뒤의 공백을 제거
s.upper()               # 대문자로 변환
```
문자열의 변경가능성(Mutability)
문자열은 "변경불가능(immutable)", 즉 읽기 전용이다. 한번 생성하면 값이 바뀌지 않는다.

```
>>> s = 'Hello World'
>>> s[1] = 'a'
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>>
```
문자열 데이터를 조작하는 모든 연산과 메서드는 항상 새로운 문자열을 생성한다.

문자열 변환
다른 타입의 값을 문자열로 변환하려면 str()을 사용한다. 그 결과로 print() 문이 생성하는 것과 같은 텍스트가 반환된다.

```
>>> x = 42
>>> str(x)
'42'
```

```
>>>
```
바이트 열(Byte String)
8비트 바이트가 끈(string)처럼 이어진 것으로 일반적으로 저수준 입출력(I/O)에 사용되며, 다음과 같이 쓴다.

```
data = b'Hello World\r\n'
```
첫 번째 따옴표 바로 앞에 소문자 b를 붙이면, 텍스트 열(text string)이 아닌 바이트 열로 지정된다.

일반적인 문자열 연산은 대부분 작동한다.

```
len(data)                          # 13
data[0:5]                          # b'Hello'
data.replace(b'Hello', b'Cruel')  # b'Cruel World\r\n'
```
인덱싱은 좀 다르게 작동하는데, 바이트 값을 정수로 반환하기 때문이다.

```
data[0]   # 72 ('H'의 ASCII 코드)
```
텍스트 열과의 변환.

```
text = data.decode('utf-8') # 바이트 열 -> 텍스트 열
data = text.encode('utf-8') # 텍스트 열 -> 바이트 열
```
'utf-8' 인자(argument)는 문자 인코딩을 지정한다. 'ascii'와 'latin1'도 일반적으로 사용하는 값이다.

원시 문자열(Raw String)
원시 문자열은 백슬래시를 해석하지 않는 문자열 리터럴이다. 소문자 "r"을 앞에 붙여 원시 문자열임을 나타낸다.

```
>>> rs = r'c:\newdata\test' # 원시(백슬래시를 해석하지 않음)
>>> rs
'c:\\newdata\\test'
```
문자열은 입력한 그대로의 리터럴 텍스트다. 백슬래시가 특별히 중요할 때 이것을 사용하면 편리하다. 예: 파일명, 정규 표현식(regular expression) 등

f 문자열(f-String)
포매팅된 표현식 대체가 있는 문자열이다.

```
>>> name = 'IBM'
>>> shares = 100
>>> price = 91.1
>>> a = f'{name:>10s} {shares:10d} {price:10.2f}'
>>> a
'       IBM        100      91.10'
>>> b = f'Cost = ${shares*price:0.2f}'
>>> b
'Cost = $9110.00'
>>>
```
참고: f 문자열은 파이썬 3.6 이상에서 지원한다. 포맷 코드의 의미는 나중에 다룬다.

연습 문제
이 연습 문제에서는 파이썬의 문자열 타입에 대한 연산을 실험한다. 파이썬의 상호작용적인 프롬프트에서 쉽게 결과를 확인할 수 있다. 중요:

인터프리터와 상호작용할 때, >>>는 파이썬이 새로운 문장 입력을 요구하는 인터프리터 프롬프트다. 이 연습 문제의 몇몇 문장은 여러 행에 걸쳐 작성되므로, 그것들을 실행하려면 '엔터(return)'를 몇 번 눌러야 한다. 그러니까 예제를 실습할 때 >>>를 입력하지 않는다.

주식 티커 심볼을 포함하는 문자열을 정의하는 것부터 해 보자.

```
>>> symbols = 'AAPL,IBM,MSFT,YHOO,SCO'
>>>
```

연습 문제 1.13: 개별 문자와 부분 문자열을 추출하기
문자열은 문자의 배열이다. 문자를 몇 개 추출해보자.

```
>>> symbols[0]
?
>>> symbols[1]
?
>>> symbols[2]
?
>>> symbols[-1]          # 마지막 문자
?
>>> symbols[-2]          # 음수 인덱스는 문자열 끝에서부터 센다
?
>>>
```

파이썬에서 문자열은 읽기 전용이다.

이것을 확인하기 위해 symbols의 첫 글자를 소문자 'a'로 바꿔보자.

```
>>> symbols[0] = 'a'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>>
```

연습 문제 1.14: 문자열 이어붙이기
문자열 데이터가 읽기 전용이기는 하지만, 새로 생성한 문자열에 변수를 재할당할 수 있다.

다음과 같이 symbols의 끝에 새로운 심벌 "GOOG"를 이어붙여 보라.

```
>>> symbols = symbols + 'GOOG'
>>> symbols
'AAPL,IBM,MSFT,YHOO,SCOGOOG'
>>>
```

아차! 이걸 원한 것이 아니다. symbols 변수에 'AAPL,IBM,MSFT,YHOO,SCO,GOOG'가 들어가게 수정

해보자.

```
>>> symbols = ?
>>> symbols
'AAPL,IBM,MSFT,YHOO,SCO,GOOG'
>>>
```

문자열의 시작에 'HPQ'를 추가해 보자.

```
>>> symbols = ?
>>> symbols
'HPQ,AAPL,IBM,MSFT,YHOO,SCO,GOOG'
>>>
```

문자열이 읽기 전용이라고 했는데, 이 예에서는 원래의 문자열을 수정해 규칙을 위반하는 것처럼 보일 수도 있다. 그런 것이 아니다. 문자열에 대한 연산은 완전히 새로운 문자열을 매번 생성한다. 변수명 심벌(symbol)이 재할당되면, 그것은 새로 생성된 문자열을 가리킨다. 사용하지 않는 문자열은 삭제된다.

연습 문제 1.15: 멤버십 테스팅(부분 문자열 테스팅)
in 연산자를 사용해 부분 문자열을 확인해 보자. 상호작용적인 프롬프트에서 다음 연산을 해 보라.

```
>>> 'IBM' in symbols
?
>>> 'AA' in symbols
True
>>> 'CAT' in symbols
?
>>>
```

'AA'를 확인할 때 왜 True가 반환될까?

연습 문제 1.16: 문자열 메서드
파이썬 프롬프트에서 문자열 메서드를 시험삼아 사용해 보자.

```
>>> symbols.lower()
?
>>> symbols
?
>>>
```

문자열은 항상 읽기 전용임을 명심하자. 연산 결과를 저장하고 싶으면 변수에 넣어라.

```
>>> lowersyms = symbols.lower()
>>>
```

몇 가지 연산을 더 해 보자.

```
>>> symbols.find('MSFT')
?
>>> symbols[13:17]
?
```

```
>>> symbols = symbols.replace('SCO','DOA')
>>> symbols
?
>>> name = '    IBM    \n'
>>> name = name.strip()     # 둘러싸는 공백을 제거
>>> name
?
>>>
```

연습 문제 1.17: f 문자열(f-strings)
변숫값을 포함하는 문자열을 생성하고 싶을 때가 있다.

그럴 때 f 문자열을 사용한다. 예:

```
>>> name = 'IBM'
>>> shares = 100
>>> price = 91.1
>>> f'{shares} shares of {name} at ${price:0.2f}'
'100 shares of IBM at $91.10'
>>>
```

연습 문제 1.10의 mortgage.py 프로그램을 수정해, f 문자열을 사용해 출력을 생성하게 해 보자. 출력을 보기 좋게 정렬해보라.

연습 문제 1.18: 정규 표현식(Regular Expressions)
기본 문자열 연산은 고급 패턴 매칭을 지원하지 않지만, 파이썬의 re 모듈을 가지고 정규 표현식을 사용할 수 있다. 정규 표현식은 그 자체로 큰 주제이지만 여기서는 간단한 예를 들겠다.

```
>>> text = 'Today is 3/27/2018. Tomorrow is 3/28/2018.'
>>> # 날짜가 있는 곳을 모두 찾음
>>> import re
>>> re.findall(r'\d+/\d+/\d+', text)
['3/27/2018', '3/28/2018']
>>> # 날짜가 있는 곳을 모두 찾아 text로 교체
>>> re.sub(r'(\d+)/(\d+)/(\d+)', r'\3-\1-\2', text)
'Today is 2018-3-27. Tomorrow is 2018-3-28.'
>>>
```

re 모듈에 대한 자세한 정보는 공식 문서 https://docs.python.org/library/re.html을 참조하라.

부연 설명
인터프리터를 가지고 실험을 하다 보면, 다른 객체(object)가 지원하는 연산에 대해 더 알고 싶을 것이다. 가령, 문자열에 어떤 연산이 있는지 어떻게 찾을 수 있을까?

파이썬 환경에 따라, 탭 자동완성을 통해 리스트를 볼 수 있을 수도 있다. 다음과 같이 해 보라.

```
>>> s = 'hello world'
>>> s.<tab key>
>>>
```

탭을 눌렀을 때 반응이 없다면, 빌트인 dir() 함수를 사용한다. 예:

```
>>> s = 'hello'
>>> dir(s)
['__add__', '__class__', '__contains__', ..., 'find', 'format',
'index', 'isalnum', 'isalpha', 'isdigit', 'islower', 'isspace',
'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'partition',
'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit',
'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase',
'title', 'translate', 'upper', 'zfill']
>>>
```

dir()은 (.) 이후에 나타날 수 있는 전체 연산의 리스트를 생성한다. 특정 연산에 대한 자세한 내용을 알고 싶으면 help() 명령을 사용하라.

```
>>> help(s.upper)
Help on built-in function upper:

upper(...)
    S.upper() -> string

    Return a copy of the string S converted to uppercase.
>>>
```

마지막 편집일시 : 2020년 10월 4일 1:26

1.5 리스트

이 섹션은 파이썬에서 값의 순서가 유지되는 컬렉션(ordered collection)을 나타내는 기본 타입인 리스트를 소개한다.

리스트 생성

대괄호를 사용해 리스트 리터럴을 정의한다.

```
names = [ 'Elwood', 'Jake', 'Curtis' ]
nums = [ 39, 38, 42, 65, 111]
```

다른 방법으로 리스트를 정의할 수도 있다. 예를 들어, split() 메서드를 가지고 문자열을 분할해 리스트를 만들 수 있다.

```
>>> line = 'GOOG,100,490.10'
>>> row = line.split(',')
>>> row
['GOOG', '100', '490.10']
>>>
```

리스트 연산

어떤 타입이든 리스트 항목(item)이 될 수 있다. append()를 사용해 새 항목을 추가한다.

```
names.append('Murphy')     # 마지막에 추가
```

```
names.insert(2, 'Aretha') # 중간에 삽입
```
리스트를 이어붙이려면 +를 사용한다.

```
s = [1, 2, 3]
t = ['a', 'b']
s + t             # [1, 2, 3, 'a', 'b']
```
리스트의 인덱스에는 정수를 사용한다. 인덱스는 0부터 시작한다.

```
names = [ 'Elwood', 'Jake', 'Curtis' ]

names[0]  # 'Elwood'
names[1]  # 'Jake'
names[2]  # 'Curtis'
```
음수 인덱스는 뒤에서부터 센다.

```
names[-1] # 'Curtis'
```
리스트의 항목을 변경할 수 있다.

```
names[1] = 'Joliet Jake'
names                        # [ 'Elwood', 'Joliet Jake', 'Curtis' ]
```
리스트의 길이.

```
names = ['Elwood','Jake','Curtis']
len(names)  # 3
```
멤버십 테스트(in, not in).

```
'Elwood' in names        # True
'Britney' not in names  # True
```
복제(s * n).

```
s = [1, 2, 3]
s * 3   # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```
리스트 이터레이션(Iteration)과 검색

for를 사용해 리스트 내용을 훑는다.

```
for name in names:
    # name을 사용
    # 예: print(name)
    ...
```
다른 프로그래밍 언어의 foreach 문과 비슷하다.

무엇의 위치를 재빨리 찾으려면 index()를 사용한다.

```
names = ['Elwood','Jake','Curtis']
names.index('Curtis')   # 2
```
원소(element)가 한 번 이상 나타나면, index()는 해당 원소가 처음 나타난 인덱스를 반환한다.

원소를 찾지 못하면 ValueError 예외를 일으킨다.

리스트 제거
특정 값 또는 인덱스를 갖는 항목을 제거할 수 있다.

```
# 값을 사용
names.remove('Curtis')

# 인덱스를 사용
del names[1]
```

항목을 제거하더라도 빈 자리가 생기지는 않는다. 다음 항목이 따라붙는다. 원소가 한 번 이상 나타나는 경우, remove()는 첫 번째 나타난 것만 제거한다.

리스트 정렬
리스트를 "제자리에서(in-place)" 정렬할 수 있다.

```
s = [10, 1, 7, 3]
s.sort()                    # [1, 3, 7, 10]

# 역순
s = [10, 1, 7, 3]
s.sort(reverse=True)        # [10, 7, 3, 1]

# 순서가 유지되는 자료형에 동일하게 적용된다
s = ['foo', 'bar', 'spam']
s.sort()                    # ['bar', 'foo', 'spam']
```

새 리스트를 만들고 싶으면 sorted()를 사용한다.

```
t = sorted(s)               # s는 바뀌지 않으며, t는 정렬된 값을 가짐
```

리스트와 수학
주의: 리스트는 수학 연산을 위해 설계한 것이 아니다.

```
>>> nums = [1, 2, 3, 4, 5]
>>> nums * 2
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
>>> nums + [10, 11, 12, 13, 14]
[1, 2, 3, 4, 5, 10, 11, 12, 13, 14]
```

특히 리스트는 매트랩(MATLAB), 옥타브(Octave), R 등의 벡터나 행렬을 나타내지 않는다. 그런 용도로 사용할 수 있는 패키지가 따로 있다.(예: numpy)

연습 문제
이 연습 문제에서는 파이썬의 리스트 자료형을 실험한다. 지난 섹션에서 주식 심벌을 포함하는 문자열을 다뤘다.

```
>>> symbols = 'HPQ,AAPL,IBM,MSFT,YHOO,DOA,GOOG'
```

문자열의 split() 연산을 사용해 이름의 리스트를 만든다.

```
>>> symlist = symbols.split(',')
```

연습 문제 1.19: 리스트 원소를 추출해 재할당

조회를 해 보자.

```
>>> symlist[0]
'HPQ'
>>> symlist[1]
'AAPL'
>>> symlist[-1]
'GOOG'
>>> symlist[-2]
'DOA'
>>>
```

값을 재할당해보자.

```
>>> symlist[2] = 'AIG'
>>> symlist
['HPQ', 'AAPL', 'AIG', 'MSFT', 'YHOO', 'DOA', 'GOOG']
>>>
```

슬라이스를 해 보자.

```
>>> symlist[0:3]
['HPQ', 'AAPL', 'AIG']
>>> symlist[-2:]
['DOA', 'GOOG']
>>>
```

빈 리스트를 생성하고 항목을 추가해보자.

```
>>> mysyms = []
>>> mysyms.append('GOOG')
>>> mysyms
['GOOG']
```

리스트의 일부를 다른 리스트에 재할당할 수 있다. 예:

```
>>> symlist[-2:] = mysyms
>>> symlist
['HPQ', 'AAPL', 'AIG', 'MSFT', 'YHOO', 'GOOG']
>>>
```

이렇게 하면 왼쪽의 리스트(symlist)는 오른쪽 리스트(mysyms)에 맞춰 크기가 변한다. 위의 예에서, symlist의 마지막 두 개 항목이 mysyms의 한 개 항목으로 교체된다.

연습 문제 1.20: 리스트 항목을 루핑

for 루프는 리스트 같은 시퀀스의 데이터를 루핑한다. 다음 루프를 타이핑하고 무슨 일이 일어나는지 지켜보라.

```
>>> for s in symlist:
        print('s =', s)
# 출력을 살펴보라
```

연습 문제 1.21: 멤버십 테스트

in이나 not in 연산자를 사용해, 심벌 리스트에 'AIG','AA', 'CAT'가 있는지 확인하라.

```
>>> # 'AIG'가 `symlist`에 있는가?(in)
True
>>> # 'AA'가 `symlist`에 있는가?(in)
False
>>> # 'CAT'가 `symlist` 없는가?(not in)
True
>>>
```

연습 문제 1.22: 항목을 추가, 삽입, 삭제

append() 메서드를 사용해 심벌 'RHT'를 symlist의 끝에 추가하라.

```
>>> # 'RHT'를 추가
>>> symlist
['HPQ', 'AAPL', 'AIG', 'MSFT', 'YHOO', 'GOOG', 'RHT']
>>>
```

insert() 메서드를 사용해 심벌 'AA'를 리스트의 두 번째 항목으로 삽입하라.

```
>>> # 'AA'를 리스트의 두 번째 항목으로
>>> symlist
['HPQ', 'AA', 'AAPL', 'AIG', 'MSFT', 'YHOO', 'GOOG', 'RHT']
>>>
```

remove() 메서드를 사용해 'MSFT'를 리스트에서 삭제하라.

```
>>> # 'MSFT'를 삭제
>>> symlist
['HPQ', 'AA', 'AAPL', 'AIG', 'YHOO', 'GOOG', 'RHT']
>>>
```

리스트 끝에 'YHOO'를 한 개 더 추가하라.

참고: 리스트에는 중복된 값이 들어가도 전혀 문제가 없다.

```
>>> # 'YHOO'를 추가
>>> symlist
['HPQ', 'AA', 'AAPL', 'AIG', 'YHOO', 'GOOG', 'RHT', 'YHOO']
>>>
```

리스트에서 'YHOO'의 위치를 빨리 찾으려면 index() 메서드를 사용한다.

```
>>> # 'YHOO'의 첫 번째 인덱스 찾기
4
>>> symlist[4]
```

```
'YHOO'
>>>
```
리스트에 'YHOO'가 몇 번 나오는지 센다.

```
>>> symlist.count('YHOO')
2
>>>
```
첫 번째 나타나는 'YHOO'를 삭제한다.

```
>>> # 첫 번째 나오는 'YHOO'를 삭제
>>> symlist
['HPQ', 'AA', 'AAPL', 'AIG', 'GOOG', 'RHT', 'YHOO']
>>>
```
모든 항목을 찾거나, 모든 항목을 지우는 메서드는 없다. 그렇지만 그렇게 할 수 있는 우아한 방법을 섹션 2에서 알아본다.

연습 문제 1.23: 정렬(Sorting)
리스트를 정렬하고 싶은가? sort() 메서드를 사용하라. 한번 사용해 보자.

```
>>> symlist.sort()
>>> symlist
['AA', 'AAPL', 'AIG', 'GOOG', 'HPQ', 'RHT', 'YHOO']
>>>
```
역순으로 정렬하고 싶다면? 다음과 같이 해 보라.

```
>>> symlist.sort(reverse=True)
>>> symlist
['YHOO', 'RHT', 'HPQ', 'GOOG', 'AIG', 'AAPL', 'AA']
>>>
```
참고: 리스트를 정렬하면 내용이 '제자리에서(in-place)' 수정된다. 그 말은 리스트의 항목은 이리저리 옮겨지지만 새로운 리스트를 생성하지는 않는다는 뜻이다.

연습 문제 1.24: 모두 합치기
문자열의 리스트를 가지고 항목들을 합쳐서(join) 한 개의 문자열을 만들고 싶은가? 다음과 같이 문자열의 join() 메서드를 사용하라(참고: 처음 보면 좀 웃기다).

```
>>> a = ','.join(symlist)
>>> a
'YHOO,RHT,HPQ,GOOG,AIG,AAPL,AA'
>>> b = ':'.join(symlist)
>>> b
'YHOO:RHT:HPQ:GOOG:AIG:AAPL:AA'
>>> c = ''.join(symlist)
>>> c
'YHOORHTHPQGOOGAIGAAPLAA'
>>>
```

연습 문제 1.25: 온갖 것들의 리스트
리스트는 어떤 종류의 객체든 담을 수 있고, 다른 리스트를 담을 수도 있다(예: 중첩된 리스트). 다음과 같이 시도해보라.

```
>>> nums = [101, 102, 103]
>>> items = ['spam', symlist, nums]
>>> items
['spam', ['YHOO', 'RHT', 'HPQ', 'GOOG', 'AIG', 'AAPL', 'AA'], [101, 102, 103]]
```
출력을 잘 들여다 보라. items는 세 개의 원소를 가진 리스트다. 첫 번째 원소는 문자열이지만, 나머지 두 개의 원소는 리스트다.

여러 가지 인덱싱 연산자를 사용해 중첩된 리스트의 항목에 액세스할 수 있다.

```
>>> items[0]
'spam'
>>> items[0][0]
's'
>>> items[1]
['YHOO', 'RHT', 'HPQ', 'GOOG', 'AIG', 'AAPL', 'AA']
>>> items[1][1]
'RHT'
>>> items[1][1][2]
'T'
>>> items[2]
[101, 102, 103]
>>> items[2][1]
102
>>>
```
매우 복잡한 리스트 구조를 만드는 것이 기술적으로는 가능하지만, 단순하게 만드는 것이 좋다. 리스트의 항목은 모두 같은 종류의 값으로 구성하는 것이 일반적이다. 예를 들어, 숫자만으로 리스트를 구성하거나, 텍스트 문자열만으로 리스트를 구성한다. 같은 리스트에 여러 종류의 데이터를 섞다보면 머리가 터질 수 있으니 될 수 있으면 피하자.

마지막 편집일시 : 2020년 10월 3일 6:01

1.6 파일 관리
대부분의 프로그램은 외부로부터 입력을 읽어야 한다. 이 섹션은 파일 액세스를 논의한다.

파일 입력과 출력
파일을 연다.

```
f = open('foo.txt', 'rt')       # 읽기를 위해 열기(텍스트)
g = open('bar.txt', 'wt')       # 쓰기를 위해 열기(텍스트)
```
모든 데이터를 읽는다.

```
data = f.read()
```

```
# 'maxbytes' 바이트까지만 읽음
data = f.read([maxbytes])
```
텍스트를 기록한다.

```
g.write('some text')
```
마쳤으면 파일을 닫는다.

```
f.close()
g.close()
```
파일을 열었으면 제대로 닫아야 하는데, 이 단계를 잊어버리기 쉽다. 따라서, 다음과 같이 with 문을 사용하면 좋다.

```
with open(filename, 'rt') as file:
    # `file` 파일을 사용
    ...
    # 명시적으로 닫지 않아도 된다
...문장
```
이렇게 하면 들여쓴 코드 블록에서 벗어날 때 파일이 자동으로 닫힌다.

파일 데이터를 읽는 일반적인 방법
파일 전체를 한번에 읽어 문자열로 처리한다.

```
with open('foo.txt', 'rt') as file:
    data = file.read()
    # `data`는 `foo.txt`의 텍스트 전체로 된 문자열이다
```
파일을 한 행씩 읽어 내려가기.

```
with open(filename, 'rt') as file:
    for line in file:
        # 행을 처리
```
파일에 쓰는 일반적인 방법
문자열 데이터를 기록한다.

```
with open('outfile', 'wt') as out:
    out.write('Hello World\n')
    ...
```
print 함수의 출력을 재지정(redirect)한다.

```
with open('outfile', 'wt') as out:
    print('Hello World', file=out)
    ...
```
연습 문제
이 연습 문제에서는 Data/portfolio.csv 파일을 사용한다. 이 파일은 주식의 포트폴리오에 대한 정보를 담은 행들의 리스트를 담고 있다. practical-python/Work/ 디렉터리에서 작업하는 것으로 가정한다. 확실하지 않으면 파이썬의 현재 작업 디렉터리(current working directory)를 확인해보자.

```
>>> import os
>>> os.getcwd()
'/Users/beazley/Desktop/practical-python/Work' # 출력은 환경에 따라 다르다
>>>
```

연습 문제 1.26: 파일 기초

먼저, 전체 파일을 한 번에 읽어 큰 문자열을 만들어 보자.

```
>>> with open('Data/portfolio.csv', 'rt') as f:
        data = f.read()

>>> data
'name,shares,price\n"AA",100,32.20\n"IBM",50,91.10\n"CAT",150,83.44\n"MSFT",200,51.23\n"GE",
95,40.37\n"MSFT",50,65.10\n"IBM",100,70.44\n'
>>> print(data)
name,shares,price
"AA",100,32.20
"IBM",50,91.10
"CAT",150,83.44
"MSFT",200,51.23
"GE",95,40.37
"MSFT",50,65.10
"IBM",100,70.44
>>>
```

위의 예에서 파이썬에 두 가지 출력 모드가 있음에 유의한다. 첫 번째 모드에서는 프롬프트에 data를 타이핑하면 따옴표와 이스케이프 코드가 포함된 원시 문자열이 나타난다. print(data)를 타이핑하면 실제 포매팅된 문자열 출력이 나타난다.

파일을 한 번에 읽는 것이 간단하기는 하지만 때에 따라서는 좋은 방법이 아닐 수도 있다. 특히 파일이 아주 크거나, 한 번에 처리하고 싶은 데이터가 포함된 경우가 그렇다.

파일을 행 단위로 읽으려면 다음과 같이 for 루프를 사용한다.

```
>>> with open('Data/portfolio.csv', 'rt') as f:
        for line in f:
            print(line, end='')

name,shares,price
"AA",100,32.20
"IBM",50,91.10
...
>>>
```

코드를 위와 같이 사용하면, 행을 차례로 읽다가 파일의 끝에 도달하면 루프가 종료된다.

특정한 상황에서, 텍스트의 단일 행을 읽거나 건너뛸 수 있다(예: 첫 행의 컬럼 헤더를 건너뛰고 싶을 것이다).

```
>>> f = open('Data/portfolio.csv', 'rt')
>>> headers = next(f)
>>> headers
'name,shares,price\n'
>>> for line in f:
    print(line, end='')

"AA",100,32.20
"IBM",50,91.10
...
>>> f.close()
>>>
```

next()는 파일에서 다음 번 행을 반환한다. 이것을 반복적으로 호출함으로써 행을 연속적으로 얻을 수 있다. 그렇지만, 알다시피 for 루프에서 데이터를 얻을 때 next()를 이미 사용한다. 따라서, 단일 행을 명시적으로 건너뛰거나 읽을 때 외에는 직접적으로 호출하지 않는다.

파일의 행을 읽었으면 문자열을 분할하는 것과 같은 처리를 할 수 있다. 예:

```
>>> f = open('Data/portfolio.csv', 'rt')
>>> headers = next(f).split(',')
>>> headers
['name', 'shares', 'price\n']
>>> for line in f:
    row = line.split(',')
    print(row)

['"AA"', '100', '32.20\n']
['"IBM"', '50', '91.10\n']
...
>>> f.close()
```

참고: 이 예에서는 with 문을 사용하지 않았으므로 f.close()를 명시적으로 호출했다.

연습 문제 1.27: 데이터 파일 읽기

파일을 읽는 방법을 알았으니 간단한 계산을 수행하는 프로그램을 작성하자.

portfolio.csv의 각 컬럼은 보유 종목의 이름, 주식 수, 매수가격에 해당한다. 이 파일을 열어 전체 행을 읽은 뒤, 포트폴리오의 전체 주식을 매수하는 데 드는 비용을 계산하는 pcost.py 프로그램을 작성한다.

힌트: 문자열을 정수로 변환하려면 int(s)를 사용한다. 문자열을 부동소수점으로 변환하려면 float(s)를 사용한다.

다음과 같이 출력하는 프로그램을 작성하라.

```
Total cost 44671.15
```

연습 문제 1.28: 다른 종류의 "파일"

일반 텍스트 파일이 아닌, gzip 압축된 데이터 파일을 읽고 싶으면 어떻게 해야 할까? 빌트인 open() 함수는 여기서 도움이 되지 않지만, 파이썬에는 gzip 압축된 파일을 읽을 수 있는 gzip 라이브러리 모듈이 있다.

한번 해 보자.

```
>>> import gzip
>>> with gzip.open('Data/portfolio.csv.gz', 'rt') as f:
    for line in f:
        print(line, end='')

... 출력을 살펴보라 ...
>>>
```

참고: 파일 모드를 'rt'로 지정하는 것이 중요하다. 이것을 잊으면 일반 텍스트 열이 아닌 바이트 열을 얻게 된다.

부연 설명: 판다스를 사용하면 되지 않나?
데이터 과학자라면 CSV 파일을 읽는 함수를 제공하는 판다스(Pandas) 같은 라이브러리를 사용하면 된다고 지적할 것이다. 맞는 말씀이다. 잘 작동한다. 하지만, 이 코스는 판다스를 배우는 코스가 아니다. 파일을 읽는 것은 CSV 파일이라는 국소적인 문제에 비해 좀 더 일반적인 문제다. 여기서 CSV 파일을 다루는 주된 이유는 그것이 잘 알려진 형식이고 직접 다루기 쉬운 편이므로, 그것을 처리하는 과정을 통해 파이썬의 기능을 소개하기 위함이다. 그러니, 판다스는 일터로 돌아가면 마음껏 써라. 이 코스에서는 표준 파이썬 기능만 사용한다.

마지막 편집일시 : 2020년 10월 4일 1:27

1.7 함수
프로그램이 커질수록 조직화가 필요해진다. 이 섹션은 함수와 라이브러리 모듈을 간략히 소개한다. 예외를 가지고 오류 처리를 하는 법도 소개한다.

커스텀 함수
재사용하고자 하는 코드를 위해 함수를 사용하라. 함수 정의는 다음과 같다.

```
def sumcount(n):
    '''
    정수 1부터 n까지의 합을 반환
    '''
    total = 0
    while n > 0:
        total += n
        n -= 1
    return total
```

다음과 같이 함수를 호출한다.

```
a = sumcount(100)
```

함수는 작업을 수행하고 결과를 반환하는 문장으로 이뤄진다. 함수가 값을 반환하려면 return 키워드를

사용해 명시해야 한다.

라이브러리 함수
파이썬에는 큰 규모의 표준 라이브러리(standard library)가 있다. import를 사용해 라이브러리 모듈에 액세스한다. 예:

```
import math
x = math.sqrt(10)

import urllib.request
u = urllib.request.urlopen('http://www.python.org/')
data = u.read()
```

라이브러리와 모듈에 대한 자세한 내용은 나중에 다룬다.

오류와 예외
함수는 오류를 예외로서 보고한다. 예외가 발생하면 함수의 실행이 중단되며 이를 처리하지 않으면 프로그램 전체가 중단될 수 있다.

파이썬 REPL에서 다음을 시도해보라.

```
>>> int('N/A')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'N/A'
>>>
```

실패를 일으킨 다른 함수 호출을 보여주는 트레이스백과 함께, 무슨 일이 일어났으며 어디서 오류가 발생했는지 설명하는 메시지가 나타난다. 이 정보는 디버깅에 도움이 된다.

예외를 붙잡아 처리하기
예외를 붙잡아 처리할 수 있다.

그렇게 하려면 try - except 문을 사용한다.

```
for line in f:
    fields = line.split()
    try:
        shares = int(fields[1])
    except ValueError:
        print("Couldn't parse", line)
    ...
```

ValueError라는 이름이 당신이 붙잡으려는 오류의 종류와 일치해야 한다.

어떤 종류의 오류가 발생할 수 있는지, 연산을 수행하기 전에 미리 정확하게 알기 어려울 때도 있다. 프로그램이 예상치 못하게 충돌한 이후에 예외 처리가 추가되곤 한다(예: "아차, 그 오류를 붙잡는 걸 깜빡했네. 처리하는 코드를 넣어야겠어!").

예외를 일으키기
예외를 일으키려면 raise 문을 사용한다.

```
raise RuntimeError('What a kerfuffle')
```

이렇게 하면 예외 트레이스백과 함께 프로그램이 중단된다. try-except 블록으로 붙잡지 않았다면 말이다.

```
% python3 foo.py
Traceback (most recent call last):
  File "foo.py", line 21, in <module>
    raise RuntimeError("What a kerfuffle")
RuntimeError: What a kerfuffle
```

연습 문제
연습 문제 1.29: 함수를 정의하기
다음과 같은 단순한 함수를 정의해보자.

```
>>> def greeting(name):
        'Issues a greeting'
        print('Hello', name)

>>> greeting('Guido')
Hello Guido
>>> greeting('Paula')
Hello Paula
>>>
```

함수의 첫 문장이 문자열이면 그것을 문서로 사용한다. help(greeting) 명령을 타이핑해 문서가 표시되는지 확인해 보라.

연습 문제 1.30: 스크립트를 함수로 탈바꿈
연습 문제 1.27의 pcost.py 프로그램 코드를 가져다가 portfolio_cost(filename) 함수를 만들어 보라. 이 함수는 파일명을 입력으로 받아 포트폴리오 데이터를 읽고, 포트폴리오의 총 비용을 부동소수점으로 반환한다.

함수를 사용하려면 프로그램을 다음과 같이 수정하자.

```
def portfolio_cost(filename):
    ...
    # 여기에 코드를 작성
    ...

cost = portfolio_cost('Data/portfolio.csv')
print('Total cost:', cost)
```

프로그램을 실행하면 이전과 똑같은 출력이 나타나야 한다. 프로그램을 실행한 후, 다음과 같이 타이핑해 함수를 상호작용적으로 호출할 수 있다.

```
bash $ python3 -i pcost.py
```

이렇게 하면 상호작용 모드에서 함수를 호출할 수 있다.

```
>>> portfolio_cost('Data/portfolio.csv')
44671.15
>>>
```

코드를 상호작용적으로 실험할 수 있어 테스트와 디버깅하기 좋다.

연습 문제 1.31: 오류 처리
필드가 누락된 파일을 가지고 함수를 실행하면 무슨 일이 일어나는가?

```
>>> portfolio_cost('Data/missing.csv')
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
    File "pcost.py", line 11, in portfolio_cost
    nshares     = int(fields[1])
ValueError: invalid literal for int() with base 10: ''
>>>
```

이 시점에 결정해야 한다. 입력 파일에서 잘못된 행을 제거하거나, 잘못된 행을 처리하도록 코드를 수정하는 것이다.

오류를 잡아서 경고 메시지를 프린트하고, 파일의 끝까지 계속 처리하게 pcost.py 프로그램을 수정해 보자.

연습 문제 1.32: 라이브러리 함수 사용하기
파이썬에는 유용한 함수의 방대한 표준 라이브러리가 있다. 여기서는 csv 모듈을 사용하면 좋다. CSV 데이터 파일을 다룰 일이 있을 때 사용해라. 다음과 같이 사용한다.

```
>>> import csv
>>> f = open('Data/portfolio.csv')
>>> rows = csv.reader(f)
>>> headers = next(rows)
>>> headers
['name', 'shares', 'price']
>>> for row in rows:
        print(row)

['AA', '100', '32.20']
['IBM', '50', '91.10']
['CAT', '150', '83.44']
['MSFT', '200', '51.23']
['GE', '95', '40.37']
['MSFT', '50', '65.10']
['IBM', '100', '70.44']
>>> f.close()
>>>
```

csv 모듈은 따옴표라든지, 콤마를 사용한 분할 등 저수준의 여러 가지 세부사항을 처리해준다. 위 출력

의 첫 번째 컬럼에서 큰 따옴표가 제거된 것을 볼 수 있다.

csv 모듈을 사용해 파싱을 하도록 pcost.py 프로그램을 수정한 다음, 이전의 예제를 실행해보라.

연습 문제 1.33: 명령행에서 읽기
pcost.py 프로그램은 입력 파일명을 하드코딩했다.

```
# pcost.py

def portfolio_cost(filename):
    ...
    # 여기에 코드를 작성
    ...

cost = portfolio_cost('Data/portfolio.csv')
print('Total cost:', cost)
```

배우고 테스트할 때는 괜찮지만 실제 프로그램에서는 그렇게 하면 안 된다.

스크립트의 인자로 파일명을 전달하라. 다음과 같이 프로그램을 바꿔라.

```
# pcost.py
import sys

def portfolio_cost(filename):
    ...
    # 여기에 코드를 작성
    ...

if len(sys.argv) == 2:
    filename = sys.argv[1]
else:
    filename = 'Data/portfolio.csv'

cost = portfolio_cost(filename)
print('Total cost:', cost)
```

명령행에서 전달받은 인자가 있는 경우 sys.argv 리스트에 들어 있다.

따라서, 이 프로그램은 다음과 같이 터미널에서 실행해야 한다.

예(유닉스 bash에서 실행):

```
bash % python3 pcost.py Data/portfolio.csv
Total cost: 44671.15
bash %
```

마지막 편집일시 : 2020년 10월 3일 5:41
---

2. 데이터로 작업하기

유용한 프로그램을 작성하려면 데이터를 다룰 수 있어야 한다. 이 섹션은 파이썬의 핵심 데이터 구조인 튜플, 리스트, 세트, 딕셔너리를 소개하고 데이터를 다루는 일반적인 방법을 논의한다. 이 섹션의 마지막 부분은 파이썬의 기본 객체 모델을 더 깊이 다룬다.

- 2.1 자료형과 자료 구조
- 2.2 컨테이너
- 2.3 출력 포매팅
- 2.4 시퀀스
- 2.5 collections 모듈
- 2.6 리스트 컴프리헨션(list comprehension)
- 2.7 객체 모델

2.1 자료형과 자료 구조

이 섹션에서는 튜플과 딕셔너리 자료형을 소개한다.

기본 자료형

파이썬에는 몇 가지 기본 자료형이 있다.

정수

부동소수점수

문자열(텍스트)

이런 것들을 도입에서 배웠다.

None 타입

email_address = None

None은 주로 선택값(optional value)이나 누락값(missing value)을 표시하는 데(placeholder) 사용한다. 조건문에서는 False로 평가한다.

```
if email_address:
    send_email(email_address, msg)
```

자료형

실제 프로그램에서는 더 복잡한 자료형을 사용한다. 주식 보유에 대한 정보의 예를 살펴 보자.

구글(GOOG) 주식을 490.10 달러에 100주 보유

이 "객체"는 세 부분으로 구성된다.

종목명 또는 심벌(문자열 "GOOG")

주식 수(정수 100)

가격(부동소수점수 490.10)

튜플(Tuple)

튜플은 함께 묶인 값의 컬렉션이다.

예:

```
s = ('GOOG', 100, 490.1)
```
다음과 같이 ()를 생략하기도 한다.

```
s = 'GOOG', 100, 490.1
```
특수한 사례(0-튜플, 1-튜플).

```
t = ()              # 빈 튜플
w = ('GOOG', )     # 항목이 한 개 있는 튜플
```
튜플은 주로 단순한 레코드 또는 자료 구조를 표현하는 데 사용한다. 일반적으로 여러 부분으로 된 단일 객체다. 좋은 비유: 튜플은 데이터베이스 테이블의 단일 행과 비슷하다.

튜플의 항목은 순서가 있다(배열과 비슷함).

```
s = ('GOOG', 100, 490.1)
name = s[0]                  # 'GOOG'
shares = s[1]                # 100
price = s[2]                 # 490.1
```
그러나, 내용을 수정할 수 없다.

```
>>> s[1] = 75
TypeError: object does not support item assignment
```
현재 튜플을 가지고 새로운 튜플을 만들 수는 있다.

```
s = (s[0], 75, s[2])
```
튜플로 묶기(packing)
튜플은 서로 관련된 항목들을 단일 엔티티(entity)로 묶는 역할을 한다.

```
s = ('GOOG', 100, 490.1)
```
튜플을 단일 객체로서 프로그램의 다른 부분으로 쉽게 전달할 수 있다.

튜플을 풀기(unpacking)
튜플을 전달받은 후에는 풀어서 각각의 변수를 사용할 수 있다.

```
name, shares, price = s
print('Cost', shares * price)
```
왼쪽의 변수 개수가 튜플 구조와 일치해야 한다.

```
name, shares = s      # 오류
Traceback (most recent call last):
...
ValueError: too many values to unpack
```
튜플 vs. 리스트
튜플은 읽기 전용 리스트처럼 보인다. 그러나, 튜플의 주된 용도는 여러 부분으로 구성된 단일 항목으로서 사용하는 것이다. 리스트는 고유한 항목들의 컬렉션으로 사용되며, 이때 모든 항목이 같은 타입인 경우가 많다.

```
record = ('GOOG', 100, 490.1)        # 포트폴리오의 한 레코드를 나타내는 튜플

symbols = [ 'GOOG', 'AAPL', 'IBM' ]  # 세 가지 주식 심벌을 나타내는 리스트
```

딕셔너리(Dictionary)

딕셔너리는 키(key)를 값(value)에 대응(mapping)시킨다. 해시 테이블(hash table) 또는 연관 배열(associative array)이라고도 부른다. 키는 값에 접근하는 인덱스 역할을 한다.

```
s = {
    'name': 'GOOG',
    'shares': 100,
    'price': 490.1
}
```

공통적인 연산

딕셔너리에서 값을 얻으려면 키 이름을 사용한다.

```
>>> print(s['name'], s['shares'])
GOOG 100
>>> s['price']
490.10
>>>
```

값을 추가하거나 수정할 때 키 이름을 사용한다.

```
>>> s['shares'] = 75
>>> s['date'] = '6/6/2007'
>>>
```

값을 삭제하려면 del 문을 사용한다.

```
>>> del s['date']
>>>
```

딕셔너리를 사용하는 이유

딕셔너리는 서로 다른 값이 많이 있고 그 값들을 수정 또는 조작해야 할 때 유용하다. 딕셔너리를 사용하면 코드의 가독성이 높아진다.

```
s['price']
# vs
s[2]
```

연습 문제

지난 몇 개의 연습 문제에서, 데이터파일 Data/portfolio.csv를 읽는 프로그램을 작성했다. csv 모듈을 사용하면 파일을 행 단위로 쉽게 읽을 수 있다.

```
>>> import csv
>>> f = open('Data/portfolio.csv')
>>> rows = csv.reader(f)
>>> next(rows)
['name', 'shares', 'price']
```

```
>>> row = next(rows)
>>> row
['AA', '100', '32.20']
>>>
```
파일을 읽는 것은 쉽지만, 데이터를 읽은 다음에는 그것을 가지고 더 많은 일을 하고 싶을 때가 있다. 예를 들어, 읽은 데이터를 저장하고 계산을 수행하고 싶을 것이다. 하지만, 원시 "행(row)" 데이터만으로는 작업하기에 충분하지 않다. 예를 들어, 간단한 수학 계산도 할 수 없다.

```
>>> row = ['AA', '100', '32.20']
>>> cost = row[1] * row[2]
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
TypeError: can't multiply sequence by non-int of type 'str'
>>>
```
원시 데이터를 좀 더 유용한 객체로 바꿔야 나중에 작업에 사용할 수 있다. 튜플이나 딕셔너리를 사용하면 된다.

연습 문제 2.1: 튜플
상호작용적인 프롬프트에서, 다음과 같이 위의 행을 나타내는 튜플을 생성하되, 숫자 컬럼은 적절한 숫자로 변환한다.

```
>>> t = (row[0], int(row[1]), float(row[2]))
>>> t
('AA', 100, 32.2)
>>>
```
이것을 사용해, 주식수와 가격을 곱한 총비용을 계산할 수 있다.

```
>>> cost = t[1] * t[2]
>>> cost
3220.0000000000005
>>>
```
파이썬에서 수학 계산이 틀린가? 답이 왜 3220.0000000000005라고 나올까?

이것은 컴퓨터의 부동소수점 하드웨어로 계산한 결과다. 십진수(decimal)를 십진법(Base-10)이 아니라 이진법(Base-2)으로 표현하기 때문에 이런 결과가 나왔다. 간단한 계산이라 할지라도, 십진수와 관련된 계산에는 작은 오류가 발생한다. 처음 본다면 놀라울 수 있겠지만, 이것은 정상적인 작동이다.

부동소수점 십진수를 사용하는 모든 프로그래밍 언어에서 발생하는 일이지만, 프린팅 과정에서 종종 숨겨진다. 예:

```
>>> print(f'{cost:0.2f}')
3220.00
>>>
```
튜플은 읽기 전용이다. 확인해 보고 싶으면 주식 수를 75로 바꿔보라.

```
>>> t[1] = 75
```

```
Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>>
```

튜플의 내용을 바꿀 수는 없지만, 항상 완전히 새로운 튜플을 생성해 기존 것을 대신할 수 있다.

```
>>> t = (t[0], 75, t[2])
>>> t
('AA', 75, 32.2)
>>>
```

기존 변수명을 재할당할 때마다 기존 값은 버려진다. 위의 할당문이 튜플을 수정하는 것처럼 보일 수 있지만, 실제로는 새로운 튜플을 생성하고 기존 것을 버렸다.

튜플은 패킹을 하고 변수로 언패킹하는 데 종종 사용된다. 다음과 같이 해 보자.

```
>>> name, shares, price = t
>>> name
'AA'
>>> shares
75
>>> price
32.2
>>>
```

위 변수들을 튜플로 패킹하자.

```
>>> t = (name, 2*shares, price)
>>> t
('AA', 150, 32.2)
>>>
```

연습 문제 2.2: 자료 구조로서의 딕셔너리

튜플 대신 딕셔너리를 생성할 수 있다.

```
>>> d = {
        'name' : row[0],
        'shares' : int(row[1]),
        'price'  : float(row[2])
    }
>>> d
{'name': 'AA', 'shares': 100, 'price': 32.2 }
>>>
```

총비용을 계산하자.

```
>>> cost = d['shares'] * d['price']
>>> cost
3220.0000000000005
>>>
```

위의 튜플에 대해서 했던 것과 같은 계산을 했다. 주식 수를 75로 변경한다.

```
>>> d['shares'] = 75
>>> d
{'name': 'AA', 'shares': 75, 'price': 32.2 }
>>>
```

튜플과 달리, 딕셔너리는 자유롭게 수정할 수 있다. 어트리뷰트를 추가하자.

```
>>> d['date'] = (6, 11, 2007)
>>> d['account'] = 12345
>>> d
{'name': 'AA', 'shares': 75, 'price':32.2, 'date': (6, 11, 2007), 'account': 12345}
>>>
```

연습 문제 2.3: 추가적인 딕셔너리 연산

딕셔너리를 리스트로 변환하면 전체 키를 얻는다.

```
>>> list(d)
['name', 'shares', 'price', 'date', 'account']
>>>
```

마찬가지로, 딕셔너리에 대해 for 문을 사용해 이터레이션을 해도 키를 얻는다,

```
>>> for k in d:
        print('k =', k)

k = name
k = shares
k = price
k = date
k = account
>>>
```

다음 코드는 키와 값을 동시에 조회한다.

```
>>> for k in d:
        print(k, '=', d[k])

name = AA
shares = 75
price = 32.2
date = (6, 11, 2007)
account = 12345
>>>
```

keys() 메서드를 사용해도 전체 키를 얻을 수 있다.

```
>>> keys = d.keys()
>>> keys
dict_keys(['name', 'shares', 'price', 'date', 'account'])
```

```
>>>
```
keys()는 특수한 dict_keys 객체를 반환한다는 점에서 독특하다.

이것은 원래 딕셔너리에 대한 오버레이로, 딕셔너리가 변경되더라도 항상 현재 키를 얻게 해준다. 다음과 같이 해 보자.

```
>>> del d['account']
>>> keys
dict_keys(['name', 'shares', 'price', 'date'])
>>>
```
d.keys()를 다시 호출하지 않았음에도 'account'가 keys에서 사라진 것에 유의하라.

키와 값을 함께 다루는 더욱 우아한 방법은 items() 메서드를 사용하는 것이다. 이 메서드를 사용해 (키, 값) 튜플을 얻을 수 있다.

```
>>> items = d.items()
>>> items
dict_items([('name', 'AA'), ('shares', 75), ('price', 32.2), ('date', (6, 11, 2007))])
>>> for k, v in d.items():
        print(k, '=', v)

name = AA
shares = 75
price = 32.2
date = (6, 11, 2007)
>>>
```
items 같은 튜플이 있으면 dict() 함수를 사용해 딕셔너리를 생성할 수 있다. 한번 해 보자.

```
>>> items
dict_items([('name', 'AA'), ('shares', 75), ('price', 32.2), ('date', (6, 11, 2007))])
>>> d = dict(items)
>>> d
{'name': 'AA', 'shares': 75, 'price':32.2, 'date': (6, 11, 2007)}
>>>
```
마지막 편집일시 : 2020년 7월 25일 10:44

2.2 컨테이너

이 섹션은 리스트, 딕셔너리, 세트를 논의한다.

개요

프로그램에서 많은 수의 객체를 다뤄야할 때가 있다.

주식 포트폴리오

주식 가격 테이블

세 가지 선택사항이 있다.

리스트(list): 순서가 유지되는(ordered) 데이터.
딕셔너리(dict): 순서가 없는(unordered) 데이터.
세트(set): 고유한 항목의 집합이며 순서가 없음.
컨테이너로서의 리스트
데이터의 순서가 중요할 때는 리스트를 사용하라. 리스트는 어떤 유형의 객체든 담을 수 있음을 명심하라. 예: 튜플의 리스트.

```
portfolio = [
    ('GOOG', 100, 490.1),
    ('IBM', 50, 91.3),
    ('CAT', 150, 83.44)
]

portfolio[0]            # ('GOOG', 100, 490.1)
portfolio[2]            # ('CAT', 150, 83.44)
```

리스트 생성(construction)
다음은 리스트에 항목을 하나하나 채워넣는 예이다.

```
records = []  # 빈 리스트로 초기화

# .append()를 사용해 항목을 추가
records.append(('GOOG', 100, 490.10))
records.append(('IBM', 50, 91.3))
...
```

다음은 파일에서 레코드를 읽어 리스트에 채우는 예이다.

```
records = []  # 빈 리스트로 초기화

with open('Data/portfolio.csv', 'rt') as f:
    next(f) # 헤더를 건너뜀
    for line in f:
        row = line.split(',')
        records.append((row[0], int(row[1]), float(row[2])))
```

컨테이너로서의 딕셔너리
딕셔너리는 빠른 임의 조회(키 이름을 사용)에 유용하다. 예: 주식 가격의 딕셔너리.

```
prices = {
   'GOOG': 513.25,
   'CAT': 87.22,
   'IBM': 93.37,
   'MSFT': 44.12
}
```

간단히 조회해보자.

```
>>> prices['IBM']
93.37
```

```
>>> prices['GOOG']
513.25
>>>
```

딕셔너리 생성

다음은 딕셔너리에 항목을 하나하나 채우는 예이다.

```
prices = {} # 빈 딕셔너리로 초기화

# 새 항목을 삽입
prices['GOOG'] = 513.25
prices['CAT'] = 87.22
prices['IBM'] = 93.37
```

다음은 파일 내용으로부터 딕셔너리를 채우는 예이다.

```
prices = {} # 빈 딕셔너리로 초기화

with open('Data/prices.csv', 'rt') as f:
    for line in f:
        row = line.split(',')
        prices[row[0]] = float(row[1])
```

참고: Data/prices.csv 파일을 가지고 이것을 시도하면 잘 되다가 마지막에 공백 행이 있어 충돌한다. 문제가 생기지 않게 코드를 수정해야 한다(연습 문제 2.6을 참조).

딕셔너리 조회

키가 존재하는지 테스트할 수 있다.

```
if key in d:
    # YES
else:
    # NO
```

존재하지 않는 값을 찾으려 할 경우 기본값을 제공할 수 있다.

```
name = d.get(key, default)
```

예:

```
>>> prices.get('IBM', 0.0)
93.37
>>> prices.get('SCOX', 0.0)
0.0
>>>
```

복합 키(composite keys)

파이썬에서는 어떤 타입의 값이든 딕셔너리의 키로 사용할 수 있다. 딕셔너리 키는 반드시 변경불가능한 타입이어야 한다. 튜플의 예:

```
holidays = {
  (1, 1) : 'New Years',
```

```
  (3, 14) : 'Pi day',
  (9, 13) : "Programmer's day",
}
```
액세스:

```
>>> holidays[3, 14]
'Pi day'
>>>
```
리스트, 세트, 다른 딕셔너리는 변경 가능(mutable)하므로 딕셔너리의 키로 사용할 수 없다.

세트
세트는 고유 항목의 모음이며 순서를 유지하지 않는다.

```
tech_stocks = { 'IBM','AAPL','MSFT' }
# 대체 구문
tech_stocks = set(['IBM', 'AAPL', 'MSFT'])
```
세트는 멤버십 테스트에 유용하다.

```
>>> tech_stocks
set(['AAPL', 'IBM', 'MSFT'])
>>> 'IBM' in tech_stocks
True
>>> 'FB' in tech_stocks
False
>>>
```
세트는 중복 제거에도 유용하다.

```
names = ['IBM', 'AAPL', 'GOOG', 'IBM', 'GOOG', 'YHOO']

unique = set(names)
# unique = set(['IBM', 'AAPL','GOOG','YHOO'])
```
그외의 집합(set) 연산:

```
names.add('CAT')          # 항목을 추가
names.remove('YHOO')      # 항목을 제거

s1 | s2                   # 합집합
s1 & s2                   # 교집합
s1 - s2                   # 차집합
```
연습 문제
이 연습 문제에서는 이 코스에서 계속 사용할 주된 프로그램 중 하나를 만들기 시작한다. Work/report.py 파일을 가지고 작업하라.

연습 문제 2.4: 튜플의 리스트
Data/portfolio.csv 파일에는 포트폴리오에 편입된 주식 목록이 있다. 연습 문제 1.30에서, 우리는 이 파일을 읽고 간단한 계산을 수행하는 portfolio_cost(filename) 함수를 작성했다.

코드가 다음과 같을 것이다.

```
# pcost.py

import csv

def portfolio_cost(filename):
    '''포트폴리오 파일의 총 비용(주식수*가격)을 계산'''
    total_cost = 0.0

    with open(filename, 'rt') as f:
        rows = csv.reader(f)
        headers = next(rows)
        for row in rows:
            nshares = int(row[1])
            price = float(row[2])
            total_cost += nshares * price
    return total_cost
```

이 코드를 참고해, 새로운 report.py 파일을 생성하라. 그 파일에서, 주어진 포트폴리오 파일을 열고 내용을 읽어 튜플의 리스트를 생성하는 read_portfolio(filename) 함수를 정의한다. 이를 위해, 위의 코드를 약간 수정해야 한다.

먼저, total_cost = 0를 정의하는 대신, 변수를 만들어 빈 리스트로 초기화한다. 예:

```
portfolio = []
```

다음으로, 총 비용을 계산하는 대신, 이전 연습 문제에서 했던 것과 똑같은 방법으로 각 행을 튜플로 바꿔서 리스트에 추가한다. 예:

```
for row in rows:
    holding = (row[0], int(row[1]), float(row[2]))
    portfolio.append(holding)
```

끝으로, 결과 portfolio 리스트를 반환한다.

작성한 함수를 상호작용적으로 실험해보라(그렇게 하려면 인터프리터에서 report.py부터 실행해야 한다).

힌트: 파일을 터미널에서 실행할 때는 -i를 사용한다.

```
>>> portfolio = read_portfolio('Data/portfolio.csv')
>>> portfolio
[('AA', 100, 32.2), ('IBM', 50, 91.1), ('CAT', 150, 83.44), ('MSFT', 200, 51.23),
    ('GE', 95, 40.37), ('MSFT', 50, 65.1), ('IBM', 100, 70.44)]
>>>
>>> portfolio[0]
('AA', 100, 32.2)
>>> portfolio[1]
```

```
('IBM', 50, 91.1)
>>> portfolio[1][1]
50
>>> total = 0.0
>>> for s in portfolio:
        total += s[1] * s[2]

>>> print(total)
44671.15
>>>
```

방금 생성한 튜플의 리스트는 2차원 배열과 매우 비슷하다. 예를 들어, portfolio[row][column]와 같이 조회해 특정 row와 column에 액세스할 수 있다(row와 column은 정수).

따라서, 다음과 같은 구문을 사용해 마지막 for 루프를 재작성할 수도 있다.

```
>>> total = 0.0
>>> for name, shares, price in portfolio:
            total += shares*price

>>> print(total)
44671.15
>>>
```

연습 문제 2.5: 딕셔너리의 리스트

연습 문제 2.4에서 작성한 함수를 수정해, 포트폴리오에 있는 주식을 튜플 대신 딕셔너리로 나타내게 해 보자. 이 딕셔너리에서 입력 파일의 컬럼들을 나타내기 위해, 각각 "name", "shares", "price"라는 필드명을 사용한다.

이 함수를 연습 문제 2.4에서 했던 것과 같은 방식으로 실험해 보라.

```
>>> portfolio = read_portfolio('Data/portfolio.csv')
>>> portfolio
[{'name': 'AA', 'shares': 100, 'price': 32.2}, {'name': 'IBM', 'shares': 50, 'price': 91.1},
    {'name': 'CAT', 'shares': 150, 'price': 83.44}, {'name': 'MSFT', 'shares': 200, 'price': 51.23},
    {'name': 'GE', 'shares': 95, 'price': 40.37}, {'name': 'MSFT', 'shares': 50, 'price': 65.1},
    {'name': 'IBM', 'shares': 100, 'price': 70.44}]
>>> portfolio[0]
{'name': 'AA', 'shares': 100, 'price': 32.2}
>>> portfolio[1]
{'name': 'IBM', 'shares': 50, 'price': 91.1}
>>> portfolio[1]['shares']
50
>>> total = 0.0
>>> for s in portfolio:
        total += s['shares']*s['price']
```

```
>>> print(total)
44671.15
>>>
```

여기서 각 항목의 필드에 접근할 때는 숫자로 된 컬럼 번호 대신 키 이름을 사용한 것을 볼 수 있다. 나중에 코드를 읽기 수월하므로 이 방법을 선호한다.

큰 딕셔너리와 리스트를 조회하는 것은 지저분해질 수 있다. 디버깅을 위해 출력을 정돈하려면 pprint 함수를 사용하는 것을 고려하라.

```
>>> from pprint import pprint
>>> pprint(portfolio)
[{'name': 'AA', 'price': 32.2, 'shares': 100},
    {'name': 'IBM', 'price': 91.1, 'shares': 50},
    {'name': 'CAT', 'price': 83.44, 'shares': 150},
    {'name': 'MSFT', 'price': 51.23, 'shares': 200},
    {'name': 'GE', 'price': 40.37, 'shares': 95},
    {'name': 'MSFT', 'price': 65.1, 'shares': 50},
    {'name': 'IBM', 'price': 70.44, 'shares': 100}]
>>>
```

연습 문제 2.6: 컨테이너로서의 딕셔너리

딕셔너리는 항목을 정수 이외의 인덱스를 사용해 조회하고자 할 때 유용하다. 파이썬 셸에서 딕셔너리를 가지고 놀아보자.

```
>>> prices = { }
>>> prices['IBM'] = 92.45
>>> prices['MSFT'] = 45.12
>>> prices
... 결과를 살펴 보라 ...
>>> prices['IBM']
92.45
>>> prices['AAPL']
... 결과를 살펴 보라 ...
>>> 'AAPL' in prices
False
>>>
```

Data/prices.csv 파일은 여러 행에 걸쳐 주식 가격을 담고 있다. 파일 내용은 다음과 같다.

```
"AA",9.22
"AXP",24.85
"BA",44.85
"BAC",11.27
"C",3.72
...
```

이와 같은 가격 정보를 읽어 딕셔너리를 생성하는 read_prices(filename) 함수를 작성하라. 딕셔너리 키는 종목명으로, 값은 주식 가격으로 한다.

위에서 했던 것처럼 빈 딕셔너리를 먼저 만들고 나서 값을 삽입하라. 이번에는 파일에서 값을 읽는다는 점이 다르다.

이 자료 구조를 사용하면 종목명을 재빨리 조회할 수 있다.

몇 가지 팁이 있다. 먼저, 앞에서 한 것처럼 csv 모듈을 사용하라. 바퀴를 재발명할 필요는 없다.

```
>>> import csv
>>> f = open('Data/prices.csv', 'r')
>>> rows = csv.reader(f)
>>> for row in rows:
        print(row)


['AA', '9.22']
['AXP', '24.85']
...
[]
>>>
```

또 다른 복잡성은 Data/prices.csv 파일에 공백 행이 있을 수 있다는 점이다. 위에서 데이터의 마지막 행은 빈 리스트다. 즉, 그 행에는 아무 데이터도 표시하지 않았다.

그것 때문에 프로그램이 예외를 내며 죽어버릴 가능성이 있다. try와 except 문을 사용해 예외를 처리하자. 생각할 거리: 예외 처리 대신 if 문을 사용해 나쁜 데이터를 차단하는 것이 나을까?

read_prices() 함수를 작성했으면, 제대로 작동하는지 테스트해보라.

```
>>> prices = read_prices('Data/prices.csv')
>>> prices['IBM']
106.28
>>> prices['MSFT']
20.89
>>>
```

연습 문제 2.7: 은퇴해도 되는지 알아보기
지금까지 배운 것을 모두 활용해 report.py 프로그램을 만들어 보자. 연습 문제 2.5의 주식 목록과 연습 문제 2.6의 가격 딕셔너리를 받아서 포트폴리오의 현재 가치와 손익을 계산한다.

마지막 편집일시 : 2020년 10월 3일 5:43

2.3 포매팅
데이터를 가지고 작업할 때 구조화된 출력(테이블 등)을 생성하고자 할 때가 있다. 예:

```
      Name      Shares        Price
----------  ----------  -----------
        AA         100        32.20
       IBM          50        91.10
```

```
        CAT          150         83.44
       MSFT          200         51.23
         GE           95         40.37
       MSFT           50         65.10
        IBM          100         70.44
```

문자열 포매팅

파이썬 3.6 이상에서는 문자열 포매팅에 f 문자열(f-string)을 사용할 수 있다.

```
>>> name = 'IBM'
>>> shares = 100
>>> price = 91.1
>>> f'{name:>10s} {shares:>10d} {price:>10.2f}'
'       IBM        100      91.10'
>>>
```

{표현식:포맷} 부분이 대체된다.

이것을 다음과 같이 print 문과 함께 사용한다.

```
print(f'{name:>10s} {shares:>10d} {price:>10.2f}')
```

포맷 코드

포맷 코드({} 내의 : 이후)는 C의 printf()와 비슷하다. 일반적인 코드는 다음과 같다.

```
d        십진 정수
b        이진 정수
x        16진 정수
f        부동소수점수 [-]m.dddddd
e        부동소수점수 [-]m.dddddde+-xx
g        E 표기를 선택적으로 사용하는 부동소수점
s        문자열
c        문자(정수로부터)
```

필드 폭과 정밀도를 조정하는 공통 수정자. 이것은 부분적인 목록이다.

```
:>10d    정수를 10자리 필드에 오른쪽 정렬
:<10d    정수를 10자리 필드에 왼쪽 정렬
:^10d    정수를 10자리 필드에 가운데 정렬
:0.2f    부동소수점수를 두 자리 정밀도로 나타냄
```

딕셔너리 포매팅

format_map() 메서드를 사용해 값들의 딕셔너리에 문자열 포매팅을 적용할 수 있다.

```
>>> s = {
    'name': 'IBM',
    'shares': 100,
    'price': 91.1
}
>>> '{name:>10s} {shares:10d} {price:10.2f}'.format_map(s)
'       IBM        100      91.10'
```

```
>>>
```
그것은 f 문자열과 같은 코드를 사용하되, 제공된 딕셔너리로부터 값을 취한다.

format() 메서드

format() 메서드는 인자 또는 키워드 인자에 대한 포매팅을 적용할 수 있다.

```
>>> '{name:>10s} {shares:10d} {price:10.2f}'.format(name='IBM', shares=100, price=91.1)
'       IBM        100      91.10'
>>> '{:10s} {:10d} {:10.2f}'.format('IBM', 100, 91.1)
'       IBM        100      91.10'
>>>
```
솔직히, format()은 다소 장황하다. 나는 f 문자열을 선호한다.

C 스타일 포매팅

포매팅 연산자 %를 사용할 수도 있다.

```
>>> 'The value is %d' % 3
'The value is 3'
>>> '%5d %-5d %10d' % (3,4,5)
'    3 4                5'
>>> '%0.2f' % (3.1415926,)
'3.14'
```
이때 오른쪽에는 단일 항목 또는 튜플이 있어야 한다. 포맷 코드는 C의 printf()와 유사하다.

참고: 이것은 바이트 문자열에 대해서만 사용 가능한 포매팅이다.

```
>>> b'%s has %n messages' % (b'Dave', 37)
b'Dave has 37 messages'
>>>
```
연습 문제

연습 문제 2.8: 숫자를 포맷하는 법

숫자를 프린팅할 때의 공통적인 문제는 십진수의 자릿수를 지정하는 것이다. 이것을 해결하는 한 가지 방법은 f 문자열을 사용하는 것이다. 다음 예를 시도해 보자.

```
>>> value = 42863.1
>>> print(value)
42863.1
>>> print(f'{value:0.4f}')
42863.1000
>>> print(f'{value:>16.2f}')
        42863.10
>>> print(f'{value:<16.2f}')
42863.10
>>> print(f'{value:*>16,.2f}')
*******42,863.10
>>>
```

f 문자열에서 사용하는 포매팅 코드에 대한 완전한 문서는 여기서 찾을 수 있다. 문자열의 % 연산자를 사용해 포매팅을 수행할 수도 있다.

```
>>> print('%0.4f' % value)
42863.1000
>>> print('%16.2f' % value)
        42863.10
>>>
```

%과 함께 사용되는 여러 가지 코드에 대한 문서를 여기서 찾을 수 있다.

문자열 포매팅을 print와 함께 사용하는 것이 일반적이기는 하지만, 반드시 그렇게 해야 하는 것은 아니다. 포매팅한 문자열을 저장할 수도 있다. 다음과 같이 변수에 할당하면 된다.

```
>>> f = '%0.4f' % value
>>> f
'42863.1000'
>>>
```

연습 문제 2.9: 데이터 수집하기

연습 문제 2.7에서 주식 포트폴리오의 손익을 계산하는 report.py라는 프로그램을 작성했다. 이 연습 문제에서는 그 프로그램을 수정해 다음과 같은 테이블을 생성한다.

Name	Shares	Price	Change
AA	100	9.22	-22.98
IBM	50	106.28	15.18
CAT	150	35.46	-47.98
MSFT	200	20.89	-30.34
GE	95	13.48	-26.89
MSFT	50	20.89	-44.21
IBM	100	106.28	35.84

이 보고서에서 "Price"는 주식의 현재 가격이며 "Change"는 처음 매수한 가격과의 차이를 나타낸다.

위와 같은 보고서를 생성하려면, 우선 테이블에 있는 데이터를 모두 수집해야 한다. 종목 리스트와 가격 딕셔너리를 입력으로 취해서 위의 테이블과 같은 행을 포함하는 튜플의 리스트를 반환하는 make_report() 함수를 작성하자.

이 함수를 report.py 파일에 추가한다. 상호작용하면 다음과 같이 작동해야 한다.

```
>>> portfolio = read_portfolio('Data/portfolio.csv')
>>> prices = read_prices('Data/prices.csv')
>>> report = make_report(portfolio, prices)
>>> for r in report:
        print(r)

('AA', 100, 9.22, -22.980000000000004)
('IBM', 50, 106.28, 15.180000000000007)
```

```
('CAT', 150, 35.46, -47.98)
('MSFT', 200, 20.89, -30.339999999999996)
('GE', 95, 13.48, -26.889999999999997)
...
>>>
```

연습 문제 2.10: 포매팅된 테이블을 프린팅하기
연습 문제 2.9의 for 루프에서 print 문을 수정해 튜플을 포매팅한다.

```
>>> for r in report:
        print('%10s %10d %10.2f %10.2f' % r)

        AA        100      9.22     -22.98
       IBM         50    106.28      15.18
       CAT        150     35.46     -47.98
      MSFT        200     20.89     -30.34
...
>>>
```

값을 확장해 f 문자열을 사용할 수도 있다. 예:

```
>>> for name, shares, price, change in report:
        print(f'{name:>10s} {shares:>10d} {price:>10.2f} {change:>10.2f}')

        AA        100      9.22     -22.98
       IBM         50    106.28      15.18
       CAT        150     35.46     -47.98
      MSFT        200     20.89     -30.34
...
>>>
```

위 문장을 report.py 프로그램에 추가하자. make_report() 함수의 출력을 사용해 보기 좋게 포매팅된 테이블을 프린트하게 프로그램을 작성하라.

연습 문제 2.11: 헤더 추가하기
다음과 같이 헤더의 튜플이 있다고 하자.

```
headers = ('Name', 'Shares', 'Price', 'Change')
```

위와 같은 헤더의 튜플을 가지고 문자열을 생성하는 코드를 프로그램에 추가하라. 헤더 폭은 10자리, 헤더 이름은 오른쪽 정렬하고, 각 필드는 공백 한 개로 구분한다.

```
'      Name     Shares      Price     Change'
```

헤더를 가지고, 다음과 같이 헤더와 데이터를 구분하는 문자열을 생성하는 코드를 작성하라. 이 문자열은 각 필드명 아래에 "-" 문자를 여러 개 사용한 것이다. 예:

```
'---------- ---------- ---------- -----------'
```

완료된 프로그램은 이 연습의 처음에 보인 것과 같은 테이블을 생성해야 한다.

```
      Name     Shares      Price     Change
```

```
---------- ---------- ---------- ----------
        AA        100       9.22     -22.98
       IBM         50     106.28      15.18
       CAT        150      35.46     -47.98
      MSFT        200      20.89     -30.34
        GE         95      13.48     -26.89
      MSFT         50      20.89     -44.21
       IBM        100     106.28      35.84
```

연습 문제 2.12: 포매팅 도전

가격에 통화 기호($)를 포함해 다음과 같이 출력하게 수정해 보라.

```
      Name     Shares      Price     Change
---------- ---------- ---------- ----------
        AA        100      $9.22     -22.98
       IBM         50   $106.28      15.18
       CAT        150     $35.46     -47.98
      MSFT        200     $20.89     -30.34
        GE         95     $13.48     -26.89
      MSFT         50     $20.89     -44.21
       IBM        100   $106.28      35.84
```

마지막 편집일시 : 2020년 7월 31일 4:13 오

2.4 시퀀스

시퀀스 자료형

파이썬에는 세 가지 시퀀스(sequence) 자료형이 있다.

문자열: 'Hello'. 문자열(string)은 문자(character)들의 시퀀스다.

리스트: [1, 4, 5].

튜플: ('GOOG', 100, 490.1).

모든 시퀀스는 순서가 유지되고, 정수로 인덱싱하며, 길이가 있다.

```
a = 'Hello'                 # 문자열
b = [1, 4, 5]               # 리스트
c = ('GOOG', 100, 490.1)  # 튜플

# 인덱싱된 순서
a[0]                        # 'H'
b[-1]                       # 5
c[1]                        # 100

# 시퀀스의 길이
len(a)                      # 5
len(b)                      # 3
len(c)                      # 3
```

시퀀스를 복제할 수 있다(s * n).

```
>>> a = 'Hello'
>>> a * 3
'HelloHelloHello'
>>> b = [1, 2, 3]
>>> b * 2
[1, 2, 3, 1, 2, 3]
>>>
```

같은 타입의 시퀀스끼리 이어붙일 수 있다(s + t).

```
>>> a = (1, 2, 3)
>>> b = (4, 5)
>>> a + b
(1, 2, 3, 4, 5)
>>>
>>> c = [1, 5]
>>> a + c
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate tuple (not "list") to tuple
```

슬라이싱(Slicing)

시퀀스의 일부(subsequence)를 취하는 것을 슬라이싱이라 한다. s[start:end] 구문을 사용한다. start와 end는 얻고자 하는 서브시퀀스의 인덱스다.

```
a = [0,1,2,3,4,5,6,7,8]

a[2:5]     # [2,3,4]
a[-5:]     # [4,5,6,7,8]
a[:3]      # [0,1,2]
```

start와 end 인덱스는 반드시 정수여야 한다.

슬라이스는 end 값을 포함하지 않는다. 수학에서 한쪽만 열린 구간(interval)과 비슷하다.

인덱스를 생략하면 리스트의 시작이나 끝을 기본값으로 사용한다.

슬라이스 재할당(re-assignment)

리스트에서 슬라이스를 다시 할당하거나 삭제할 수 있다.

```
# 재할당
a = [0,1,2,3,4,5,6,7,8]
a[2:4] = [10,11,12]        # [0,1,10,11,12,4,5,6,7,8]
```

참고: 재할당된 슬라이스는 길이가 똑같을 필요가 없다.

```
# 삭제
a = [0,1,2,3,4,5,6,7,8]
del a[2:4]                 # [0,1,4,5,6,7,8]
```

시퀀스 축소(Reduction)

시퀀스를 단일 값으로 줄이는 몇 가지 함수가 있다.

```
>>> s = [1, 2, 3, 4]
```

```
>>> sum(s)
10
>>> min(s)
1
>>> max(s)
4
>>> t = ['Hello', 'World']
>>> max(t)
'World'
>>>
```

시퀀스에 대한 이터레이션

for 루프는 시퀀스의 원소에 대해 이터레이션을 수행한다.

```
>>> s = [1, 4, 9, 16]
>>> for i in s:
...     print(i)
...
1
4
9
16
>>>
```

각 이터레이션에서 새 항목을 얻어 작업한다. 이러한 새 값이 이터레이션 변수에 할당된다. 이 예에서 이터레이션 변수는 x다.

```
for x in s:          # `x`는 이터레이션 변수
    ...문장
```

각 이터레이션에서, 이터레이션 변수에 있는 이전 값을 덮어쓴다. 루프가 끝난 후 변수는 최종 값을 갖는다.

break 문

break 문을 사용해 루프에서 일찍 빠져나갈 수 있다.

```
for name in namelist:
    if name == 'Jake':
        break
    ...
    ...
문장
```

문장의 실행을 break하면 루프에서 빠져나가서 다음 문장을 수행한다. break 문은 가장 안쪽 루프에만 적용된다. 만약 이 루프가 다른 루프 안에 있다면, 바깥쪽 루프는 빠져나지 않는다.

continue 문

한 원소를 건너뛰고 다음으로 가려면 continue 문을 사용한다.

```
for line in lines:
```

```
    if line == '\n':     # 공백 행을 건너뜀
        continue
    # 나머지 문장들
    ...
```
현재 항목에 관심이 없거나 처리 과정에서 무시해야 할 때 유용하다.

정수에 대해 루핑
카운트를 하고 싶으면 range()를 써라.

```
for i in range(100):
    # i = 0,1,...,99
```
구문은 range([start,] end [,step])이다.

```
for i in range(100):
    # i = 0,1,...,99
for j in range(10,20):
    # j = 10,11,..., 19
for k in range(10,50,2):
    # k = 10,12,...,48
    # step이 1이 아니라 2일 때 어떤 식으로 카운트하는지 눈여겨 보라.
```
end 값은 포함하지 않는다. 이는 슬라이스의 작동과 같다.
start는 선택적이고, 기본값이 0이다.
step도 선택적이며, 기본값이 1이다.
range()는 필요에 따라 값을 계산한다. 실제로 많은 숫자를 저장하지는 않는다.
enumerate() 함수
enumerate 함수는 이터레이션에 카운터 값을 추가한다.

```
names = ['Elwood', 'Jake', 'Curtis']
for i, name in enumerate(names):
    # i = 0, name = 'Elwood'를 가지고 루프
    # i = 1, name = 'Jake'
    # i = 2, name = 'Curtis'
```
일반적 형태는 enumerate(시퀀스 [, start = 0])이다. start는 선택적이다. enumerate()를 사용하는 좋은 예는 파일을 읽으면서 행 번호를 추적하는 것이다.

```
with open(filename) as f:
    for lineno, line in enumerate(f, start=1):
        ...
```
enumerate는 다음 코드를 축약하는 셈이다.

```
i = 0
for x in s:
    문장
    i += 1
```
enumerate를 사용하면 타이핑을 적게 할 수 있고 속도도 약간 더 빠르다.

for와 튜플
여러 개의 이터레이션 변수(iteration variable)를 사용해 루핑을 할 수 있다.

```
points = [
  (1, 4),(10, 40),(23, 14),(5, 6),(7, 8)
]
for x, y in points:
    # x = 1, y = 4를 가지고 루핑
    #             x = 10, y = 40
    #             x = 23, y = 14
    #             ...
```

여러 개의 변수를 사용할 때, 각 튜플은 이터레이션 변수의 세트로 언팩(unpacked)된다. 변수의 개수는 각 튜플의 항목 수와 일치해야 한다.

zip() 함수
zip 함수는 여러 시퀀스를 결합해 이터레이터(iterator)를 만든다.

```
columns = ['name', 'shares', 'price']
values = ['GOOG', 100, 490.1 ]
pairs = zip(columns, values)
# ('name','GOOG'), ('shares',100), ('price',490.1)
```

결과를 얻으려면 반드시 이터레이션을 해야 한다. 앞서 본 것처럼 여러 개의 변수를 사용해 튜플을 언팩할 수 있다.

```
for column, value in pairs:
    ...
```

zip의 일반적인 용도는 딕셔너리를 생성하기 위한 키/값 쌍을 생성하는 것이다.

```
d = dict(zip(columns, values))
```

연습 문제
연습 문제 2.13: 카운팅
기본적인 카운팅을 해 보자.

```
>>> for n in range(10):              # 0 ... 9를 카운트
        print(n, end=' ')

0 1 2 3 4 5 6 7 8 9
>>> for n in range(10,0,-1):         # 10 ... 1을 카운트
        print(n, end=' ')

10 9 8 7 6 5 4 3 2 1
>>> for n in range(0,10,2):          # 0, 2, ... 8을 카운트
        print(n, end=' ')

0 2 4 6 8
>>>
```

연습 문제 2.14: 더 많은 시퀀스 연산
시퀀스 축소 연산을 상호작용적으로 실험해 보자.

```
>>> data = [4, 9, 1, 25, 16, 100, 49]
>>> min(data)
1
>>> max(data)
100
>>> sum(data)
204
>>>
```

데이터에 대해 루핑을 해 보자.

```
>>> for x in data:
        print(x)

4
9
...
>>> for n, x in enumerate(data):
        print(n, x)

0 4
1 9
2 1
...
>>>
```

파이썬 초보자들은 for 문, len(), range()를 사용해 C 프로그램처럼 보이는 끔직한 코드를 작성하곤 한다.

```
>>> for n in range(len(data)):
        print(data[n])

4
9
1
...
>>>
```

이러지 마라! 가독성도 떨어지는데다 메모리를 비효율적으로 사용하며 실행 속도도 느려진다. 평범한 for 루프를 써서 데이터를 이터레이트하라. 인덱스를 써야 할 합당한 이유가 있을 때는 enumerate()를 사용하라.

연습 문제 2.15: 실용적인 enumerate() 예제
주식 포트폴리오 데이터가 있는 Data/missing.csv의 일부 행에는 데이터가 누락되어 있다. enumerate()를 사용해 pcost.py 프로그램이 잘못된 입력을 만나면 행 번호와 경고 메시지를 프린트하게 해 보라.

```
>>> cost = portfolio_cost('Data/missing.csv')
Row 4: Couldn't convert: ['MSFT', '', '51.23']
Row 7: Couldn't convert: ['IBM', '', '70.44']
>>>
```

이렇게 하려면 코드를 몇 군데 수정해야 한다.

```
...
for rowno, row in enumerate(rows, start=1):
    try:
        ...
    except ValueError:
        print(f'Row {rowno}: Bad row: {row}')
```

연습 문제 2.16: zip() 함수 사용하기

Data/portfolio.csv 파일의 첫 행에는 컬럼 헤더가 있다. 이전 코드에서는 그 부분을 버렸다.

```
>>> f = open('Data/portfolio.csv')
>>> rows = csv.reader(f)
>>> headers = next(rows)
>>> headers
['name', 'shares', 'price']
>>>
```

헤더를 잘 활용할 수 있다면 좋지 않을까? zip() 함수를 이용해 보자. 먼저, 파일 헤더를 데이터 행과 짝을 짓는다.

```
>>> row = next(rows)
>>> row
['AA', '100', '32.20']
>>> list(zip(headers, row))
[ ('name', 'AA'), ('shares', '100'), ('price', '32.20') ]
>>>
```

zip()이 어떻게 컬럼 헤더와 컬럼 값의 짝을 만드는지 눈여겨 보라. 그 결과를 리스트로 바꾸기 위해 list()를 사용했으므로 확인할 수 있다. zip()으로 이터레이터를 생성하여 for 루프에서 소비하는 것이 보통이다.

이러한 짝 짓기는 딕셔너리를 만들어내는 중간 단계다. 이번에는 이렇게 해 보자.

```
>>> record = dict(zip(headers, row))
>>> record
{'price': '32.20', 'name': 'AA', 'shares': '100'}
>>>
```

이러한 변환 방식은 데이터 파일을 많이 처리할 때 매우 유용한 트릭이다. 예를 들어, pcost.py 프로그램이 다양한 입력을 처리하게 하되, 종목명, 주식 수, 가격이 나타나는 실제 컬럼 번호를 참고하고 싶다고 하자.

pcost.py의 portfolio_cost() 함수를 다음과 같이 수정한다.

```
# pcost.py

def portfolio_cost(filename):
    ...
        for rowno, row in enumerate(rows, start=1):
            record = dict(zip(headers, row))
            try:
                nshares = int(record['shares'])
                price = float(record['price'])
                total_cost += nshares * price
            # 위의 int()와 float() 변환 시 발생하는 오류를 처리
            except ValueError:
                print(f'Row {rowno}: Bad row: {row}')
        ...
```

이제 완전히 다른 데이터 파일을 함수에 적용해 보자. Data/portfoliodate.csv는 다음과 같다.

```
name,date,time,shares,price
"AA","6/11/2007","9:50am",100,32.20
"IBM","5/13/2007","4:20pm",50,91.10
"CAT","9/23/2006","1:30pm",150,83.44
"MSFT","5/17/2007","10:30am",200,51.23
"GE","2/1/2006","10:45am",95,40.37
"MSFT","10/31/2006","12:05pm",50,65.10
"IBM","7/9/2006","3:15pm",100,70.44
>>> portfolio_cost('Data/portfoliodate.csv')
44671.15
>>>
```

데이터 파일이 앞에서 사용한 것과 완전히 달라지더라도 프로그램이 여전히 작동할 것이다. 멋지지 않은가!

코드를 많이 바꾸지 않았지만 그 효과는 상당하다. 새로운 버전의 portfolio_cost() 함수에서는 고정된 파일 형식을 하드코딩하는 대신, 어느 CSV 파일이 들어오더라도 원하는 값을 골라낼 수 있게 됐다. 필요한 컬럼이 파일에 있기만 하면 코드는 잘 작동한다.

섹션 2.3에서 작성한 report.py 프로그램에, 컬럼 헤더를 선택하는 기술을 적용해보자.

Data/portfoliodate.csv 파일에 대해 report.py 프로그램을 실행해, 이전과 같은 답을 만들어내는지 확인해 보라.

연습 문제 2.17: 딕셔너리 뒤집기
딕셔너리는 키를 값에 매팽한다. 다음은 주식 가격 딕셔너리의 예다.

```
>>> prices = {
        'GOOG' : 490.1,
        'AA' : 23.45,
```

```
        'IBM' : 91.1,
        'MSFT' : 34.23
    }
>>>
```

items() 메서드를 사용하면 (키, 값) 쌍을 얻는다.

```
>>> prices.items()
dict_items([('GOOG', 490.1), ('AA', 23.45), ('IBM', 91.1), ('MSFT', 34.23)])
>>>
```

그런데, 만약 (값, 키)의 쌍이 필요하다면 어떻게 해야 할까? 힌트: zip()을 사용한다.

```
>>> pricelist = list(zip(prices.values(),prices.keys()))
>>> pricelist
[(490.1, 'GOOG'), (23.45, 'AA'), (91.1, 'IBM'), (34.23, 'MSFT')]
>>>
```

이렇게 해야 할 이유가 있을까? 딕셔너리 데이터에 대해 특정한 데이터 처리를 할 수 있다는 점을 한 가지 이유로 들 수 있다.

```
>>> min(pricelist)
(23.45, 'AA')
>>> max(pricelist)
(490.1, 'GOOG')
>>> sorted(pricelist)
[(23.45, 'AA'), (34.23, 'MSFT'), (91.1, 'IBM'), (490.1, 'GOOG')]
>>>
```

또한 이것은 튜플의 중요 기능을 보여준다. 튜플을 비교하면, 첫 번째 항목부터 원소 하나하나를 비교하게 된다. 이는 문자열의 문자 하나하나를 비교하는 것과 비슷하다.

zip()은 이와 같이 서로 다른 자리의 데이터를 가지고 짝을 지어야 하는 상황에 종종 사용된다. 예를 들어, 명명된 값의 딕셔너리를 만들기 위해, 컬럼명과 컬럼값의 쌍을 만들 수 있다.

참고로, zip()으로 1:1 쌍만 만들 수 있는 것이 아니다. 여러 개의 입력 리스트를 가지고도 그런 일을 할 수 있다.

```
>>> a = [1, 2, 3, 4]
>>> b = ['w', 'x', 'y', 'z']
>>> c = [0.2, 0.4, 0.6, 0.8]
>>> list(zip(a, b, c))
[(1, 'w', 0.2), (2, 'x', 0.4), (3, 'y', 0.6), (4, 'z', 0.8))]
>>>
```

또한, zip()은 가장 짧은 입력 시퀀스의 마지막 원소에서 멈춘다.

```
>>> a = [1, 2, 3, 4, 5, 6]
>>> b = ['x', 'y', 'z']
>>> list(zip(a,b))
[(1, 'x'), (2, 'y'), (3, 'z')]
```

>>>
마지막 편집일시 : 2020년 7월 31일 4:13

2.5 collections 모듈
collections 모듈에는 데이터 처리를 위한 유용한 객체가 많이 있다. 그중 몇 가지를 간략히 소개한다.

예: 카운트하기
보유한 주식이 다음과 같을 때, 종목별로 합산해 나타내고 싶다고 하자.

```
portfolio = [
    ('GOOG', 100, 490.1),
    ('IBM', 50, 91.1),
    ('CAT', 150, 83.44),
    ('IBM', 100, 45.23),
    ('GOOG', 75, 572.45),
    ('AA', 50, 23.15)
]
```
위 리스트에 IBM과 GOOG가 두 개씩 있다. 종목별로 합산해 보자.

Counter
해법: Counter를 사용한다.

```
from collections import Counter
total_shares = Counter()
for name, shares, price in portfolio:
    total_shares[name] += shares

total_shares['IBM']      # 150
```
예: 일대다(One-Many) 매핑
문제: 하나의 키를 여러 개의 값에 매핑하려고 한다.

```
portfolio = [
    ('GOOG', 100, 490.1),
    ('IBM', 50, 91.1),
    ('CAT', 150, 83.44),
    ('IBM', 100, 45.23),
    ('GOOG', 75, 572.45),
    ('AA', 50, 23.15)
]
```
앞의 예와 같이, IBM을 키로 삼으면 두 개의 튜플이 있다.

해법: defaultdict를 사용한다.

```
from collections import defaultdict
holdings = defaultdict(list)
for name, shares, price in portfolio:
```

```
    holdings[name].append((shares, price))
holdings['IBM'] # [ (50, 91.1), (100, 45.23) ]
```

defaultdict을 사용하면 키에 액세스할 때마다 기본값을 얻는다.

예: 이력을 유지하기
문제: 마지막 N개의 이력(history)을 유지하고 싶다.

해법: deque를 사용한다.

```
from collections import deque

history = deque(maxlen=N)
with open(filename) as f:
    for line in f:
        history.append(line)
        ...
```

연습 문제
collections 모듈은 가장 유용한 라이브러리 모듈 중 하나로, 표와 인덱싱 같은 데이터를 다룰 때 특히 유용하다.

연습 문제를 통해 간단히 살펴보자. report.py 프로그램을 상호작용 모드로 실행해 주식 포트폴리오를 적재하자.

```
bash % python3 -i report.py
```

연습 문제 2.18: Counter로 표 형태의 데이터 다루기
각 종목의 총 주식수를 나타내는 표를 만들고 싶다고 하자. Counter 객체를 사용해 쉽게 할 수 있다. 한번 해 보자.

```
>>> portfolio = read_portfolio('Data/portfolio.csv')
>>> from collections import Counter
>>> holdings = Counter()
>>> for s in portfolio:
        holdings[s['name']] += s['shares']

>>> holdings
Counter({'MSFT': 250, 'IBM': 150, 'CAT': 150, 'AA': 100, 'GE': 95})
>>>
```

portfolio의 MSFT와 IBM을 어떻게 한 곳에 모았는지 주의 깊게 관찰하라

Counter를 사전처럼 사용해, 각각의 값을 조회할 수 있다.

```
>>> holdings['IBM']
150
>>> holdings['MSFT']
250
>>>
```

순위를 매기고 싶으면 다음과 같이 한다.

```
>>> # 가장 많이 보유한 종목 3가지
>>> holdings.most_common(3)
[('MSFT', 250), ('IBM', 150), ('CAT', 150)]
>>>
```

다른 포트폴리오를 가지고 새로운 Counter를 만들어보자.

```
>>> portfolio2 = read_portfolio('Data/portfolio2.csv')
>>> holdings2 = Counter()
>>> for s in portfolio2:
        holdings2[s['name']] += s['shares']

>>> holdings2
Counter({'HPQ': 250, 'GE': 125, 'AA': 50, 'MSFT': 25})
>>>
```

모든 보유 종목을 간단한 연산 하나로 결합해보자.

```
>>> holdings
Counter({'MSFT': 250, 'IBM': 150, 'CAT': 150, 'AA': 100, 'GE': 95})
>>> holdings2
Counter({'HPQ': 250, 'GE': 125, 'AA': 50, 'MSFT': 25})
>>> combined = holdings + holdings2
>>> combined
Counter({'MSFT': 275, 'HPQ': 250, 'GE': 220, 'AA': 150, 'IBM': 150, 'CAT': 150})
>>>
```

Counter의 능력을 아주 조금만 맛보았다. 표 형태의 값을 다룰 일이 있을 때, 이것을 사용할지 고려해보라.

부연 설명: collections 모듈
collections 모듈은 파이썬에서 가장 유용한 라이브러리 모듈 중 하나다. 사실 그것을 좀 더 설명할 수도 있었지만, 지금은 주의가 분산될 수 있어 그렇게 하지 않았다. 나중에 한가할 때 collections에 대해 읽어보라.

마지막 편집일시 : 2020년 10월 3일 6:02

2.6 리스트 컴프리헨션
리스트의 항목을 처리하는 것은 일반적인 작업이다. 이 섹션은 강력한 도구인 리스트 컴프리헨션을 소개한다.

새 리스트 생성하기
리스트 컴프리헨션은 시퀀스의 각 원소에 연산을 적용함으로써 새로운 리스트를 생성한다.

```
>>> a = [1, 2, 3, 4, 5]
>>> b = [2*x for x in a ]
>>> b
```

```
[2, 4, 6, 8, 10]
>>>
```
다른 예:

```
>>> names = ['Elwood', 'Jake']
>>> a = [name.lower() for name in names]
>>> a
['elwood', 'jake']
>>>
```
일반적인 구문: [<표현식> for <변수명> in <시퀀스>].

필터링
리스트 컴프리헨션을 통해 원소를 거를 수 있다.

```
>>> a = [1, -5, 4, 2, -2, 10]
>>> b = [2*x for x in a if x > 0 ]
>>> b
[2, 8, 4, 20]
>>>
```
용례
리스트 컴프리헨션은 엄청나게 유용하다. 예를 들어, 특정 딕셔너리 필드의 값을 수집할 수 있다.

```
stocknames = [s['name'] for s in stocks]
```
시퀀스에 데이터베이스 질의와 비슷한 것을 수행할 수 있다.

```
a = [s for s in stocks if s['price'] > 100 and s['shares'] > 50 ]
```
리스트 컴프리헨션과 시퀀스 축소를 조합할 수도 있다.

```
cost = sum([s['shares']*s['price'] for s in stocks])
```
일반적인 구문
[<표현식> for <변수명> in <시퀀스> if <조건>]
이것이 의미하는 바는 다음과 같다.

```
result = []
for 변수명 in 시퀀스:
    if 조건:
        result.append(표현식)
```
리스트 컴프리헨션의 유래
리스트 컴프리헨션은 수학에서 왔다(조건제시법).

```
a = [ x * x for x in s if x > 0 ] # 파이썬

a = { x^2 | x ∈ s, x > 0 }          # 수학
```
이 기능은 몇몇 언어에 구현돼 있다. 코더 대부분은 수학 시간에 배운 것까지 떠올리지는 않을 것이다. 그러므로, 리스트를 쉽게 만들어주는 쿨한 구문 정도로 받아들여도 좋다.

연습 문제
report.py 프로그램을 실행해 상호작용 모드에서 주식 포트폴리오를 적재한다.

```
bash % python3 -i report.py
```

이제, 파이썬 상호작용 프롬프트에서 다음과 같은 문장을 타이핑한다. 이 연산들은 다양한 데이터 축소(reduction), 변환(transform) 및 포트폴리오 데이터에 대한 질의를 수행한다.

연습 문제 2.19: 리스트 컴프리헨션
리스트 컴프리헨션 구문에 익숙해지도록 연습해 보자.

```
>>> nums = [1,2,3,4]
>>> squares = [ x * x for x in nums ]
>>> squares
[1, 4, 9, 16]
>>> twice = [ 2 * x for x in nums if x > 2 ]
>>> twice
[6, 8]
>>>
```

리스트 컴프리헨션을 통해 새로운 리스트를 생성하는 동시에 데이터를 변환 및 필터링했다.

연습 문제 2.20: 시퀀스 축소
단 하나의 파이썬 문장을 사용해 포트폴리오의 총 비용을 계산하자.

```
>>> portfolio = read_portfolio('Data/portfolio.csv')
>>> cost = sum([ s['shares'] * s['price'] for s in portfolio ])
>>> cost
44671.15
>>>
```

딘일 문장으로 포트폴리오의 현재 가격도 계산해 보자.

```
>>> value = sum([ s['shares'] * prices[s['name']] for s in portfolio ])
>>> value
28686.1
>>>
```

위의 두 연산은 모두 맵 축소(map-reduction)의 예다. 리스트 컴프리헨션은 리스트에 걸친 연산에 대해 매핑한다.

```
>>> [ s['shares'] * s['price'] for s in portfolio ]
[3220.0000000000005, 4555.0, 12516.0, 10246.0, 3835.1499999999996, 3254.9999999999995, 7044.0]
>>>
```

sum() 함수는 결과에 대해 축소를 수행한다.

```
>>> sum(_)
44671.15
>>>
```

이 지식을 가지고, 당신은 빅데이터 스타트업 회사를 시작할 준비가 되었다.

연습 문제 2.21: 데이터 질의
다양한 데이터 질의의 예를 따라해 보자.

먼저, 포트폴리오에서 100주 이상 보유한 종목의 리스트를 뽑아보자.

```
>>> more100 = [ s for s in portfolio if s['shares'] > 100 ]
>>> more100
[{'price': 83.44, 'name': 'CAT', 'shares': 150}, {'price': 51.23, 'name': 'MSFT', 'shares': 200}]
>>>
```

보유한 MSFT와 IBM 주식 전체.

```
>>> msftibm = [ s for s in portfolio if s['name'] in {'MSFT','IBM'} ]
>>> msftibm
[{'price': 91.1, 'name': 'IBM', 'shares': 50}, {'price': 51.23, 'name': 'MSFT', 'shares': 200},
  {'price': 65.1, 'name': 'MSFT', 'shares': 50}, {'price': 70.44, 'name': 'IBM', 'shares': 100}]
>>>
```

보유 종목 중 가격이 10,000 달러가 넘는 것의 리스트.

```
>>> cost10k = [ s for s in portfolio if s['shares'] * s['price'] > 10000 ]
>>> cost10k
[{'price': 83.44, 'name': 'CAT', 'shares': 150}, {'price': 51.23, 'name': 'MSFT', 'shares': 200}]
>>>
```

연습 문제 2.22: 데이터 추출
portfolio에서 얻은 name과 shares를 가지고 튜플의 리스트 (name, shares)를 만들자.

```
>>> name_shares =[ (s['name'], s['shares']) for s in portfolio ]
>>> name_shares
[('AA', 100), ('IBM', 50), ('CAT', 150), ('MSFT', 200), ('GE', 95), ('MSFT', 50), ('IBM', 100)]
>>>
```

대괄호([,])를 중괄호({, })로 바꾸면 세트 컴프리헨션이 된다. 그 결과는 고유한 값들의 세트가 된다.

예를 들어, 다음은 portfolio에서 고유한 종목명의 세트를 구한다.

```
>>> names = { s['name'] for s in portfolio }
>>> names
{ 'AA', 'GE', 'IBM', 'MSFT', 'CAT' }
>>>
```

key:value 쌍을 지정하면 딕셔너리를 만들 수 있다. 다음은 종목명을 보유 주식수와 매핑하는 딕셔너리를 만드는 예다.

```
>>> holdings = { name: 0 for name in names }
>>> holdings
{'AA': 0, 'GE': 0, 'IBM': 0, 'MSFT': 0, 'CAT': 0}
>>>
```

이 기능을 딕셔너리 컴프리헨션(dictionary comprehension)이라 한다. 테이블로 나타내자.

```
>>> for s in portfolio:
        holdings[s['name']] += s['shares']

>>> holdings
{ 'AA': 100, 'GE': 95, 'IBM': 150, 'MSFT':250, 'CAT': 150 }
>>>
```

prices 딕셔너리의 항목 중 포트폴리오에 포함된 종목만 추려보자.

```
>>> portfolio_prices = { name: prices[name] for name in names }
>>> portfolio_prices
{'AA': 9.22, 'GE': 13.48, 'IBM': 106.28, 'MSFT': 20.89, 'CAT': 35.46}
>>>
```

연습 문제 2.23: CSV 파일에서 데이터 추출하기

리스트, 세트, 딕셔너리를 다양하게 조합해 데이터 처리에 활용할 수 있다. CSV 파일에서 선택된 컬럼을 추출하는 예를 살펴보자.

먼저, CSV 파일에서 헤더 정보가 있는 행을 읽는다.

```
>>> import csv
>>> f = open('Data/portfoliodate.csv')
>>> rows = csv.reader(f)
>>> headers = next(rows)
>>> headers
['name', 'date', 'time', 'shares', 'price']
>>>
```

다음으로, 실제 데이터 컬럼을 나열하는 변수명을 정의한다.

```
>>> select = ['name', 'shares', 'price']
>>>
```

이제, 입력 CSV 파일에서 컬럼의 인덱스를 찾는다.

```
>>> indices = [ headers.index(colname) for colname in select ]
>>> indices
[0, 3, 4]
>>>
```

끝으로, 데이터의 행을 읽어 딕셔너리 컴프리헨션을 사용해 딕셔너리를 만든다.

```
>>> row = next(rows)
>>> record = { colname: row[index] for colname, index in zip(select, indices) }    # 딕셔너리 컴프리헨션
>>> record
{'price': '32.20', 'name': 'AA', 'shares': '100'}
>>>
```

무슨 일이 일어났는지 이해했으면, 파일의 나머지 부분도 읽어보자.

```
>>> portfolio = [ { colname: row[index] for colname, index in zip(select, indices) } for row in rows ]
>>> portfolio
[{'price': '91.10', 'name': 'IBM', 'shares': '50'}, {'price': '83.44', 'name': 'CAT', 'shares': '150'},
  {'price': '51.23', 'name': 'MSFT', 'shares': '200'}, {'price': '40.37', 'name': 'GE', 'shares': '95'},
  {'price': '65.10', 'name': 'MSFT', 'shares': '50'}, {'price': '70.44', 'name': 'IBM', 'shares': '100'}]
>>>
```

세상에, read_portfolio() 함수를 한 문장으로 줄였다.

부연 설명

리스트 컴프리헨션은 파이썬에서 데이터를 변환, 필터링, 수집하는 데 일반적으로 사용된다. 그 구문이 복잡하므로, 가능하면 리스트 컴프리헨션을 단순하게 유지하라. 여러 단계의 코드로 나눠 구현해도 괜찮다. 예를 들어, 마지막 예제를 당신의 순진한 동료에게 불쑥 내밀어도 괜찮을지 잘 모르겠다.

어쨌든, 데이터를 빠르게 조작하는 방법을 아는 것은 매우 유용한 기술이다 데이터 가져오기, 내보내기, 추출 등 일회성 문제를 해결해야 하는 상황이 많다. 리스트 컴프리헨션을 숙달하면 해결책을 고안하는 데 걸리는 시간을 상당히 줄일 수 있다. collections 모듈도 잊지 말자.

마지막 편집일시 : 2021년 3월 7일 9:55

2.7 객체

이 섹션은 파이썬의 내부 객체 모델을 자세히 소개하고 메모리 관리, 복사, 타입 검사를 논의한다.

할당(Assignment)

파이썬의 많은 연산자는 값의 할당(assigning) 혹은 저장(storing)과 관련이 있다.

```
a = value            # 변수에 할당
s[n] = value         # 리스트에 할당
s.append(value)      # 리스트에 추가
d['key'] = value     # 딕셔너리에 추가
```

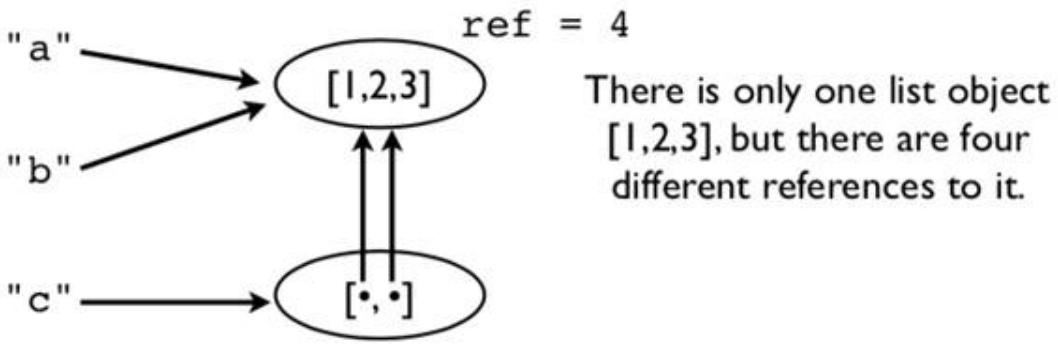
주의: 할당 연산은 할당될 값의 사본을 절대 만들지 않는다. 모든 할당은 레퍼런스 복사(혹은 포인터 복사)일 뿐이다.

할당의 예

다음 코드를 생각해 보자.

```
a = [1,2,3]
b = a
c = [a,b]
```

메모리 연산의 그림을 그려 보자. 이 예에는 단 한 개의 리스트 객체 [1,2,3]이 있지만, 그것을 가리키는 레퍼런스(reference)는 4개가 있다.



레퍼런스

이것이 의미하는 바는 값을 수정하면 모든 레퍼런스에 영향을 미친다는 것이다.

```
>>> a.append(999)
>>> a
[1,2,3,999]
>>> b
[1,2,3,999]
>>> c
[[1,2,3,999], [1,2,3,999]]
>>>
```

원래의 리스트를 변경하자 모든 리스트가 바뀌어 버렸다(으악!). 사본을 만들지 않았기 때문이다. 모든 것이 같은 리스트를 가리키고 있었던 것이다.

값을 재할당

값을 재할당한다고 해서, 이전 값에서 사용한 메모리를 덮어쓰는 것이 절대 아니다.

```
a = [1,2,3]
b = a
a = [4,5,6]

print(a)      # [4, 5, 6]
print(b)      # [1, 2, 3]    원래 값이 있음
```

기억해줘: 변수는 이름일 뿐, 메모리 위치가 아니다.

위험성

이 공유방식을 이해하지 못하면 자기 발등을 찍는 일이 생길 수 있다. 일반적인 시나리오. 어떤 데이터가 프라이빗(private) 사본이라고 생각해서 수정했다가 프로그램의 다른 부분에서 사용하는 데이터를 오염시키는 사고가 일어날 수 있다.

설명: 이것이 기본 자료형(int, float, string)이 변경 불가능(읽기 전용)인 한 가지 이유다.

id와 레퍼런스

is 연산자를 사용해 두 값이 정확히 같은 객체인지 검사할 수 있다.

```
>>> a = [1,2,3]
>>> b = a
>>> a is b
True
>>>
```

is는 객체의 id(정수)를 비교한다. id는 id()를 사용해 얻을 수 있다.

```
>>> id(a)
3588944
>>> id(b)
3588944
>>>
```

참고: 객체 검사에는 늘 ==를 사용하는 게 좋다. is는 종종 예상치 못한 작동을 한다.

```
>>> a = [1,2,3]
>>> b = a
>>> c = [1,2,3]
>>> a is b
True
>>> a is c
False
>>> a == c
True
>>>
```

얕은 복사(shallow copy)

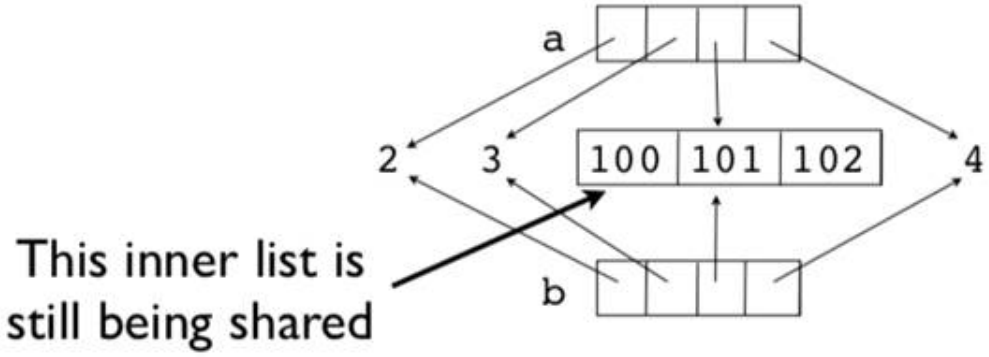
리스트와 딕셔너리에는 복사를 위한 메서드가 있다.

```
>>> a = [2,3,[100,101],4]
>>> b = list(a) # 사본을 만듦
>>> a is b
False
```

리스트가 새로 만들어지긴 했지만 리스트 항목은 공유된다.

```
>>> a[2].append(102)
>>> b[2]
[100,101,102]
>>>
>>> a[2] is b[2]
True
>>>
```

예를 들어, 내부 리스트 [100, 101, 102]이 공유된다. 이것을 얕은 복사라 한다. 그림을 보자.



얕은 복사

깊은 복사(deep copy)
객체와 그 안에 있는 모든 객체의 사본이 필요할 때가 종종 있다. 이를 위해 copy 모듈을 사용할 수 있다.

```
>>> a = [2,3,[100,101],4]
>>> import copy
>>> b = copy.deepcopy(a)
>>> a[2].append(102)
>>> b[2]
[100,101]
>>> a[2] is b[2]
False
>>>
```

이름, 값, 타입
변수명은 타입을 갖지 않는다. 그저 이름일 뿐이다. 하지만, 값에는 타입이 있다.

```
>>> a = 42
>>> b = 'Hello World'
>>> type(a)
<type 'int'>
>>> type(b)
<type 'str'>
```

type()을 실행하면 값의 타입을 알려준다. 타입명은 함수로 사용되어 해당 타입의 값을 생성하거나, 다른 타입의 값을 해당 타입으로 변환한다.

타입 검사(Type Checking)
어떤 객체가 특정 타입이 맞는지 확인하는 방법이다.

```
if isinstance(a, list):
    print('a is a list')
```

가능한 여러 타입 중 하나에 대해 확인한다.

```
if isinstance(a, (list,tuple)):
    print('a is a list or tuple')
```
주의: 타입 검사를 지나치게 사용하지 말 것. 과도한 코드 복잡성을 초래할 수 있다. 다른 사람이 당신의 코드를 사용할 때 저지를 수 있는 일반적인 실수를 방지할 수 있을 때만 사용하라.

모든 것이 객체다

숫자, 문자열, 리스트, 함수, 예외, 클래스, 인스턴스(instance) 등 모든 것은 객체다. 모든 객체에 대해 이름을 붙이고, 데이터로 전달하고, 컨테이너에 넣는 등의 일을 아무 제약 없이 할 수 있다. 특별 대우를 받는 객체는 없다. '모든 객체가 일급'이라고도 표현한다.

간단한 예를 보자.

```
>>> import math
>>> items = [abs, math, ValueError ]
>>> items
[<built-in function abs>,
  <module 'math' (builtin)>,
  <type 'exceptions.ValueError'>]
>>> items[0](-45)
45
>>> items[1].sqrt(2)
1.4142135623730951
>>> try:
        x = int('not a number')
    except items[2]:
        print('Failed!')
Failed!
>>>
```
여기서 items는 함수, 모듈, 예외를 포함하는 리스트다. 원래 이름 대신에 리스트의 항목을 바로 사용할 수 있다.

```
items[0](-45)        # abs
items[1].sqrt(2)     # math
except items[2]:     # ValueError
```
강력함에는 책임이 따른다. 할 수 있다고 해서 함부로 사용하지는 말자.

연습 문제

연습 문제를 통해 일급 객체의 강력함을 느껴보자.

연습 문제 2.24: 일급 데이터

Data/portfolio.csv 파일에서, 다음과 같이 컬럼으로 조직화된 데이터를 읽는다.

```
name,shares,price
"AA",100,32.20
"IBM",50,91.10
```

...
이전 코드에서, 우리는 파일을 읽는 데 csv 모듈을 사용했지만, 여전히 수작업으로 형 변환을 수행해야 했다. 예:

```
for row in rows:
    name   = row[0]
    shares = int(row[1])
    price  = float(row[2])
```

약간의 리스트 기본 연산을 사용해 이러한 유형의 변환을 좀 더 영리하게 수행할 수 있다.

각 컬럼을 적절한 타입으로 변환하는 함수의 이름을 담는 파이썬 리스트를 만들자.

```
>>> types = [str, int, float]
>>>
```

이런 리스트를 만드는 것조차 가능한 이유는 파이썬에서는 모든 것이 일급(first-class)이기 때문이다. 함수의 리스트를 만들고 싶다면 얼마든지 그렇게 할 수 있다. 방금 생성한 리스트의 항목은 값 x를 주어진 타입으로 변환하는 함수들이다(예: str(x), int(x), float(x)).

이제, 위 파일에서 데이터의 행을 읽는다.

```
>>> import csv
>>> f = open('Data/portfolio.csv')
>>> rows = csv.reader(f)
>>> headers = next(rows)
>>> row = next(rows)
>>> row
['AA', '100', '32.20']
>>>
```

이 값들은 타입이 잘못되어 계산에 부적합하다. 예:

```
>>> row[1] * row[2]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't multiply sequence by non-int of type 'str'
>>>
```

하지만, 데이터를 types에 지정한 타입과 짝 지을 수 있다. 예:

```
>>> types[1]
<type 'int'>
>>> row[1]
'100'
>>>
```

값 하나를 변환해 보자.

```
>>> types[1](row[1])       # int(row[1])와 같음
100
```

```
>>>
```
다른 값도 변환해 보자.

```
>>> types[2](row[2])       # float(row[2])와 같음
32.2
>>>
```
변환한 값을 가지고 계산해 보자.

```
>>> types[1](row[1])*types[2](row[2])
3220.0000000000005
>>>
```
zip으로 컬럼 타입과 필드를 묶은 결과는 다음과 같다.

```
>>> r = list(zip(types, row))
>>> r
[(<type 'str'>, 'AA'), (<type 'int'>, '100'), (<type 'float'>,'32.20')]
>>>
```
타입 변환과 값이 짝 지어진 것을 볼 수 있다. 예를 들어, int는 값 '100'과 짝을 이룬다.

리스트에 zip을 사용하면 모든 값을 하나하나 변환할 수 있어 유용하다. 다음과 같이 해 보라.

```
>>> converted = []
>>> for func, val in zip(types, row):
            converted.append(func(val))
...
>>> converted
['AA', 100, 32.2]
>>> converted[1] * converted[2]
3220.0000000000005
>>>
```
위 코드에서 어떤 일이 일어나는지 이해해야 한다. 루프에서, func 변수는 타입 변환 함수들(예: str, int 등) 중 하나이며, val 변수는 'AA', '100'과 같은 값들 중 하나다. 표현식 func(val)은 값을 변환한다(타입 캐스트와 비슷하다).

위의 코드를 단일 리스트 컴프리헨션으로 압축할 수 있다.

```
>>> converted = [func(val) for func, val in zip(types, row)]
>>> converted
['AA', 100, 32.2]
>>>
```
연습 문제 2.25: 딕셔너리 만들기
키 이름과 값의 시퀀스를 갖고 있을 때 dict() 함수를 가지고 딕셔너리를 쉽게 만들 수 있는 것을 기억하는가? 컬럼 헤더로부터 딕셔너리를 만들자.

```
>>> headers
['name', 'shares', 'price']
```

```
>>> converted
['AA', 100, 32.2]
>>> dict(zip(headers, converted))
{'price': 32.2, 'name': 'AA', 'shares': 100}
>>>
```
물론, 리스트 컴프리헨션에 능통하다면, 딕셔너리 컴프리헨션을 사용해 전 과정을 한 번에 끝낼 수 있다.

```
>>> { name: func(val) for name, func, val in zip(headers, types, row) }
{'price': 32.2, 'name': 'AA', 'shares': 100}
>>>
```
연습 문제 2.26: 큰 그림

이 연습 문제의 기법을 사용해, 컬럼 위주의 데이터 파일의 필드를 파이썬 딕셔너리로 쉽게 변환하는 문장을 작성할 수 있다.

설명을 위해, 다음과 같은 데이터 파일에서 데이터를 읽는다고 가정하자.

```
>>> f = open('Data/dowstocks.csv')
>>> rows = csv.reader(f)
>>> headers = next(rows)
>>> row = next(rows)
>>> headers
['name', 'price', 'date', 'time', 'change', 'open', 'high', 'low', 'volume']
>>> row
['AA', '39.48', '6/11/2007', '9:36am', '-0.18', '39.67', '39.69', '39.45', '181800']
>>>
```
비슷한 트릭을 사용해 필드를 변환하자.

```
>>> types = [str, float, str, str, float, float, float, float, int]
>>> converted = [func(val) for func, val in zip(types, row)]
>>> record = dict(zip(headers, converted))
>>> record
{'volume': 181800, 'name': 'AA', 'price': 39.48, 'high': 39.69,
'low': 39.45, 'time': '9:36am', 'date': '6/11/2007', 'open': 39.67,
'change': -0.18}
>>> record['name']
'AA'
>>> record['price']
39.48
>>>
```
보너스: date를 (6, 11, 2007)과 같은 튜플로 파싱하려면 이 예제를 어떻게 수정해야 할까?

이 연습 문제에서 한 것을 곰곰 생각해 보라. 이 개념을 나중에 다시 살펴본다.

마지막 편집일시 : 2020년 10월 4일 12:28 오

==========

3. 프로그램 조직화

지금까지 파이썬 기초를 배웠으며 짧은 스크립트를 작성했다. 하지만 더 큰 프로그램을 작성하려면 조직화할 필요가 있다. 이 섹션은 함수를 작성하고 오류를 처리하는 방법을 자세히 다루며 모듈을 소개한다. 끝에 가서는 여러 파일에 걸쳐 함수로 분할된 프로그램을 작성할 수 있다. 스크립트를 작성하는 데 유용한 코드 템플릿도 제공한다.

- 3.1 함수와 스크립트 작성
- 3.2 함수에 대한 자세한 내용
- 3.3 예외 처리
- 3.4 모듈
- 3.5 메인 모듈
- 3.6 유용성을 위한 설계 논의

3.1 스크립팅

파이썬 스크립트를 작성하는 방법을 자세히 살펴보자.

스크립트(Script)란?

스크립트는 여러 문장을 실행한 다음 멈추는 프로그램이다.

```
# program.py

문장1
문장2
문장3
...
```

위의 프로그램은 스크립트에 가깝다고 할 수 있다.

문제점

유용한 스크립트를 작성해 사용하다 보면 기능이 점차 늘어난다. 관련된 다른 문제에도 적용하고 싶을 수 있다. 시간이 흐름에 따라, 스크립트는 점점 더 중요한 애플리케이션이 되어 버릴 수도 있다. 이제 주의를 기울이지 않으면 큰 문제가 일어날 수 있다. 그러므로 체계를 세워야 한다.

정의하기

나중에 재사용하려면 이름을 붙여둬야 한다.

```
def square(x):
    return x*x

a = 42
b = a + 2      # `a`가 정의돼 있어야 함

z = square(b) # `square`와 `b`가 정의돼 있어야 함
```

순서가 중요하다. 변수와 함수의 정의를 항상 위쪽에 둬야 한다.

함수 정의하기
단일 작업에 관련된 코드를 한 곳에 모아두는 것이 현명하다. 함수 사용하기.

```
def read_prices(filename):
    prices = {}
    with open(filename) as f:
        f_csv = csv.reader(f)
        for row in f_csv:
            prices[row[0]] = float(row[1])
    return prices
```

함수는 반복적인 연산을 단순화하기도 한다.

```
oldprices = read_prices('oldprices.csv')
newprices = read_prices('newprices.csv')
```

함수란 무엇인가?
함수는 문장의 시퀀스에 이름을 붙인 것이다.

```
def funcname(args):
  문장
  문장
  ...
  return 결과
```

함수 내에 모든 파이썬 문장을 사용할 수 있다.

```
def foo():
    import math
    print(math.sqrt(2))
    help(math)
```

파이썬에는 특수한 문장이 없다(그러므로 기억하기 쉽다).

함수 정의
함수는 어떠한 순서로든 정의할 수 있다.

```
def foo(x):
    bar(x)

def bar(x):
    문장

# 또는
def bar(x):
    문장

def foo(x):
    bar(x)
```

함수는 반드시 프로그램 실행 중 사용(호출)하기 전에 정의되어 있어야 한다.

```
foo(3)          # foo가 정의돼 있어야 한다
```
함수는 상향식(bottom-up)으로 작성하는 것이 일반적이다.

상향식
함수는 빌딩 블록과 같이 다뤄진다. 작고 단순한 블록부터 먼저 만든다.

```
# myprogram.py
def foo(x):
    ...

def bar(x):
    ...
    foo(x)          # 위에 정의됨
    ...

def spam(x):
    ...
    bar(x)          # 아래에 정의됨
    ...

spam(42)            # 함수를 사용하는 코드가 끝에 나타남
```
나중에 나오는 함수는 앞에 나온 함수를 바탕으로 한다. 다시 말하지만, 이것은 스타일일 뿐이다. 위 프로그램에서 중요한 것은 spam(42) 호출을 마지막에 해야 한다는 것 뿐이다.

함수 설계
함수는 블랙 박스(black box)로 간주하는 것이 이상적이다. 전달된 입력만 가지고 연산을 하며 글로벌 변수의 사용이나 신비한 부작용을 배제해야 한다. 모듈화(Modularity)와 예측가능성(Predictability)을 목표로 하라.

문서 문자열(Doc Strings)
문서를 doc-string 형태로 코드에 포함하는 것이 좋다. doc-strings은 함수 이름 바로 뒤에 직접 쓴 문자열이다. help(), IDE 및 기타 도구에서 인식한다.

```
def read_prices(filename):
    '''
    CSV 파일에서 이름과 가격 데이터를 읽음
    '''
    prices = {}
    with open(filename) as f:
        f_csv = csv.reader(f)
        for row in f_csv:
            prices[row[0]] = float(row[1])
    return prices
```
문서 문자열을 작성할 때는 함수가 무슨 일을 하는지를 짧은 문장으로 작성하는 것이 좋다. 추가 정보가 필요할 때는 짧은 사용 예제와 함께 인자에 대한 자세한 설명을 넣어라.

타입 애너테이션(Type Annotations)
함수 정의에 선택적인 타입 힌트(type hint)를 추가할 수 있다.

```
def read_prices(filename: str) -> dict:
    '''
    CSV 파일에서 이름과 가격 데이터를 읽음
    '''
    prices = {}
    with open(filename) as f:
        f_csv = csv.reader(f)
        for row in f_csv:
            prices[row[0]] = float(row[1])
    return prices
```

힌트는 실제로 연산을 하지는 않는다. 순전히 정보성이다. 하지만, IDE, 코드 검사기, 기타 도구에서 사용된다.

연습 문제
섹션 2에서 주식 포트폴리오의 성과를 나타내는 보고서를 프린트하는 report.py 프로그램을 작성했다. 이 프로그램에는 몇 가지 함수가 있다. 예:

```
# report.py
import csv

def read_portfolio(filename):
    '''
    주식 포트폴리오 파일을 읽어 딕셔너리의 리스트를 생성.
    name, shares, price를 키로 사용.
    '''
    portfolio = []
    with open(filename) as f:
        rows = csv.reader(f)
        headers = next(rows)

        for row in rows:
            record = dict(zip(headers, row))
            stock = {
                'name' : record['name'],
                'shares' : int(record['shares']),
                'price' : float(record['price'])
            }
            portfolio.append(stock)
    return portfolio
```

...
하지만, 스크립트로 된 일련의 계산을 수행하는 프로그램 부분도 있다. 이 코드는 프로그램의 끝부분 근처에 있다. 예:

```
...

# 보고서를 출력

headers = ('Name', 'Shares', 'Price', 'Change')
print('%10s %10s %10s %10s'  % headers)
print(('-' * 10 + ' ') * len(headers))
for row in report:
    print('%10s %10d %10.2f %10.2f' % row)
...
```

이 연습 문제에서, 이 프로그램의 함수 사용 부분을 좀 더 강력하게 조직화한다.

연습 문제 3.1: 프로그램을 함수의 컬렉션으로 구조화
계산과 출력을 포함한 모든 주요 연산을 함수의 컬렉션에 의해 수행하도록 report.py 프로그램을 수정하자. 특히,

보고서를 출력하는 print_report(report) 함수를 작성한다.
프로그램의 마지막 부분을 수정해 일련의 함수만 남기고 계산 코드를 없애자.
연습 문제 3.2: 프로그램 실행을 위한 최상위 함수를 작성
프로그램의 마지막 부분을 가지고 단일 함수 portfolio_report(portfolio_filename, prices_filename)로 패키징하라. 그 함수에서 다음과 같이 호출해, 이전과 똑같은 보고서를 생성하자.

```
portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
```

이 최종 버전에는 일련의 함수 정의와 함께, 맨 마지막에 portfolio_report() 함수 호출만 유일하게 남긴다(이 함수가 이 프로그램과 관련된 모든 단계를 실행).

프로그램을 단일 함수로 바꾸면 입력이 바뀌더라도 쉽게 실행할 수 있다. 예를 들어, 상호작용 모드에서 프로그램을 실행한 후 다음과 같은 문장을 실행해 보라.

```
>>> portfolio_report('Data/portfolio2.csv', 'Data/prices.csv')
... 출력을 보라 ...
>>> files = ['Data/portfolio.csv', 'Data/portfolio2.csv']
>>> for name in files:
        print(f'{name:-^43s}')
        portfolio_report(name, 'prices.csv')
        print()

... 출력을 보라 ...
>>>
```

부연 설명
파이썬을 사용해 문장을 순서대로 나열한 비구조적인 스크립팅 코드를 아주 쉽게 작성할 수 있다. 큰 그림에서, 가능하다면 함수를 활용하는 것이 좋다. 언젠가는 스크립트가 커져서 진작 조직화해두지 않은 것을 아쉬워하게 된다. 또한, 파이썬은 함수를 사용하면 더 빨리 실행된다.

마지막 편집일시 : 2020년 7월 2일 12:40

3.2 함수의 작동
앞에서 이미 함수를 소개했지만, 실제로 어떻게 작동하는지 좀 더 깊이 알아보자. 이 섹션의 목적은 약간의 갭을 메우고 호출 관례, 스코핑 규칙 같은 문제를 논의하는 것이다.

함수 호출하기
이 함수를 생각해 보자.

```
def read_prices(filename, debug):
    ...
```

위치 인자를 사용해 함수를 호출할 수 있다.

```
prices = read_prices('prices.csv', True)
```

키워드 인자를 가지고 함수를 호출할 수도 있다.

```
prices = read_prices(filename='prices.csv', debug=True)
```

기본 인자
인자를 선택 사항으로 하고 싶을 때가 있다. 그런 경우에는 함수 정의에 기본값을 정해두라.

```
def read_prices(filename, debug=False):
    ...
```

기본값이 할당되면 인자는 함수 호출시 선택사항이 된다.

```
d = read_prices('prices.csv')
e = read_prices('prices.dat', True)
```

참고: 기본값이 있는 인자는 반드시 인자 리스트의 끝에 두어야 한다(선택적이지 않은 인자가 모두 앞쪽에 와야 한다).

선택적 인자를 키워드 인자로 하면 좋다.
두 가지 호출 방식을 비교, 대조해보자.

```
parse_data(data, False, True) # ?????

parse_data(data, ignore_errors=True)
parse_data(data, debug=True)
parse_data(data, debug=True, ignore_errors=True)
```

키워드 인자를 사용하는 것이 대체로 가독성이 높다. 플래그 역할을 하는 인자라든지, 선택적 기능과 관련된 경우 특히 그렇다.

설계 모범 사례
함수 인자에는 항상 짧고 의미 있는 이름을 붙인다.

함수를 키워드 호출 스타일로 사용하고 싶을 수도 있다.

```
d = read_prices('prices.csv', debug=True)
```

파이썬 개발 도구는 도움말 기능과 문서에서 이름을 표시한다.

값을 반환
return 문을 값을 반환한다.

```
def square(x):
    return x * x
```

반환값이 주어지지 않거나 return 문이 없으면 None을 반환한다.

```
def bar(x):
    문장
    return

a = bar(4)       # a = None

# 또는
def foo(x):
    statements  # `return`이 없음

b = foo(4)       # b = None
```

여러 개의 반환값
함수는 단 하나의 값만 반환할 수 있다. 하지만, 튜플에 여러 값을 담아 반환할 수 있다.

```
def divide(a,b):
    q = a // b       # 몫
    r = a % b        # 나머지
    return q, r      # 튜플을 반환
```

용례:

```
x, y = divide(37,5) # x = 7, y = 2

x = divide(37, 5)   # x = (7, 2)
```

변수 스코프
프로그램은 변수에 값을 할당한다.

```
x = value # 글로벌 변수

def foo():
    y = value # 로컬 변수
```
변수 할당은 함수 정의의 안팎에서 일어난다. 바깥에서 정의된 변수는 '글로벌'이다. 함수 내의 변수는 '로컬'이다.

로컬 변수
함수 내에 할당된 변수는 프라이빗이다.

```
def read_portfolio(filename):
    portfolio = []
    for line in open(filename):
        fields = line.split(',')
        s = (fields[0], int(fields[1]), float(fields[2]))
        portfolio.append(s)
    return portfolio
```
이 예에서, filename, portfolio, line, fields, s는 로컬 변수다. 이 변수들은 함수 호출 이후에 보존되지 않으므로 이것들에 액세스할 수 없다.

```
>>> stocks = read_portfolio('portfolio.csv')
>>> fields
Traceback (most recent call last):
File "<stdin>", line 1, in ?
NameError: name 'fields' is not defined
>>>
```
로컬은 다른 곳의 변수와 충돌할 일이 없다.

글로벌 변수
함수는 같은 파일에 정의된 글로벌들의 값에 자유롭게 액세스할 수 있다.

```
name = 'Dave'

def greeting():
    print('Hello', name)  # 글로벌 변수 `name`을 사용
```
단, 함수는 글로벌을 수정할 수 없다.

```
name = 'Dave'

def spam():
  name = 'Guido'
```

```
spam()
print(name) # 'Dave'를 프린트
```
기억해줘: 함수 내의 모든 할당은 로컬이다.

글로벌을 수정하기
글로벌 변수를 수정해야만 한다면 다음과 같은 방법으로 선언한다.

```
name = 'Dave'

def spam():
    global name
    name = 'Guido' # 위 글로벌 name을 변경
```
글로벌을 사용하기 전에 선언이 먼저 나타나야 하며, 관련된 변수는 반드시 함수와 같은 파일에 있어야 한다. 글로벌을 사용하는 법을 알아보기는 했지만, 별로 좋은 것은 아니다. global을 아예 사용하지 않는 것이 상책이다. 함수가 외부의 상태를 변경할 필요가 있다면, 차라리 클래스를 사용해라(나중에 다룬다).

인자 전달
함수를 호출할 때, 인자 변수는 전달된 값을 참조하는 이름이다. 이 변수들은 사본이 아니다(섹션 2.7을 참조). 변경가능한 자료형(예: list, dict)을 전달하면 제자리에서 수정될 수 있다.

```
def foo(items):
    items.append(42)     # 입력 객체를 수정

a = [1, 2, 3]
foo(a)
print(a)                 # [1, 2, 3, 42]
```
핵심: 함수는 입력 인자의 사본을 받지 않는다.

재할당 vs 수정
변수를 수정하는 것과 변수명을 재할당하는 것의 미묘한 차이를 이해해야 한다.

```
def foo(items):
    items.append(42)     # 입력 객체를 수정

a = [1, 2, 3]
foo(a)
print(a)                 # [1, 2, 3, 42]

# VS
```

```
def bar(items):
    items = [4,5,6]     # 다른 객체를 가리키도록 로컬 `items` 변수를 변경

b = [1, 2, 3]
bar(b)
print(b)                    # [1, 2, 3]
```

기억해줘: 변수 할당은 메모리를 절대 덮어쓰지 않는다. 이름은 새 값에 바운드된다.

연습 문제

이 연습 문제들은 이 코스에서 가장 강력하고 어려운 부분을 구현한다. 지금까지의 연습 문제를 통해 익힌 여러 개념을 한 곳에 모은다. 최종적인 해답은 25줄 가량의 코드에 불과하지만, 시간을 들여 각 부분을 이해하도록 노력하라.

report.py 프로그램에서 중요한 부분은 CSV 파일을 읽는 것이다. 예를 들어 read_portfolio() 함수는 포트폴리오 데이터의 행을 포함한 파일을 읽으며, read_prices() 함수는 가격 데이터의 행을 포함한 파일을 읽는다. 두 함수 모두 저수준의 "fiddly" 비트가 있고 기능이 서로 비슷한다. 예를 들어, 파일을 열고 csv 모듈을 사용해 감싸서 값을 새로운 타입으로 변환한다.

파일 파싱을 실제로 많이 한다면 이런 것들을 깔끔하게 정리해 좀 더 일반적으로 사용할 수 있게 하는 것이 좋다. 이것이 우리 목표다.

Work/fileparse.py 파일을 열어 이 연습을 시작하자. 이 파일을 가지고 작업한다.

연습 문제 3.3: CSV 파일 읽기

먼저 CSV 파일을 읽어 딕셔너리의 리스트에 넣는 문제부터 해결하자. fileparse.py 파일에 다음과 같은 함수를 정의한다.

```
# fileparse.py
import csv

def parse_csv(filename):
    '''
    CSV 파일을 파싱해 레코드의 목록을 생성
    '''
    with open(filename) as f:
        rows = csv.reader(f)

        # 헤더를 읽음
        headers = next(rows)
        records = []
```

```
        for row in rows:
            if not row:     # 데이터가 없는 행을 건너뜀
                continue
            record = dict(zip(headers, row))
            records.append(record)

    return records
```

이 함수는 CSV 파일을 읽어 딕셔너리의 리스트를 생성하되, 파일을 열고 csv 모듈을 가지고 감싸고, 공백 행을 건너뛰는 등의 세부사항을 숨긴다.

한번 사용해 보자.

힌트: python3 -i fileparse.py.

```
>>> portfolio = parse_csv('Data/portfolio.csv')
>>> portfolio
[{'price': '32.20', 'name': 'AA', 'shares': '100'}, {'price': '91.10', 'name': 'IBM', 'shares': '50'}, {'price': '83.44', 'name': 'CAT', 'shares': '150'}, {'price': '51.23', 'name': 'MSFT', 'shares': '200'}, {'price': '40.37', 'name': 'GE', 'shares': '95'}, {'price': '65.10', 'name': 'MSFT', 'shares': '50'}, {'price': '70.44', 'name': 'IBM', 'shares': '100'}]
>>>
```

이 함수는 좋긴 하지만, 모든 것을 문자열로 표현하기 때문에 계산에 사용할 수 없다는 아쉬움이 있다. 이 점을 곧 수정하겠지만, 일단은 계속 만들어 나가자.

연습 문제 3.4: 컬럼 선택기 구축하기

CSV 파일의 모든 데이터가 아니라 일부에만 관심이 있는 때가 많을 것이다. 다음과 같이, 가져올 컬럼을 사용자가 선택할 수 있게 parse_csv() 함수를 수정해 보자.

```
>>> # 전체 데이터를 읽기
>>> portfolio = parse_csv('Data/portfolio.csv')
>>> portfolio
[{'price': '32.20', 'name': 'AA', 'shares': '100'}, {'price': '91.10', 'name': 'IBM', 'shares': '50'}, {'price': '83.44', 'name': 'CAT', 'shares': '150'}, {'price': '51.23', 'name': 'MSFT', 'shares': '200'}, {'price': '40.37', 'name': 'GE', 'shares': '95'}, {'price': '65.10', 'name': 'MSFT', 'shares': '50'}, {'price': '70.44', 'name': 'IBM', 'shares': '100'}]

>>> # 일부 데이터만 읽기
>>> shares_held = parse_csv('Data/portfolio.csv', select=['name','shares'])
>>> shares_held
```

```
[{'name': 'AA', 'shares': '100'}, {'name': 'IBM', 'shares': '50'}, {'name': 'CAT', 'shares': '150'}, {'name': 'MSFT', 'shares': '200'}, {'name': 'GE', 'shares': '95'}, {'name': 'MSFT', 'shares': '50'}, {'name': 'IBM', 'shares': '100'}]
>>>
```

컬럼 선택기의 예가 연습 문제 2.23에 있다. 그렇지만, 여기 한 가지 방법이 있다.

```
# fileparse.py
import csv

def parse_csv(filename, select=None):
    '''
    CSV 파일을 파싱해 레코드의 목록을 생성
    '''
    with open(filename) as f:
        rows = csv.reader(f)

        # 헤더를 읽음
        headers = next(rows)

        # 컬럼 선택기가 주어지면, 지정한 컬럼의 인덱스를 찾는다.
        # 또한 결과 딕셔너리에 사용할 헤더의 집합을 좁힌다
        if select:
            indices = [headers.index(colname) for colname in select]
            headers = select
        else:
            indices = []

        records = []
        for row in rows:
            if not row:     # 데이터가 없는 행을 건너뜀
                continue
            # 특정 컬럼이 선택되었으면 필터링
            if indices:
                row = [ row[index] for index in indices ]

            # 딕셔너리를 만듦
            record = dict(zip(headers, row))
            records.append(record)

    return records
```

이 부분에 몇 가지 트릭이 있다. 컬럼 선택을 행 인덱스에 매핑하는 부분이 가장 중요할 것이

다. 예를 들어, 입력 파일에 다음과 같은 헤더가 있다고 하자.

```
>>> headers = ['name', 'date', 'time', 'shares', 'price']
>>>
```

선택한 컬럼은 다음과 같다고 하자.

```
>>> select = ['name', 'shares']
>>>
```

적절한 선택을 위해, 선택한 컬럼명을 파일의 컬럼 인덱스에 매핑해야 한다. 이 단계에서 그 일을 한다.

```
>>> indices = [headers.index(colname) for colname in select ]
>>> indices
[0, 3]
>>>
```

달리 말해, "name"은 컬럼 0이고 "shares"는 컬럼 3이다. 파일에서 데이터 행을 읽을 때, 인덱스를 사용해 원하는 컬럼만 얻을 수 있다.

```
>>> row = ['AA', '6/11/2007', '9:50am', '100', '32.20' ]
>>> row = [ row[index] for index in indices ]
>>> row
['AA', '100']
>>>
```

연습 문제 3.5: 형변환 수행하기

반환된 데이터에 형변환을 적용할지 선택할 수 있게 parse_csv() 함수를 수정해보자. 예:

```
>>> portfolio = parse_csv('Data/portfolio.csv', types=[str, int, float])
>>> portfolio
[{'price': 32.2, 'name': 'AA', 'shares': 100}, {'price': 91.1, 'name': 'IBM', 'shares': 50}, {'price': 83.44, 'name': 'CAT', 'shares': 150}, {'price': 51.23, 'name': 'MSFT', 'shares': 200}, {'price': 40.37, 'name': 'GE', 'shares': 95}, {'price': 65.1, 'name': 'MSFT', 'shares': 50}, {'price': 70.44, 'name': 'IBM', 'shares': 100}]

>>> shares_held = parse_csv('Data/portfolio.csv', select=['name', 'shares'], types=[str, int])
>>> shares_held
[{'name': 'AA', 'shares': 100}, {'name': 'IBM', 'shares': 50}, {'name': 'CAT', 'shares': 150}, {'name': 'MSFT', 'shares': 200}, {'name': 'GE', 'shares': 95}, {'name': 'MSFT', 'shares': 50}, {'name': 'IBM', 'shares': 100}]
>>>
```

이것을 연습 문제 2.24에서 이미 탐구했다. 다음 코드를 당신의 코드에 포함시키자.

```
...
if types:
    row = [func(val) for func, val in zip(types, row) ]
...
```

연습 문제 3.6: 헤더를 다루기

CSV 파일에 헤더 정보가 없는 경우도 있다 예를 들어, prices.csv는 다음과 같이 되어 있다.

```
"AA",9.22
"AXP",24.85
"BA",44.85
"BAC",11.27
...
```

튜플의 리스트를 생성함으로써 이런 파일을 다룰 수 있게 parse_csv() 함수를 수정해보자. 예:

```
>>> prices = parse_csv('Data/prices.csv', types=[str,float], has_headers=False)
>>> prices
[('AA', 9.22), ('AXP', 24.85), ('BA', 44.85), ('BAC', 11.27), ('C', 3.72), ('CAT', 35.46), ('CVX', 66.67), ('DD', 28.47), ('DIS', 24.22), ('GE', 13.48), ('GM', 0.75), ('HD', 23.16), ('HPQ', 34.35), ('IBM', 106.28), ('INTC', 15.72), ('JNJ', 55.16), ('JPM', 36.9), ('KFT', 26.11), ('KO', 49.16), ('MCD', 58.99), ('MMM', 57.1), ('MRK', 27.58), ('MSFT', 20.89), ('PFE', 15.19), ('PG', 51.94), ('T', 24.79), ('UTX', 52.61), ('VZ', 29.26), ('WMT', 49.74), ('XOM', 69.35)]
>>>
```

이렇게 변경하려면 데이터의 첫행이 헤더 파일로 인식되지 않게 코드를 수정해야 한다. 또한, 컬럼명을 키로 삼아 딕셔너리를 생성하지 않게 수정해야 한다.

연습 문제 3.7: 다른 컬럼 구분자(delimitier)를 사용하기

콤마를 구분자로 사용하는 것이 일반적이지만, 탭이나 공백을 컬럼 구분자로 사용하는 경우도 있다. 그 예로, Data/portfolio.dat은 다음과 같다.

```
name shares price
"AA" 100 32.20
"IBM" 50 91.10
"CAT" 150 83.44
"MSFT" 200 51.23
"GE" 95 40.37
"MSFT" 50 65.10
"IBM" 100 70.44
```

csv.reader() 함수는 구분자를 변경하는 기능이 있다.

rows = csv.reader(f, delimiter=' ')
구분자를 변경할 수 있게 parse_csv() 함수를 수정해보자.

예:

```
>>> portfolio = parse_csv('Data/portfolio.dat', types=[str, int, float], delimiter=' ')
>>> portfolio
[{'price': '32.20', 'name': 'AA', 'shares': '100'}, {'price': '91.10', 'name': 'IBM', 'shares': '50'}, {'price': '83.44', 'name': 'CAT', 'shares': '150'}, {'price': '51.23', 'name': 'MSFT', 'shares': '200'}, {'price': '40.37', 'name': 'GE', 'shares': '95'}, {'price': '65.10', 'name': 'MSFT', 'shares': '50'}, {'price': '70.44', 'name': 'IBM', 'shares': '100'}]
>>>
```

부연 설명
여기까지 잘 수행했다면 정말 유용한 라이브러리 함수를 만든 셈이다. 이것을 가지고 임의의 CSV 파일에서 원하는 컬럼을 선택해 형변환을 해보라. 파일의 내부 작동이나 csv 모듈 사용에 크게 신경쓸 필요가 없다.

마지막 편집일시 : 2020년 10월 4일 1:13

3.3 오류 검사
예외를 이미 다뤘지만, 이 섹션에서 오류 검사와 예외 처리에 대해 좀 더 알아보자.

프로그램은 어떻게 실패하는가
파이썬은 인자 타입이나 값에 대한 검사나 검증을 수행하지 않는다. 어떤 데이터든 함수에 기술한 것과 호환되기만 하면 함수가 작동한다.

```
def add(x, y):
    return x + y

add(3, 4)                 # 7
add('Hello', 'World')    # 'HelloWorld'
add('3', '4')            # '34'
```

함수에 오류가 있으면 실행시간에 예외로서 나타난다.

```
def add(x, y):
    return x + y

>>> add(3, '4')
Traceback (most recent call last):
```

```
...
TypeError: unsupported operand type(s) for +:
'int' and 'str'
>>>
```

코드를 검증하기 위해 테스팅을 매우 중요시한다(나중에 다룬다).

예외

예외는 오류에 대한 신호로 사용된다. 예외를 일으키고 싶으면 raise 문을 사용하라.

```
if name not in authorized:
    raise RuntimeError(f'{name} not authorized')
```

예외를 붙잡으려면 try-except를 사용한다.

```
try:
    authenticate(username)
except RuntimeError as e:
    print(e)
```

예외 처리

예외는 처음 일치하는 except까지 전파된다.

```
def grok():
    ...
    raise RuntimeError('Whoa!')    # 여기서 예외가 발생

def spam():
    grok()                         # 호출하면 예외가 발생

def bar():
    try:
        spam()
    except RuntimeError as e:      # 예외를 여기서 잡음
        ...

def foo():
    try:
        bar()
    except RuntimeError as e:      # 예외는 여기 도달하지 않음
        ...

foo()
```

예외를 처리하려면 except 블록에 문장을 넣는다. 예외를 처리하는 데 사용하고 싶은 문장을

무엇이든 넣을 수 있다.

```
def grok(): ...
    raise RuntimeError('Whoa!')

def bar():
    try:
      grok()
    except RuntimeError as e:   # 예외를 여기서 잡음
        statements              # 이 문장을 사용
        문장
        ...

bar()
```

처리한 다음, try-except 이후 첫 번째 문장으로 실행을 재개한다.

```
def grok(): ...
    raise RuntimeError('Whoa!')

def bar():
    try:
      grok()
    except RuntimeError as e:   # 예외를 여기서 잡음
        문장
        문장
        ...
    문장                        # 예외를 여기서 재개
    문장                        # 여기서 계속
    ...

bar()
```

빌트인 예외

스무 가지가 넘는 빌트인 예외가 있다. 예외 이름은 무엇이 잘못됐는지 지시한다(예: 잘못된 값을 제공하면 ValueError가 일어난다). 이 목록에 있는 것 외에도 더 있다. 더 알고 싶으면 문서를 참조하라.

ArithmeticError
AssertionError
EnvironmentError
EOFError
ImportError

IndexError
KeyboardInterrupt
KeyError
MemoryError
NameError
ReferenceError
RuntimeError
SyntaxError
SystemError
TypeError
ValueError
예외 값
예외에는 관련 값이 있다. 무엇이 잘못됐는지 구체적으로 알려주는 정보가 포함된다.

```
raise RuntimeError('Invalid user name')
```

이 값은 예외 인스턴스의 일부이며 except에 제공된 변수에 배치된다.

```
try:
    ...
except RuntimeError as e:   # 발생한 예외를 `e`로 가리킨다
    ...
```

e는 예외 타입의 인스턴스다. 하지만, 이것을 프린트하면 문자열처럼 보인다.

```
except RuntimeError as e:
    print('Failed : Reason', e)
```

여러 오류를 잡기
except 블록을 여러 개 사용해서 서로 다른 종류의 예외를 붙잡을 수 있다.

```
try:
  ...
except LookupError as e:
  ...
except RuntimeError as e:
  ...
except IOError as e:
  ...
except KeyboardInterrupt as e:
  ...
```

또는, 처리하는 명령문이 동일하다면 그룹화할 수 있다.

```
try:
```

```
  ...
except (IOError,LookupError,RuntimeError) as e:
  ...
```

모든 오류를 붙잡기

예외를 붙잡으려면 다음과 같이 Exception을 사용한다.

```
try:
    ...
except Exception:        # 위험! 아래를 참조
    print('An error occurred')
```

일반적으로, 코드를 이런 식으로 작성하는 것은 좋은 생각이 아니다. 실패 이유를 알 수 없기 때문이다.

오류를 붙잡는 잘못된 방식

예외를 바람직하지 않은 방식으로 사용하는 예를 살펴보자.

```
try:
    go_do_something()
except Exception:
    print('Computer says no')
```

이렇게 하면 발생할 수 있는 모든 오류를 붙잡아 버리므로, 전혀 예상치 못한 이유로 문제가 생기는 경우(예: 파이썬 모듈을 설치하지 않았을 때 등) 디버깅이 불가능하다.

더 나은 방법

모든 오류를 다 붙잡고 싶으면 다음과 같이 접근하는 것이 좋다.

```
try:
    go_do_something()
except Exception as e:
    print('Computer says no. Reason :', e)
```

이렇게 하면 실패한 이유를 구체적으로 보고한다. 발생할 수 있는 모든 오류를 붙잡도록 코드를 작성할 때는 이와 같이 오류를 보고하는 메커니즘을 갖춰두는 것이 좋다.

더 좋은 방법은 오류의 범위를 좁히는 것이다. 실제로 처리할 수 있는 오류만 잡아라. 처리할 수 없는 오류는 다른 코드에서 처리하게 내버려두라.

예외를 다시 일으키기

붙잡은 오류를 전파하려면 raise를 사용한다.

```
try:
    go_do_something()
```

```
except Exception as e:
    print('Computer says no. Reason :', e)
    raise
```
이렇게 하면 행동을 취하고(예: 로깅) 호출자에게 오류를 전달할 수 있다.

예외 모범 사례
예외를 붙잡지 않는다. 일찍 큰 소리를 내며 실패한다. 중요한 문제라면 다른 사람이 신경 써 줄 것이다. 그렇게 해 줄 다른 사람이 없을 때만 예외를 붙잡아라. 회복해서 진행이 가능한 오류만 붙잡으라는 뜻이다.

finally 문
예외가 발생했는지의 여부와 관계 없이 실행해야 할 코드를 지정한다.

```
lock = Lock()
...
lock.acquire()
try:
    ...
finally:
    lock.release()  # 이것은 항상 실행된다! 예외 발생 여부와 관계 없이.
```
리소스를 안전하게 관리하기 위해 사용한다(잠금, 파일 등).

with 문
요즘은 코드에서 try-finally를 with 문으로 대신하곤 한다.

```
lock = Lock()
with lock:
    # 잠금을 획득
    ...
# 잠금 해제
```
좀 더 친숙한 예를 보자.

```
with open(filename) as f:
    # 파일을 사용
    ...
# 파일을 닫음
```
with는 리소스 사용 컨텍스트(context)를 정의한다. 예외가 컨텍스트에서 벗어나면 리소스는 해제된다. with 문은 해당 구문을 지원하도록 특별히 프로그래밍된 객체에 대해서만 작동한다.

연습 문제

연습 문제 3.8: 예외 일으키기
지난 섹션에서 작성한 parse_csv() 함수는 사용자 정의 컬럼을 선택할 수 있게 해주지만, 입력 데이터 파일에 컬럼 헤더가 있어야만 올바로 작동한다.

select와 has_headers=False 인자가 함께 전달되는 경우 예외를 일으키게 코드를 수정해 보라. 예:

```
>>> parse_csv('Data/prices.csv', select=['name','price'], has_headers=False)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "fileparse.py", line 9, in parse_csv
    raise RuntimeError("select argument requires column headers")
RuntimeError: select argument requires column headers
>>>
```

이 검사 외에, 함수에서 또 다른 종류의 입력값 검사를 수행해야 하는 것은 아닌지 궁금할 것이다. 예를 들어, 파일명이 문자열인지, 타입이 리스트인지, 아니면 다른 것인지 확인해야 할까?

일반적인 규칙으로, 그러한 테스트를 건너뛰고, 잘못된 입력이 들어오면 프로그램이 실패하게 내버려두는 것이 최선일 때가 많다. 트레이스백 메시지가 문제를 지목하여 디버깅을 도와줄 것이다.

위에서 확인을 추가한 이유는 프로그램이 무의미하게 실행되는 것을 피하기 위해서다(예: 컬럼 헤더를 필요로 하는 기능을 사용함에도 불구하고, 그와 동시에 아무 헤더도 없다고 지정하는 것).

이는 호출하는 코드에 프로그래밍 오류가 있음을 가리킨다. "발생해서는 안 되는" 케이스를 검사하는 것은 좋은 생각이다.

연습 문제 3.9: 예외를 붙잡기
앞에서 작성한 parse_csv() 함수는 파일의 전체 내용을 처리하는 데 사용된다. 하지만, 실제 세계에서는 입력 파일이 손상되거나, 값이 누락되거나, 오류 데이터가 섞여 있을 수 있다. 다음과 같이 실험해보자.

```
>>> portfolio = parse_csv('Data/missing.csv', types=[str, int, float])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "fileparse.py", line 36, in parse_csv
    row = [func(val) for func, val in zip(types, row)]
ValueError: invalid literal for int() with base 10: ''
>>>
```

레코드 생성 도중 발생한 모든 ValueError 예외를 붙잡아, 변환이 불가능한 행에 대해 경고 메시지를 프린트하도록 parse_csv() 함수를 수정하라.

메시지에 행 번호와 실패 이유를 나타내는 정보를 포함한다. 함수를 테스트하기 위해, 위의 Data/missing.csv 파일을 읽는다. 예:

```
>>> portfolio = parse_csv('Data/missing.csv', types=[str, int, float])
Row 4: Couldn't convert ['MSFT', '', '51.23']
Row 4: Reason invalid literal for int() with base 10: ''
Row 7: Couldn't convert ['IBM', '', '70.44']
Row 7: Reason invalid literal for int() with base 10: ''
>>>
>>> portfolio
[{'price': 32.2, 'name': 'AA', 'shares': 100}, {'price': 91.1, 'name': 'IBM', 'shares': 50}, {'price': 83.44, 'name': 'CAT', 'shares': 150}, {'price': 40.37, 'name': 'GE', 'shares': 95}, {'price': 65.1, 'name': 'MSFT', 'shares': 50}]
>>>
```

연습 문제 3.10: 오류 억제
사용자가 명시적으로 원하는 경우 오류 메시지를 발생시키지 않게 parse_csv() 함수를 수정하라. 예:

```
>>> portfolio = parse_csv('Data/missing.csv', types=[str,int,float], silence_errors=True)
>>> portfolio
[{'price': 32.2, 'name': 'AA', 'shares': 100}, {'price': 91.1, 'name': 'IBM', 'shares': 50}, {'price': 83.44, 'name': 'CAT', 'shares': 150}, {'price': 40.37, 'name': 'GE', 'shares': 95}, {'price': 65.1, 'name': 'MSFT', 'shares': 50}]
>>>
```

대부분의 프로그램에서 오류 처리는 가장 어려운 부분이다. 일반적인 규칙은 오류를 조용히 무시해버리지 말라는 것이다. 그렇게 하는 것보다는, 문제를 보고하되 사용자가 원할 때에 한해 오류 메시지를 내지 않는 것이 낫다.

마지막 편집일시 : 2021년 3월 8일 1:19

3.4 모듈
이 섹션은 모듈의 개념을 소개하고 여러 파일에 걸쳐 함수를 다룬다.

모듈과 임포트(import)
모든 파이썬 소스 파일은 모듈이다.

```
# foo.py
```

```
def grok(a):
    ...
def spam(b):
    ...
```

import 문은 모듈을 적재(load)하고 실행(execute)한다.

```
# program.py
import foo

a = foo.grok(2)
b = foo.spam('Hello')
...
```

네임스페이스(Namespace)

모듈은 이름이 있는 변수들의 컬렉션이며, 종종 네임스페이스라고도 부른다. 이름은 모두 글로벌 변수이며 함수는 소스 파일에 정의된다. 임포트한 뒤, 모듈명은 프리픽스(prefix)로 사용된다. 네임스페이스라고 부르는 것은 그 때문이다.

```
import foo

a = foo.grok(2)
b = foo.spam('Hello')
...
```

모듈명은 파일명과 직접 연관된다(foo -> foo.py).

글로벌 정의

글로벌(global) 스코프에 정의된 모든 것은 모듈 네임스페이스에 존재한다. 같은 변수 x를 정의하는 두 모듈을 생각해 보자.

```
# foo.py
x = 42
def grok(a):
    ...
# bar.py
x = 37
def spam(a):
    ...
```

두 경우 x 정의는 서로 다른 변수를 참조한다. 하나는 foo.x이고, 다른 하나는 bar.x다. 서로 다른 모듈이 같은 이름을 사용할 수 있으며, 이때 이름은 서로 충돌하지 않는다.

모듈은 독립적이다(isolated).

환경으로서의 모듈
모듈은 내부에 정의된 모든 코드를 감싸는 환경을 형성한다.

```
# foo.py
x = 42

def grok(a):
    print(x)
```

글로벌 변수들은 항상 감싸는 모듈(같은 파일)에 바인딩된다. 각 소스 파일은 개별적인 소우주다.

모듈 실행
모듈이 임포트될 때, 파일의 끝에 도달할 때까지 모듈의 모든 문장이 차례대로 실행된다. 모듈 네임스페이스의 내용은 모두 글로벌 이름이며 실행 과정의 끝에 정의된다. 글로벌 스코프에서 작업(프린팅, 파일 생성 등)을 수행하는 스크립팅 문장이 있다면, 임포트할 때 수행되는 것을 볼 수 있을 것이다.

import as 문
모듈을 임포트하면서 그 이름을 바꿔 사용할 수 있다.

```
import math as m
def rectangular(r, theta):
    x = r * m.cos(theta)
    y = r * m.sin(theta)
    return x, y
```

그냥 임포트할 때와 똑같이 작동한다. 한 파일 내에서 모듈 이름을 바꿔 부를 뿐이다.

from 모듈 import
다음과 같이 모듈에서 심볼을 선택해 로컬에서 사용할 수 있다.

```
from math import sin, cos

def rectangular(r, theta):
    x = r * cos(theta)
    y = r * sin(theta)
    return x, y
```

이렇게 하면 모듈 프리픽스 없이도 모듈의 일부를 사용할 수 있다. 자주 사용하는 이름을 가져올 때 유용한 방식이다.

임포트에 대한 설명
임포트를 어떤 방식으로 사용하든, 모듈이 작동하는 방식은 달라지지 않는다.

```
import math
# vs
import math as m
# vs
from math import cos, sin
...
```
특히, import는 항상 파일 전체를 실행시키며 모듈은 환경과 여전히 분리된다.

import 모듈 as 문은 로컬의 이름만 바꾼다. from math import cos, sin 문은 여전히 math 모듈 전체를 적재한다. 이것을 수행하면 모듈에 있는 cos와 sin이 로컬 스페이스에 복사될 뿐이다.

모듈 적재
각 모듈은 단 한 번만 적재 및 실행된다. 참고: 임포트를 여러 번 하더라도 이전에 적재한 모듈의 레퍼런스를 반환한다.

sys.modules은 적재된 모듈 전체의 딕셔너리다.

```
>>> import sys
>>> sys.modules.keys()
['copy_reg', '__main__', 'site', '__builtin__', 'encodings', 'encodings.encodings', 'posixpath', ...]
>>>
```
주의: 만약 모듈의 소스 코드를 변경한 후에 import 문을 수행하면 혼동이 생긴다. 모듈을 임포트하면 sys.modules에 캐시되므로, 임포트를 여러 번 수행하면 이전에 적재된 모듈이 반환되며, 모듈을 수정하더라도 반영되지 않는다. 수정한 모듈을 적재하는 가장 안전한 방법은 파이썬을 종료하고 다시 시작하는 것이다.

모듈 위치
파이썬은 모듈을 찾을 때 경로 리스트(sys.path)를 참고한다.

```
>>> import sys
>>> sys.path
[
  '',
  '/usr/local/lib/python36/python36.zip',
  '/usr/local/lib/python36',
  ...
]
```
현재 작업 디렉터리가 가장 먼저 온다.

모듈 검색 경로
언급한 바와 같이, sys.path에는 검색 경로가 있다. 필요한 경우 수작업으로 변경할 수 있다.

```
import sys
sys.path.append('/project/foo/pyfiles')
```

환경 변수를 통해 경로를 추가할 수도 있다.

```
% env PYTHONPATH=/project/foo/pyfiles python3
Python 3.6.0 (default, Feb 3 2017, 05:53:21)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.38)]
>>> import sys
>>> sys.path
['','/project/foo/pyfiles', ...]
```

일반적으로, 모듈 검색 경로를 수작업으로 조정할 필요는 없다. 하지만, 일반적이지 않은 위치에 있다든지 현재 작업 디렉터리에서 액세스할 준비가 되지 않은 파이썬 코드를 임포트할 일이 생기곤 한다.

연습 문제
이 연습 문제는 모듈과 관련 있기 때문에, 파이썬을 적절한 환경에서 실행하는 것이 매우 중요하다. 프로그래머들이 겪는 모듈과 관련된 문제는 대부분 현재 작업 디렉터리 또는 파이썬 경로 설정과 관련된다. 이 코스에서는 작성하는 모든 코드가 Work/ 디렉터리에 있는 것으로 가정한다. 최선의 결과를 얻으려면, 인터프리터도 그 디렉터리에서 실행하라. 그렇게 하지 않으려면 sys.path에 practical-python/Work를 추가해야 한다.

연습 문제 3.11: 모듈 임포트
섹션 3에서 우리는 CSV 데이터 파일의 내용을 파싱하는 범용 함수 parse_csv()를 작성했다.

이제 그 함수를 다른 프로그램에서 사용하는 법을 알아보자. 먼저, 새로운 셸 창을 시작한다. 파일이 있는 폴더로 이동한다. 그 파일들을 임포트할 것이다.

파이썬 상호작용 모드를 시작한다.

```
bash % python3
Python 3.6.1 (v3.6.1:69c0db5050, Mar 21 2017, 01:21:04)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

이전에 작성한 프로그램을 임포트해보자. 이전과 똑같이 출력이 표시된다. 모듈을 임포트하면 코드가 실행된다.

```
>>> import bounce
... 출력을 보라 ...
>>> import mortgage
... 출력을 보라 ...
>>> import report
... 출력을 보라 ...
>>>
```

위와 같이 작동하지 않는다면 파이썬을 잘못된 디렉터리에서 실행하고 있을 가능성이 높다. 이제 fileparse 모듈을 임포트하고 도움말을 표시해 보자.

```
>>> import fileparse
>>> help(fileparse)
... 출력을 보라 ...
>>> dir(fileparse)
... 출력을 보라 ...
>>>
```

이 모듈을 사용해 데이터를 읽는다.

```
>>> portfolio = fileparse.parse_csv('Data/portfolio.csv',select=['name','shares','price'],
types=[str,int,float])
>>> portfolio
... 출력을 보라 ...
>>> pricelist = fileparse.parse_csv('Data/prices.csv',types=[str,float],
has_headers=False)
>>> pricelist
... 출력을 보라 ...
>>> prices = dict(pricelist)
>>> prices
... 출력을 보라 ...
>>> prices['IBM']
106.11
>>>
```

모듈 이름을 포함하지 않도록 함수를 임포트해 보자.

```
>>> from fileparse import parse_csv
>>> portfolio = parse_csv('Data/portfolio.csv', select=['name','shares','price'],
types=[str,int,float])
>>> portfolio
... 출력을 보라 ...
>>>
```

연습 문제 3.12: 라이브러리 모듈 사용하기

섹션 2에서 다음과 같은 주식 보고서를 생성하는 report.py 프로그램을 작성했다.

```
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100       9.22      -22.98
       IBM          50     106.28       15.18
       CAT         150      35.46      -47.98
      MSFT         200      20.89      -30.34
        GE          95      13.48      -26.89
      MSFT          50      20.89      -44.21
       IBM         100     106.28       35.84
```

모든 입력 파일의 처리에 fileparse 모듈에 있는 함수를 사용하도록 프로그램을 수정하라. 이 작업 위해 fileparse를 모듈로서 임포트하고 read_portfolio() 와 read_prices() 함수에서 parse_csv() 함수를 사용하게 수정한다.

이 연습 문제의 처음에 예로 든 상호작용적인 예제를 참고하라. 수정 후에도 이전과 똑같이 출력되어야 한다.

연습 문제 3.14: 라이브러리 임포트를 더 많이 사용하기
섹션 1에서, 포트폴리오를 읽어 비용을 계산하는 pcost.py 프로그램을 작성했다.

```
>>> import pcost
>>> pcost.portfolio_cost('Data/portfolio.csv')
44671.15
>>>
```

report.read_portfolio() 함수를 사용하게 pcost.py 파일을 수정하라.

부연 설명
이 연습 문제를 마치면 세 개의 프로그램을 갖게 된다. fileparse.py에는 범용 parse_csv() 함수가 있다. report.py는 훌륭한 보고서를 생성하며, read_portfolio()와 read_prices() 함수도 갖는다. 끝으로, pcost.py는 포트폴리오 비용을 계산하며, report.py를 위해 작성된 read_portfolio() 함수를 사용한다.

마지막 편집일시 : 2020년 10월 3일 7:09

3.5 메인 모듈
이 섹션은 메인 프로그램 혹은 메인 모듈의 개념을 소개한다.

메인 함수
여러 프로그래밍 언어에는 메인(main) 함수 또는 메서드 개념이 있다.

```
// c / c++
int main(int argc, char *argv[]) {
    ...
}
// 자바
class myprog {
    public static void main(String args[]) {
        ...
    }
}
```

이것은 애플리케이션을 시작할 때 처음 실행하는 함수다.

파이썬 메인 모듈

파이썬에는 메인 함수나 메서드가 없다. 그 대신 메인 모듈이 있다. 처음 실행하는 소스 파일이 메인 모듈이다.

```
bash % python3 prog.py
...
```

인터프리터에 무엇을 전달하든, 그것이 메인 모듈이 된다. 이름이 무엇이든 상관없다.

__main__ 확인

모듈이 메인 스크립트로 작동하는지 알아보는 관례적인 방법이다.

```
# prog.py
...
if __name__ == '__main__':
    # 메인 프로그램으로서 실행 ...
    문장
    ...
```

위 if 문에 속한 문장이 메인 프로그램이 된다.

메인 프로그램 vs. 라이브러리 임포트

모든 파이썬 파일은 메인으로 실행하거나 라이브러리로서 임포트된다.

```
bash % python3 prog.py # 메인으로서 실행
import prog    # 라이브러리 임포트되어 실행
```

두 경우 모두 __name__은 모듈의 이름이다. 그러나 메인으로서 실행할 때만 __main__이 된다.

라이브러리 임포트로서 실행할 때는 메인 프로그램으로서 실행하는 문장을 실행하고 싶지 않을 수 있다. 그러므로 어떤 쓰임새인지를 if로 검사하는 것이다.

```
if __name__ == '__main__':
    # 임포트될 때는 실행하지 않는다 ...
```

프로그램 템플릿

다음은 파이썬 프로그램을 작성하는 공통적인 프로그램 템플릿이다.

```
# prog.py
# Import 문(라이브러리)
import modules

# 함수
def spam():
    ...

def blah():
    ...

# 메인 함수
def main():
    ...

if __name__ == '__main__':
    main()
```

명령행 도구

파이썬은 종종 명령행 도구로 사용된다.

```
bash % python3 report.py portfolio.csv prices.csv
```

그 말은 스크립트를 셸 또는 터미널에서 실행한다는 뜻이다. 주용도는 자동화, 백그라운드 작업 등이다.

명령행 인자

명령행은 텍스트 열의 리스트다.

```
bash % python3 report.py portfolio.csv prices.csv
```

이러한 텍스트 열의 리스트를 sys.argv에서 찾을 수 있다.

```
# 위 bash 명령에서
sys.argv # ['report.py, 'portfolio.csv', 'prices.csv']
```

인자를 처리하는 간단한 예를 살펴보자.

```
import sys
```

```
if len(sys.argv) != 3:
    raise SystemExit(f'Usage: {sys.argv[0]} ' 'portfile pricefile')
portfile = sys.argv[1]
pricefile = sys.argv[2]
...
```

표준 입출력

표준 입출력(stdio)은 일반 파일과 똑같이 작동하는 파일이다.

```
sys.stdout
sys.stderr
sys.stdin
```

print는 기본적으로 sys.stdout로 지정(direct)된다. 입력은 sys.stdin으로부터 읽는다. 트레이스백(Traceback)과 오류(error)는 sys.stderr로 지정된다.

stdio는 터미널, 파일, 파이프 등에 연결될 수 있다.

```
bash % python3 prog.py > results.txt
# 또는
bash % cmd1 | python3 prog.py | cmd2
```

환경 변수(Environment Variable)

환경 변수는 셸에서 설정된다.

```
bash % setenv NAME dave
bash % setenv RSH ssh
bash % python3 prog.py
```

os.environ은 이러한 값을 담는 딕셔너리다.

```
import os

name = os.environ['NAME'] # 'dave'
```

변경은 프로그램이 나중에 실행시키는 서브프로세스에 반영된다.

프로그램 종료(Program Exit)

프로그램 종료는 예외를 통해 처리된다.

```
raise SystemExit
raise SystemExit(exitcode)
raise SystemExit('Informative message')
```

대안.

```
import sys
sys.exit(exitcode)
```
0이 아닌 종료 코드(exit code)는 오류를 나타낸다.

#! 행

유닉스에서 #! 행은 스크립트를 파이썬으로서 실행할 수 있다. 다음과 같이 스크립트 파일 첫 행에 추가한다.

```
#!/usr/bin/env python3
# prog.py
...
```
실행 권한이 필요하다.

```
bash % chmod +x prog.py
# Then you can execute
bash % prog.py
... 출력 ...
```
참고: 윈도우의 Python Launcher도 언어 버전을 확인하기 위해 #! 행을 찾는다.

스크립트 템플릿

끝으로, 명령행 스크립트로서 실행되는 파이썬 프로그램을 위한 공통적인 코드 터미널이 있다.

```
#!/usr/bin/env python3
# prog.py

# Import statements (libraries)
import modules

# Functions
def spam():
    ...

def blah():
    ...

# 메인 함수
def main(argv):
    # 명령행 인자, 환경 변수 등을 파싱.
    ...
```

```
if __name__ == '__main__':
    import sys
    main(sys.argv)
```

연습 문제

연습 문제 3.15: main() 함수

report.py 파일에 명령행 옵션의 리스트를 받아 이전과 같은 출력을 생성하는 main() 함수를 추가하자. 그것을 다음과 같이 상호작용적으로 실행할 수 있어야 한다.

```
>>> import report
>>> report.main(['report.py', 'Data/portfolio.csv', 'Data/prices.csv'])
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84
>>>
```

비슷한 main() 함수를 갖도록 pcost.py 파일을 수정하자.

```
>>> import pcost
>>> pcost.main(['pcost.py', 'Data/portfolio.csv'])
Total cost: 44671.15
>>>
```

연습 문제 3.16: 스크립트 만들기

report.py와 pcost.py 프로그램을 수정해 명령행에서 스크립트로서 실행할 수 있게 해 보자.

```
bash $ python3 report.py Data/portfolio.csv Data/prices.csv
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84

bash $ python3 pcost.py Data/portfolio.csv
```

Total cost: 44671.15
마지막 편집일시 : 2020년 10월 4일 1:28

3.6 설계에 대한 논의
이 섹션에서는 앞에서 결정한 설계를 재검토한다.

파일명 vs 이터러블
같은 출력을 반환하는 다음 두 프로그램을 비교해 보자.

```
# 파일명을 제공
def read_data(filename):
    records = []
    with open(filename) as f:
        for line in f:
            ...
            records.append(r)
    return records

d = read_data('file.csv')
# 행을 제공
def read_data(lines):
    records = []
    for line in lines:
        ...
        records.append(r)
    return records

with open('file.csv') as f:
    d = read_data(f)
```

어느 함수를 선호하는가? 이유는?
이 함수 중 어느 것이 더 유연한가?
심오한 아이디어: "덕 타이핑"
컴퓨터 프로그래밍에서 덕 타이핑(Duck Typing)이란, 어떤 객체를 특정 목적으로 사용할 수 있는지 판단하는 것과 관련된 개념이다. 덕 테스트(duck test)의 예를 들어 설명한다.

만약 어떤 것이 오리처럼 생겼고, 오리처럼 헤엄치고, 오리처럼 꽥꽥거린다면, 그것은 아마도 오리일 것이다.

위 read_data()의 두 번째 버전에서, 함수는 이터러블 객체를 예상한다. 파일의 행에 국한되지 않는다.

```
def read_data(lines):
    records = []
    for line in lines:
        ...
        records.append(r)
    return records
```

이는 우리가 다른 lines를 가지고 사용할 수 있음을 의미한다.

```
# CSV 파일
lines = open('data.csv')
data = read_data(lines)
```

```
# ZIP 압축된 파일
lines = gzip.open('data.csv.gz','rt')
data = read_data(lines)

# 표준 입력
lines = sys.stdin
data = read_data(lines)

# 문자열의 리스트
lines = ['ACME,50,91.1','IBM,75,123.45', ... ]
data = read_data(lines)
```
이 설계는 상당히 유연하다.

질문: 이러한 유연성을 환영해야 할까, 아니면 꺼려야 할까?

라이브러리 설계 모범 사례
유연한 코드 라이브러리는 더 나은 서비스를 제공할 수 있다. 옵션을 제한하지 말자. 유연성이 높을수록 더 강력해진다.

연습 문제
연습 문제 3.17: 파일명에서 파일 비슷한 객체로
앞에서 parse_csv() 함수가 있는 fileparse.py 파일을 작성했다. 이 함수는 다음과 같이 작동한다.

```
>>> import fileparse
>>> portfolio = fileparse.parse_csv('Data/portfolio.csv', types=[str,int,float])
>>>
```
이 함수는 파일명이 전달될 것으로 예상한다. 이 코드의 유연성을 좀 더 높이면 좋을 것이다. 파일과 유사하거나 이터러블한 객체를 다룰 수 있게 함수를 수정해 보라. 예:

```
>>> import fileparse
>>> import gzip
>>> with gzip.open('Data/portfolio.csv.gz', 'rt') as file:
...         port = fileparse.parse_csv(file, types=[str,int,float])
...
>>> lines = ['name,shares,price', 'AA,100,34.23', 'IBM,50,91.1', 'HPE,75,45.1']
>>> port = fileparse.parse_csv(lines, types=[str,int,float])
>>>
```
새로운 코드에 예전처럼 파일명을 전달하면 무슨 일이 일어나는가?

```
>>> port = fileparse.parse_csv('Data/portfolio.csv', types=[str,int,float])
>>> port
... 출력을 살펴보라(미쳤다) ...
>>>
```
그렇다. 조심해야 한다. 이런 일이 생기지 않게 안전 장치를 둘 수는 없을까?

연습 문제 3.18: 기존 함수를 수정하기
parse_csv()의 수정된 버전과 작동할 수 있게, report.py 파일의 read_portfolio()와 read_prices() 함수를 수정해 보라. 코드를 조금만 고쳐야 한다. 수정한 후에는 report.py와 pcost.py 프로그램이 이전과 같은 방식으로 작동해야 한다.

마지막 편집일시 : 2020년 6월 30일 9:56
==========

4. 클래스와 객체

지금까지 우리가 작성한 프로그램은 파이썬 빌트인 자료형만 사용했다. 이 섹션에서는 클래스와 객체 개념을 도입한다. 새로운 객체를 만들 수 있는 class 문을 배운다. 상속 개념도 소개하며, 확장 가능한 프로그램을 구축하는 데 공통적으로 사용하는 도구를 다룬다. 끝으로, 특수한 메소드, 동적 어트리뷰트 찾기, 새로운 예외 정의 등 클래스와 관련한 기능을 살펴본다.

- 4.1 클래스
- 4.2 상속
- 4.3 특수 메서드
- 4.4 새로운 예외 정의하기

4.1 클래스

이 섹션은 class 문을 소개하고 새로운 객체를 생성하는 아이디어를 제시한다.

객체 지향(Object Oriented, OO) 프로그래밍
코드를 객체의 모음으로서 조직화하는 프로그래밍 기법이다.

객체는 다음으로 구성된다.

데이터(Data). 어트리뷰트(Attribute)
행위(Behavior). 메서드(method) - 객체에 적용되는 함수.
이 코스에서 이미 객체지향을 사용했다.

예를 들어 리스트를 조작할 때,

```
>>> nums = [1, 2, 3]
>>> nums.append(4)        # 메서드
>>> nums.insert(1,10)    # 메서드
>>> nums
[1, 10, 2, 3, 4]          # 데이터
>>>
```

nums는 리스트의 인스턴스다.

메서드(append()와 insert())는 인스턴스(nums)에 붙어 있다.

class 문
class 문을 사용해 새로운 객체를 정의한다.

```
class Player:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
        self.health = 100

    def move(self, dx, dy):
        self.x += dx
        self.y += dy

    def damage(self, pts):
        self.health -= pts
```

간단히 말해서 클래스는 인스턴스라 부르는 것에 대한 여러 연산을 수행하는 함수의 집합체다.

인스턴스

인스턴스는 프로그램에서 다루는 실제 객체다.

클래스를 함수로서 호출하여 생성한다.

```
>>> a = Player(2, 3)
>>> b = Player(10, 20)
>>>
```

a와 b는 Player의 인스턴스다.

강조: class 문은 정의(definition)일 뿐이다(그 자체로는 아무 일도 하지 않는다). 함수 정의와 마찬가지다.

인스턴스 데이터

인스턴스에는 자체 로컬 데이터가 있다.

```
>>> a.x
2
>>> b.x
10
```

이 데이터는 __init__()에 의해 초기화된다.

```
class Player:
    def __init__(self, x, y):
        # `self`에 저장된 값은 인스턴스 데이터다
        self.x = x
        self.y = y
        self.health = 100
```

어트리뷰트의 개수나 유형에는 제한이 없다.

인스턴스 메서드
인스턴스 메서드는 객체의 인스턴스에 적용되는 함수다.

```
class Player:
    ...
    # `move`는 메서드다
    def move(self, dx, dy):
        self.x += dx
        self.y += dy
```

객체 그 자체가 항상 첫 번째 인자로 전달된다.

```
>>> a.move(1, 2)

# `a`를 `self`와 매치
# `1`을 `dx`와 매치
# `2`를 `dy`와 매치
def move(self, dx, dy):
```

관례적으로 인스턴스를 self라고 한다. 그러나 실제로 어떤 이름을 쓰는지는 중요하지 않다. 객체는 항상 첫 번째 인자로 전달된다. 이 인자를 self라고 부르는 것은 파이썬 프로그래밍 스타일일 뿐이다.

클래스 스코핑
클래스는 이름의 스코프를 정의하지 않는다.

```
class Player:
    ...
    def move(self, dx, dy):
        self.x += dx
        self.y += dy

    def left(self, amt):
        move(-amt, 0)        # NO. 글로벌 `move` 함수를 호출
        self.move(-amt, 0)  # YES. 위에 정의한 `move` 메서드를 호출.
```

인스턴스에 대한 연산을 하고 싶으면, 항상 명시적으로 참조한다(예: self).

연습 문제
지금부터 나오는 연습 문제는 이전 섹션에서 작성한 코드를 변경한다. 연습 문제 3.18의 코드가 올바로 작동하는 것을 가지고 시작해야 한다. 그렇지 못할 경우, Solutions/3_18 디렉터리의 해답 코드를 가지고 실습하라. 복사해서 써도 된다.

연습 문제 4.1: 데이터 구조로서의 객체

섹션 2와 3에서, 튜플과 딕셔너리로서 표현되는 데이터를 가지고 작업했다. 예를 들어, 보유 주식을 다음과 같이 튜플로 표현했다.

```
s = ('GOOG',100,490.10)
```

또는 다음과 같이 딕셔너리를 사용했다.

```
s = { 'name'   : 'GOOG',
      'shares' : 100,
      'price'  : 490.10
}
```

그러한 데이터를 조작하는 함수를 작성할 수도 있다. 예:

```
def cost(s):
    return s['shares'] * s['price']
```

그렇지만 프로그램이 커져감에 따라 조직화를 더 잘 하고 싶어질 것이다. 따라서 클래스를 정의함으로써 데이터를 표현하는 접근을 취할 수 있다. stock.py라는 파일을 만들어 보유 주식 한 종목을 표현하는 Stock 클래스를 정의하자. Stock 인스턴스에는 name, shares, price 어트리뷰트가 있다. 예:

```
>>> import stock
>>> a = stock.Stock('GOOG',100,490.10)
>>> a.name
'GOOG'
>>> a.shares
100
>>> a.price
490.1
>>>
```

Stock 객체를 몇 개 더 생성해 조작해보자. 예:

```
>>> b = stock.Stock('AAPL', 50, 122.34)
>>> c = stock.Stock('IBM', 75, 91.75)
>>> b.shares * b.price
6117.0
>>> c.shares * c.price
6881.25
>>> stocks = [a, b, c]
>>> stocks
[<stock.Stock object at 0x37d0b0>, <stock.Stock object at 0x37d110>, <stock.Stock object at 0x37d050>]
>>> for s in stocks:
```

```
    print(f'{s.name:>10s} {s.shares:>10d} {s.price:>10.2f}')

... 출력을 보라 ...
>>>
```

여기서 강조하고 싶은 것은 Stock 클래스가 객체의 인스턴스를 생성하는 팩토리처럼 작동한다는 점이다. 기본적으로, 그것을 함수로서 호출해 새로운 객체를 생성할 수 있다. 또한, 각 객체는 고유하다는 점이 중요하다. 객체가 갖는 데이터는 다른 객체가 생성될 때 갖게 되는 데이터와 별개다.

클래스에 의해 정의된 객체는, 구문은 좀 다르지만 딕셔너리와도 비슷하다. 예를 들어, s['name']과 s['price'] 대신, s.name과 s.price라고 작성한다.

연습 문제 4.2: 메서드를 추가하기

클래스에서는 객체에 함수를 붙일 수 있는데, 이를 메서드라 한다. 메서드는 객체 내에 저장된 데이터를 조작하는 함수다. Stock 객체에 cost()와 sell() 메서드를 추가하자. 다음과 같이 작동한다.

```
>>> import stock
>>> s = stock.Stock('GOOG', 100, 490.10)
>>> s.cost()
49010.0
>>> s.shares
100
>>> s.sell(25)
>>> s.shares
75
>>> s.cost()
36757.5
>>>
```

연습 문제 4.3: 인스턴스의 리스트를 생성하기

딕셔너리의 리스트로부터 Stock 인스턴스의 리스트를 만들어 보자. 그런 다음 총 비용을 계산한다.

```
>>> import fileparse
>>> with open('Data/portfolio.csv') as lines:
...         portdicts = fileparse.parse_csv(lines, select=['name','shares','price'],
types=[str,int,float])
...
>>> portfolio = [ stock.Stock(d['name'], d['shares'], d['price']) for d in
portdicts]
>>> portfolio
```

```
[<stock.Stock object at 0x10c9e2128>, <stock.Stock object at 0x10c9e2048>, <stock.Stock object at 0x10c9e2080>,
 <stock.Stock object at 0x10c9e25f8>, <stock.Stock object at 0x10c9e2630>, <stock.Stock object at 0x10ca6f748>,
 <stock.Stock object at 0x10ca6f7b8>]
>>> sum([s.cost() for s in portfolio])
44671.15
>>>
```

연습 문제 4.4: 클래스를 사용하기

포트폴리오를 읽어 연습 문제 4.3에 보인 것과 같은 Stock 인스턴스의 리스트를 만들도록report.py 프로그램의 read_portfolio() 함수를 수정하라. 완료하면, 딕셔너리 대신 Stock 인스턴스를 사용하게 report.py와 pcost.py의 코드를 모두 수정한다.

힌트: 코드를 너무 많이 수정하면 안 된다. 딕셔너리 액세스 s['shares']를 s.shares로 바꾸는 것과 같이 소소한 변경을 한다.

이전과 같은 방법으로 함수를 실행할 수 있어야 한다.

```
>>> import pcost
>>> pcost.portfolio_cost('Data/portfolio.csv')
44671.15
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84
>>>
```

마지막 편집일시 : 2020년 10월 4일 12:17

4.2 상속

상속은 확장성 있는 프로그램을 작성하는 일반적인 도구다. 이 섹션에서 그 개념을 알아보자.

도입

상속은 기존 객체를 특수화하는 데 사용된다.

```
class Parent:
    ...

class Child(Parent):
    ...
```

위의 Child 클래스와 같은 것을 파생된 클래스(derived class) 또는 하위 클래스(subclass)라 한다. 위의 Parent 클래스는 기본 클래스(base class) 또는 상위 클래스(superclass)라 한다. class Child(Parent):에서 클래스명 뒤의 ()에 Parent를 지정했다.

확장(Extending)
상속을 통해 기존 클래스를 취하여 다음과 같은 일을 할 수 있다.

새로운 메서드를 추가
기존 메서드 일부를 재정의(redefine)
인스턴스에 새로운 어트리뷰트를 추가
그리하여 기존 코드를 확장하게 된다.

예
이 클래스에서 시작한다고 하자.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price

    def cost(self):
        return self.shares * self.price

    def sell(self, nshares):
        self.shares -= nshares
```

상속을 통해 이것의 어느 부분이든 변경할 수 있다.

새 메서드를 추가

```
class MyStock(Stock):
    def panic(self):
        self.sell(self.shares)
```

용례:

```
>>> s = MyStock('GOOG', 100, 490.1)
>>> s.sell(25)
>>> s.shares
75
>>> s.panic()
>>> s.shares
0
>>>
```

기존 메서드를 재정의

```
class MyStock(Stock):
    def cost(self):
        return 1.25 * self.shares * self.price
```

용례:

```
>>> s = MyStock('GOOG', 100, 490.1)
>>> s.cost()
61262.5
>>>
```

새 메서드는 기존 것을 대체한다. 다른 메서드는 영향을 받지 않는다. 굉장하다.

오버라이딩(Overriding)

때로는 클래스가 기존 메서드를 확장하되, 원래 구현을 재정의에 포함하고 싶을 수 있다. 이를 위해 super()를 사용한다.

```
class Stock:
    ...
    def cost(self):
        return self.shares * self.price
    ...

class MyStock(Stock):
    def cost(self):
        # `super`에 대한 호출을 확인
        actual_cost = super().cost()
        return 1.25 * actual_cost
```

이전 버전을 호출하기 위해 super()를 사용한다.

주의: 파이썬 2는 이것보다 구문이 복잡하다.

```
actual_cost = super(MyStock, self).cost()
```

__init__와 상속
__init__를 재정의하려면 부모를 초기화해야 한다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price

class MyStock(Stock):
    def __init__(self, name, shares, price, factor):
        # `super`와 `__init__`에 대한 호출을 확인
        super().__init__(name, shares, price)
        self.factor = factor

    def cost(self):
        return self.factor * super().cost()
```

super에 __init__() 메서드를 호출해야 한다. 이것이 앞에서 본 것과 같이 이전 버전을 호출하는 방법이다.

상속을 사용하기
상속은 관련 객체를 조직화하는 데 사용되곤 한다.

```
class Shape:
    ...

class Circle(Shape):
    ...

class Rectangle(Shape):
    ...
```

논리적 계층 구조나 분류법을 생각해 보라. 그렇지만 좀 더 일반적이고도 실용적인 용도는 재사용 가능 혹은 확장 가능한 코드를 만드는 것이다. 예를 들어, 프레임워크에 정의된 기본 클래스를 당신이 커스터마이즈할 수 있다.

```
class CustomHandler(TCPHandler):
    def handle_request(self):
        ...
        # 커스텀 처리
```

기본 클래스는 일반 목적 코드를 포함한다. 당신의 클래스는 기본 클래스를 상속해 특수한 부분을 커스터마이즈한다.

"is a" 관계
상속은 타입 간의 관계를 설정한다.

```
class Shape:
    ...

class Circle(Shape):
    ...
```

객체 인스턴스를 확인하라.

```
>>> c = Circle(4.0)
>>> isinstance(c, Shape)
True
>>>
```

중요: 부모 클래스의 인스턴스를 다루는 코드는 자식 클래스의 인스턴스에 대해서도 작동하는 것이 이상적이다.

object 기본 클래스
만약 클래스에 부모가 없으면 object를 기본으로 삼을 수 있다.

```
class Shape(object):
    ...
```

파이썬에서 object는 모든 객체의 부모다.

*참고: 기술적으로 이것이 필요하지는 않지만, 파이썬 2에서 필요했기 때문에 남겨둔 경우가 많다. object를 생략하더라도 암시적으로 상속한다.

다중 상속
여러 클래스로부터 상속하도록 클래스에 정의할 수 있다.

```
class Mother:
    ...

class Father:
    ...

class Child(Mother, Father):
    ...
```

Child 클래스는 Mother와 Father의 특징을 모두 상속받는다. 다중 상속은 세부적으로 들어가면 골치가 아파진다. 정확히 이해하기 전에는 사용하지 마라. 다음 섹션에서 추

가 정보를 제공하겠지만, 이 코스에서 다중 상속을 사용할 일은 없다.

연습 문제

상속의 주 용도는 다양한 방법으로 확장 또는 커스터마이즈된 코드를 작성하는 것에 있다(특히 라이브러리나 프레임워크에서). 설명을 위해 report.py 프로그램의 print_report() 함수를 생각해 보자. 그것은 다음과 같이 보일 것이다.

```
def print_report(reportdata):
    '''
    (name, shares, price, change) 튜플의 리스트로부터 보기 좋게 포매팅한 테이블을 프린팅.
    '''
    headers = ('Name','Shares','Price','Change')
    print('%10s %10s %10s %10s' % headers)
    print(('-'*10 + ' ')*len(headers))
    for row in reportdata:
        print('%10s %10d %10.2f %10.2f' % row)
```

report 프로그램을 실행하면 다음과 같이 출력한다.

```
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
      Name     Shares      Price     Change
---------- ---------- ---------- ----------
        AA        100       9.22     -22.98
       IBM         50     106.28      15.18
       CAT        150      35.46     -47.98
      MSFT        200      20.89     -30.34
        GE         95      13.48     -26.89
      MSFT         50      20.89     -44.21
       IBM        100     106.28      35.84
```

연습 문제 4.5: 확장성 문제

print_report() 함수가 일반 텍스트, HTML, CSV, XML 같이 다양한 출력 포맷을 지원하게 수정하고 싶다고 하자. 그렇게 하려고 모든 일을 처리하는 아주 큰 함수를 작성하려고 시도할 수 있다. 하지만 그렇게 했다가는 유지보수를 하기 힘들만큼 지저분해질 수 있다. 이때가 상속을 사용하기에 아주 좋은 기회다.

시작하려면 테이블을 생성하는 단계에 집중하자. 테이블의 첫 행에는 헤더가 있다. 그 후, 테이블 데이터 행들이 나타난다. 다음 단계를 따라 자체 클래스에 집어넣자. 다음과 같은 클래스를 정의하는 tableformat.py 파일을 작성한다.

```
# tableformat.py
```

```
class TableFormatter:
    def headings(self, headers):
        '''
        테이블 헤딩을 반환.
        '''
    raise NotImplementedError()

    def row(self, rowdata):
        '''
        테이블 데이터의 단일 행을 반환.
        '''
    raise NotImplementedError()
```

이 클래스는 아무 일도 하지 않지만, 곧 정의할 추가 클래스의 설계 사양과 역할을 한다. 이러한 클래스를 "추상 기본 클래스(abstract base class)"라고 부른다.

입력으로 받은 TableFormatter 객체의 메서드를 호출해 출력을 생성하도록 print_report() 함수를 수정하자. 예를 들면 다음과 같다.

```
# report.py
...

def print_report(reportdata, formatter):
    '''
    (name, shares, price, change) 튜플의 리스트로부터 보기 좋게 포매팅한 테이블을 프린팅.
    '''
    formatter.headings(['Name','Shares','Price','Change'])
    for name, shares, price, change in reportdata:
        rowdata = [ name, str(shares), f'{price:0.2f}', f'{change:0.2f}' ]
        formatter.row(rowdata)
```

print_report()에 인자를 추가했으므로, portfolio_report() 함수도 수정해야 한다. TableFormatter를 생성하게 변경한다.

```
# report.py

import tableformat

...
def portfolio_report(portfoliofile, pricefile):
    '''
```

```
    주어진 포트폴리오와 가격 데이터 파일을 가지고 주식 보고서를 작성.
    '''
    # 데이터 파일 읽기
    portfolio = read_portfolio(portfoliofile)
    prices = read_prices(pricefile)

    # 보고서 데이터 생성
    report = make_report_data(portfolio, prices)

    # 프린트
    formatter = tableformat.TableFormatter()
    print_report(report, formatter)
```

새 코드를 실행하자.

```
>>>                ================================                RESTART
================================
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
... 충돌 ...
```

곧바로 NotImplementedError 예외를 내며 충돌한다. 썩 즐겁지는 않지만, 이것은 예상했던 바다. 다음 부분으로 이어 가자.

연습 문제 4.6: 상속을 통해 다른 출력을 생성하기

(a) 부분에서 정의한 TableFormatter 클래스는 상속을 통해 확장할 의도를 가지고 정의했다. 사실, 그것이 아이디어의 전부다. 다음과 같이 TextTableFormatter 클래스를 정의한다.

```
# tableformat.py
...
class TextTableFormatter(TableFormatter):
    '''
    테이블을 일반 텍스트 포맷으로 출력
    '''
    def headings(self, headers):
        for h in headers:
            print(f'{h:>10s}', end=' ')
        print()
        print(('-'*10 + ' ')*len(headers))

    def row(self, rowdata):
        for d in rowdata:
```

```
            print(f'{d:>10s}', end=' ')
        print()
```

portfolio_report() 함수를 다음과 같이 수정하고 사용해 보자.

```
# report.py
...
def portfolio_report(portfoliofile, pricefile):
    '''
    주어진 포트폴리오와 가격 데이터 파일을 가지고 주식 보고서를 작성.
    '''
    # 데이터 파일 읽기
    portfolio = read_portfolio(portfoliofile)
    prices = read_prices(pricefile)

    # 보고서 데이터 생성
    report = make_report_data(portfolio, prices)

    # 프린트
    formatter = tableformat.TextTableFormatter()
    print_report(report, formatter)
```

이전과 같은 출력을 생성해야 한다.

```
>>>                 ================================                RESTART
================================
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
       Name      Shares       Price      Change
---------- ---------- ---------- ----------
         AA        100       9.22      -22.98
        IBM         50     106.28       15.18
        CAT        150      35.46      -47.98
       MSFT        200      20.89      -30.34
         GE         95      13.48      -26.89
       MSFT         50      20.89      -44.21
        IBM        100     106.28       35.84
>>>
```

하지만, 출력을 다른 것으로 바꾸자. 출력을 CSV 포맷으로 출력하는 새로운 클래스 CSVTableFormatter를 정의한다.

```
# tableformat.py
...
```

```
class CSVTableFormatter(TableFormatter):
    '''
    포트폴리오 데이터를 CSV 포맷으로 출력.
    '''
    def headings(self, headers):
        print(','.join(headers))

    def row(self, rowdata):
        print(','.join(rowdata))
```

메인 프로그램을 다음과 같이 수정한다.

```
def portfolio_report(portfoliofile, pricefile):
    '''
    주어진 포트폴리오와 가격 데이터 파일을 가지고 주식 보고서를 작성.
    '''
    # 데이터 파일 읽기
    portfolio = read_portfolio(portfoliofile)
    prices = read_prices(pricefile)

    # 보고서 데이터 생성
    report = make_report_data(portfolio, prices)

    # 프린트
    formatter = tableformat.CSVTableFormatter()
    print_report(report, formatter)
```

이제 다음과 같은 CSV 출력을 볼 수 있을 것이다.

```
>>>                ================================                RESTART
================================
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
Name,Shares,Price,Change
AA,100,9.22,-22.98
IBM,50,106.28,15.18
CAT,150,35.46,-47.98
MSFT,200,20.89,-30.34
GE,95,13.48,-26.89
MSFT,50,20.89,-44.21
IBM,100,106.28,35.84
```

마찬가지로, 다음과 같은 출력을 생성하는 HTMLTableFormatter 클래스를 정의한다.

<tr><th>Name</th><th>Shares</th><th>Price</th><th>Change</th></tr>
<tr><td>AA</td><td>100</td><td>9.22</td><td>-22.98</td></tr>
<tr><td>IBM</td><td>50</td><td>106.28</td><td>15.18</td></tr>
<tr><td>CAT</td><td>150</td><td>35.46</td><td>-47.98</td></tr>
<tr><td>MSFT</td><td>200</td><td>20.89</td><td>-30.34</td></tr>
<tr><td>GE</td><td>95</td><td>13.48</td><td>-26.89</td></tr>
<tr><td>MSFT</td><td>50</td><td>20.89</td><td>-44.21</td></tr>
<tr><td>IBM</td><td>100</td><td>106.28</td><td>35.84</td></tr>
CSVTableFormatter 객체 대신 HTMLTableFormatter 객체를 생성하게 메인 프로그램을 수정해 코드를 테스트하자.

연습 문제 4.7: 다형성
객체 지향 프로그래밍의 주된 특징은 기존 코드를 변경할 필요 없이 객체를 프로그램에 끼워넣을 수 있다는 것이다. 예를 들어, TableFormatter 객체를 사용할 것으로 예상되는 프로그램을 작성했다고 하면, 어떤 종류의 TableFormatter를 주는지에 관계없이 작동할 것이다. 이러한 행위를 '다형성(polymorphism)'이라 한다.

한 가지 잠재적인 문제는 사용자가 원하는 포매터를 선택하게 하는 방법을 알아내는 것이다. TextTableFormatter 같은 클래스명을 직접 사용하게 하는 것은 종종 성가시다. 따라서 단순한 접근을 고려할 수 있다. 다음과 같이 코드에 if 문을 포함할 수 있을 것이다.

```
def portfolio_report(portfoliofile, pricefile, fmt='txt'):
    '''
    주어진 포트폴리오와 가격 데이터 파일을 가지고 주식 보고서를 작성.
    '''
    # 데이터 파일 읽기
    portfolio = read_portfolio(portfoliofile)
    prices = read_prices(pricefile)

    # 보고서 데이터 생성
    report = make_report_data(portfolio, prices)

    # 프린트
    if fmt == 'txt':
        formatter = tableformat.TextTableFormatter()
    elif fmt == 'csv':
        formatter = tableformat.CSVTableFormatter()
    elif fmt == 'html':
        formatter = tableformat.HTMLTableFormatter()
    else:
```

```
        raise RuntimeError(f'Unknown format {fmt}')
    print_report(report, formatter)
```

이 코드에서, 사용자는 'txt'나 'csv' 같이 단순화된 이름을 지정함으로써 포맷을 선택한다. 하지만, portfolio_report() 함수에 커다란 if 문을 넣는 것이 최선인가? 그 코드를 다른 범용 함수로 옮기는 것이 나을 것이다.

'txt', 'csv', 'html' 같은 출력 이름에 따라 사용자가 포매터를 생성하는 create_formatter(name) 함수를 tableformat.py 파일에 추가한다. portfolio_report()를 다음과 같이 수정하자.

```
def portfolio_report(portfoliofile, pricefile, fmt='txt'):
    '''
    주어진 포트폴리오와 가격 데이터 파일을 가지고 주식 보고서를 작성.
    '''
    # 데이터 파일 읽기
    portfolio = read_portfolio(portfoliofile)
    prices = read_prices(pricefile)

    # 보고서 데이터 생성
    report = make_report_data(portfolio, prices)

    # 프린트
    formatter = tableformat.create_formatter(fmt)
    print_report(report, formatter)
```

여러 포맷으로 함수를 호출해 함수가 제대로 작동하는지 확인해 보자.

연습 문제 4.8: 모두 합치기

portfolio_report() 함수가 출력 포맷을 지정하는 선택적 인자를 받게 report.py 프로그램을 수정하라. 예:

```
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv', 'txt')
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84
>>>
```

명령행에서 포맷을 지정하게 메인 프로그램을 수정한다.

```
bash $ python3 report.py Data/portfolio.csv Data/prices.csv csv
Name,Shares,Price,Change
AA,100,9.22,-22.98
IBM,50,106.28,15.18
CAT,150,35.46,-47.98
MSFT,200,20.89,-30.34
GE,95,13.48,-26.89
MSFT,50,20.89,-44.21
IBM,100,106.28,35.84
bash $
```

논의

라이브러리와 프레임워크에서 확장성 있는 코드를 작성하는 것은 상속의 주된 용도다. 예를 들어, 프레임워크는 기본 클래스를 제공하고 당신은 그것을 상속해 객체를 정의한다. 그런 다음 기능을 구현하는 다양한 메서드를 채운다.

또 한 가지 심오한 개념은 '추상화를 직접 작성'하는 것이다. 우리는 이 연습 문제에서 테이블을 포매팅하는 클래스를 직접 정의했다. 코드를 들여다보고 '포매팅 라이브러리라든지 다른 사람이 이미 만들어놓은 것을 사용해야겠다'고 생각할지 모르겠다. 하지만 직접 만든 클래스와 라이브러리를 둘 다 사용해야 한다. 스스로 클래스를 만들면 커플링이 줄어들고 더 유연해진다. 당신의 애플리케이션이 당신의 클래스의 프로그래밍 인터페이스를 사용하는 한, 내부 구현을 원하는 대로 변경할 수 있다. 완전한 맞춤 코드를 작성할 수 있다. 다른 사람이 만든 서드 파티 패키지를 사용할 수 있다. 사용하던 서드 파티 패키지보다 더 좋은 것을 찾게 되면 다른 것으로 바꿀 수 있다. 패키지를 교체하는 것이 별 문제가 되지 않을 것이다. 인터페이스를 유지하는 한 애플리케이션 코드가 전혀 깨지지 않을 것이기 때문이다. 그것은 강력한 아이디어이며, 이와 같은 상속을 고려해야 하는 이유다.

객체 지향 프로그램 설계는 매우 어려울 수 있다. 더 알고 싶으면 디자인 패턴을 다루는 책을 살펴보기 바란다(이 연습 문제에서 일어난 일을 이해하면 객체를 실용적으로 다루는 데 상당한 도움이 된다).

마지막 편집일시 : 2020년 10월 3일 5:44

4.3 특수한 메서드

특수한 메서드를 사용함으로써, 파이썬의 행위를 여러 면에서 커스터마이즈할 수 있다. 이러한 특수 메서드를 "마법(magic)" 메서드라고도 부른다. 이 섹션에서 그 개념을 알아보자. 동적 어트리뷰트 액세스와 바운드 메서드도 논의한다.

도입

클래스에 특수 메서드를 정의할 수 있다. 파이썬 인터프리터는 이러한 메서드들을 특별하게 다룬다. 항상 앞에 __가 붙는다. (예: __init__)

```
class Stock(object):
    def __init__(self):
        ...
    def __repr__(self):
        ...
```

특수 메서드는 많이 있지만, 그중 몇 가지만 예를 들어 보자.

문자열 변환을 위한 특수 메서드
객체에는 두 가지 문자열 표현이 있다.

```
>>> from datetime import date
>>> d = date(2012, 12, 21)
>>> print(d)
2012-12-21
>>> d
datetime.date(2012, 12, 21)
>>>
```

str() 함수를 사용해 멋지게 프린트할 수 있는 출력을 생성한다.

```
>>> str(d)
'2012-12-21'
>>>
```

repr() 함수는 개발자를 위한 상세한 표현에 사용된다.

```
>>> repr(d)
'datetime.date(2012, 12, 21)'
>>>
```

이러한 str()과 repr() 함수는 특수한 메서드를 사용해, 클래스로부터 문자열을 생성한다.

```
class Date(object):
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    # Used with `str()`
    def __str__(self):
```

```
        return f'{self.year}-{self.month}-{self.day}'

    # `repr()`과 함께 사용
    def __repr__(self):
        return f'Date({self.year},{self.month},{self.day})'
```

참고: __repr__()이 반환하는 문자열을 eval()에 전달하면 객체를 다시 만들 수 있도록 하는 것이 관례다. 그렇게 하는 것이 불가능하다면, 그 대신 읽기 쉬운 표현을 사용한다.

수학을 위한 특수 메서드

다음과 같은 메서드가 수학 연산에 사용된다.

```
a + b        a.__add__(b)
a - b        a.__sub__(b)
a * b        a.__mul__(b)
a / b        a.__truediv__(b)
a // b       a.__floordiv__(b)
a % b        a.__mod__(b)
a << b       a.__lshift__(b)
a >> b       a.__rshift__(b)
a & b        a.__and__(b)
a | b        a.__or__(b)
a ^ b        a.__xor__(b)
a ** b       a.__pow__(b)
-a           a.__neg__()
~a           a.__invert__()
abs(a)       a.__abs__()
```

항목 접근을 위한 특수 메서드

컨테이너를 구현하는 데 다음과 같은 메서드를 사용한다.

```
len(x)       x.__len__()
x[a]         x.__getitem__(a)
x[a] = v     x.__setitem__(a,v)
del x[a]     x.__delitem__(a)
```

이것들을 다음과 같이 클래스에서 사용할 수 있다.

```
class Sequence:
    def __len__(self):
        ...
    def __getitem__(self,a):
        ...
```

```
    def __setitem__(self,a,v):
        ...
    def __delitem__(self,a):
        ...
```

메서드 호출

메서드 호출은 두 단계 과정으로 이뤄진다.

메서드가 있는지 확인(lookup): . 연산자

메서드 호출: () 연산자

```
>>> s = Stock('GOOG',100,490.10)
>>> c = s.cost  # 메서드 확인
>>> c
<bound method Stock.cost of <Stock object at 0x590d0>>
>>> c()          # 메서드 호출
49010.0
>>>
```

바운드 메서드

함수 호출 연산자 ()에 의해 호출되지 않은 메서드를 바운드 메서드(bound method)라 한다. 바운드 메서드는 그것이 어느 인스턴스에서 왔는지를 반환한다.

```
>>> s = Stock('GOOG', 100, 490.10)
>>> s
<Stock object at 0x590d0>
>>> c = s.cost
>>> c
<bound method Stock.cost of <Stock object at 0x590d0>>
>>> c()
49010.0
>>>
```

바운드 메서드는 부주의하고 명백하지 않은 오류의 원인이 되곤 한다. 예:

```
>>> s = Stock('GOOG', 100, 490.10)
>>> print('Cost : %0.2f' % s.cost)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: float argument required
>>>
```

또는 디버깅하기 힘든 잘못된 행동을 한다.

```
f = open(filename, 'w')
...
```

f.close # 아차, 아무 일도 하지 않았다. `f`는 여전히 열려 있다.
두 경우 모두 괄호를 붙이는 것을 잊었기 때문에 오류가 발생했다. 예: s.cost() 또는 f.close().

어트리뷰트 액세스
어트리뷰트에 액세스, 조작, 관리하는 또 다른 방법이 있다.

```
getattr(obj, 'name')          # obj.name과 같음
setattr(obj, 'name', value)   # obj.name = value와 같음
delattr(obj, 'name')          # del obj.name과 같음
hasattr(obj, 'name')          # 어트리뷰트가 존재하는지 테스트
```
예:

```
if hasattr(obj, 'x'):
    x = getattr(obj, 'x'):
else:
    x = None
```
참고: getattr()에도 유용한 기본값 arg가 있다.

```
x = getattr(obj, 'x', None)
```
연습 문제
연습 문제 4.9: 객체를 프린팅하기 위한 더 나은 출력
stock.py에 정의한 Stock 객체를 수정해 __repr__() 메서드가 더 유용한 출력을 생성하게 해 보자. 예:

```
>>> goog = Stock('GOOG', 100, 490.1)
>>> goog
Stock('GOOG', 100, 490.1)
>>>
```
이러한 변경을 한 후에 주식 포트폴리오를 읽고 결과 리스트를 볼 때 무슨 일이 일어나는지 확인하라. 예:

```
>>> import report
>>> portfolio = report.read_portfolio('Data/portfolio.csv')
>>> portfolio
... see what the output is ...
>>>
```
연습 문제 4.10: getattr()을 사용하는 예
getattr()은 어트리뷰트를 읽는 대체 메커니즘이다. 이것을 사용해 극도로 유연한 코드를 작성할 수 있다. 다음 예제부터 시도하자.

```
>>> import stock
>>> s = stock.Stock('GOOG', 100, 490.1)
>>> columns = ['name', 'shares']
>>> for colname in columns:
        print(colname, '=', getattr(s, colname))

name = GOOG
shares = 100
>>>
```

출력 데이터가 columns 변수에 나열된 어트리뷰트 이름에 의해 완전히 결정되는 것을 주의깊게 관찰하라.

이 아이디어를 tableformat.py 파일의 일반화된 함수 print_table()에 적용한다. 이 함수는 임의의 객체의 리스트의 사용자 정의 어트리뷰트를 보여주는 테이블을 프린트한다. 이전의 print_report() 함수와 마찬가지로, print_table()은 출력 포맷을 지정하는 TableFormatter 인스턴스를 받는다. 다음과 같이 실행한다.

```
>>> import report
>>> portfolio = report.read_portfolio('Data/portfolio.csv')
>>> from tableformat import create_formatter, print_table
>>> formatter = create_formatter('txt')
>>> print_table(portfolio, ['name','shares'], formatter)
      name      shares
---------- ----------
        AA         100
       IBM          50
       CAT         150
      MSFT         200
        GE          95
      MSFT          50
       IBM         100

>>> print_table(portfolio, ['name','shares','price'], formatter)
      name      shares       price
---------- ---------- ----------
        AA         100        32.2
       IBM          50        91.1
       CAT         150       83.44
      MSFT         200       51.23
        GE          95       40.37
      MSFT          50        65.1
```

```
        IBM          100        70.44
>>>
```
마지막 편집일시 : 2020년 9월 9일 8:39

4.4 예외 정의하기

사용자 정의 예외는 클래스에 의해 정의된다.

```
class NetworkError(Exception):
    pass
```
예외는 항상 Exception으로부터 상속한다.

보통 그것들은 빈 클래스다. 몸체에 pass를 사용한다.

예외 계층을 직접 만들 수 있다.

```
class AuthenticationError(NetworkError):
     pass

class ProtocolError(NetworkError):
    pass
```
연습 문제

연습 문제 4.11: 커스텀 예외 정의하기

라이브러리에 자체적인 예외를 정의하는 것이 좋다.

공통적인 프로그래밍 오류로 인해 발생하는 파이썬 예외와 라이브러리 내부적으로 발생해 특정한 사용상의 문제를 가리키는 예외를 구분하기 쉬워진다.

지난 연습 문제의 create_formatter() 함수를 수정해, 사용자가 잘못된 포맷 이름을 제공했을 때 커스텀 FormatError 예외를 일으키게 해 보자.

예:

```
>>> from tableformat import create_formatter
>>> formatter = create_formatter('xls')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "tableformat.py", line 71, in create_formatter
    raise FormatError('Unknown table format %s' % name)
FormatError: Unknown table format xls
>>>
```
마지막 편집일시 : 2020년 9월 9일 8:41

---

5. 파이썬 객체의 내부 작동

이 섹션은 파이썬 객체의 내부 작동을 다룬다. 다른 프로그래밍 언어를 다뤄본 프로그래머라면 파이썬의 클래스 개념이 부족하다는 것을 발견하곤 한다. 예를 들어, 접근 제어(예: private, protected)가 없고, self 인자도 이상해 보이고, 객체 관련 작업은 "무엇이든 자유롭게" 할 수 있는 것처럼 느껴진다. 그것이 사실일 수도 있지만, 일반적인 프로그래밍 방식에서부터 객체 내부의 더 나은 캡슐화까지 어떻게 작동하는지 알게 될 것이다.
내부를 상세히 알지 못한다고 해서 생산성이 떨어지는 것은 아니다. 그렇지만, 파이썬 코더 대부분은 클래스가 작동하는 기본 원리를 이해한다. 그러므로 우리도 한번 알아보자.

- 5.1 딕셔너리 톺아보기(객체 구현)
- 5.2 캡슐화 기법

5.1 딕셔너리 톺아보기

파이썬 객체 시스템은 딕셔너리와 관련된 구현에 많은 부분 기반을 둔다. 이 섹션에서 그에 대해 논의한다.

딕셔너리

딕셔너리가 이름 붙은 값들의 컬렉션이라는 것을 상기하자.

```
stock = {
    'name' : 'GOOG',
    'shares' : 100,
    'price' : 490.1
}
```

딕셔너리는 주로 단순한 자료 구조를 위해 사용된다. 하지만, 인터프리터에서 중요하게 사용되며 파이썬에서 가장 중요한 자료형이다.

딕셔너리와 모듈

모듈에서, 딕셔너리는 모든 글로벌 변수와 함수를 보유한다.

```
# foo.py

x = 42
def bar():
    ...

def spam():
    ...
```

foo.__dict__나 globals()를 조사해보면 딕셔너리가 보일 것이다.

```
{
```

```
    'x' : 42,
    'bar' : <function bar>,
    'spam' : <function spam>
}
```

딕셔너리와 객체

사용자 정의 객체도 인스턴스 데이터와 클래스를 위해 딕셔너리를 사용한다. 사실, 객체 시스템 대부분이 딕셔너리 위에 세워진 추가적인 계층이라 할 수 있다.

인스턴스 데이터 __dict__는 딕셔너리다.

```
>>> s = Stock('GOOG', 100, 490.1)
>>> s.__dict__
{'name' : 'GOOG', 'shares' : 100, 'price': 490.1 }
```

self에 할당할 때 이 딕셔너리와 인스턴스를 채운다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price
```

인스턴스 데이터 self.__dict__는 다음과 같이 보인다.

```
{
    'name': 'GOOG',
    'shares': 100,
    'price': 490.1
}
```

인스턴스마다 자체적인 프라이빗 딕셔너리가 생긴다.

```
s = Stock('GOOG', 100, 490.1)      # {'name' : 'GOOG','shares' : 100, 'price': 490.1 }
t = Stock('AAPL', 50, 123.45)      # {'name' : 'AAPL','shares' : 50, 'price': 123.45 }
```

어떤 클래스의 인스턴스를 100개 만들었다고 하면, 데이터를 담는 딕셔너리 100개가 있는 셈이다.

클래스 멤버

메서드를 보유하는 독립적인 딕셔너리가 있다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
```

```
        self.price = price

    def cost(self):
        return self.shares * self.price

    def sell(self, nshares):
        self.shares -= nshares
```

이 딕셔너리는 Stock.__dict__에 있다.

```
{
    'cost': <function>,
    'sell': <function>,
    '__init__': <function>
}
```

인스턴스와 클래스

인스턴스와 클래스는 서로 연결돼 있다. __class__ 어트리뷰트는 클래스를 다시 참조한다.

```
>>> s = Stock('GOOG', 100, 490.1)
>>> s.__dict__
{ 'name': 'GOOG', 'shares': 100, 'price': 490.1 }
>>> s.__class__
<class '__main__.Stock'>
>>>
```

인스턴스 딕셔너리는 각 인스턴스에 대해 고유한 데이터를 갖는다. 클래스 딕셔너리는 전체 인스턴스에서 공유하는 데이터를 갖는다.

어트리뷰트 액세스

객체로 작업할 때는 . 연산자를 사용해 데이터와 메서드에 액세스한다.

```
x = obj.name          # 얻기
obj.name = value      # 설정
del obj.name          # 삭제
```

이 연산자는 딕셔너리에 직접 연결된다.

인스턴스 수정

객체를 수정하는 연산은 딕셔너리를 갱신한다.

```
>>> s = Stock('GOOG', 100, 490.1)
>>> s.__dict__
{ 'name':'GOOG', 'shares': 100, 'price': 490.1 }
>>> s.shares = 50        # 설정
```

```
>>> s.date = '6/7/2007' # 설정
>>> s.__dict__
{ 'name': 'GOOG', 'shares': 50, 'price': 490.1, 'date': '6/7/2007' }
>>> del s.shares          # 삭제
>>> s.__dict__
{ 'name': 'GOOG', 'price': 490.1, 'date': '6/7/2007' }
>>>
```

어트리뷰트를 읽기

인스턴스에서 어트리뷰트를 읽는다고 하자.

```
x = obj.name
```

어트리뷰트는 두 곳에 존재할 수 있다.

로컬 인스턴스 딕셔너리.

클래스 딕셔너리.

두 딕셔너리를 모두 확인한다. 먼저 로컬 __dict__를 확인한다. 거기서 찾을 수 없으면 __class__를 통해 클래스의 __dict__에서 찾는다.

```
>>> s = Stock(...)
>>> s.name
'GOOG'
>>> s.cost()
49010.0
>>>
```

이러한 조회 방식으로 인해 class의 멤버가 모든 인스턴스에서 공유된다.

상속이 이뤄지는 원리

클래스는 다른 클래스로부터 상속할 수 있다.

```
class A(B, C):
    ...
```

기본 클래스는 각 클래스의 튜플에 저장된다.

```
>>> A.__bases__
(<class '__main__.B'>, <class '__main__.C'>)
>>>
```

이것은 부모 클래스에 대한 링크를 제공한다.

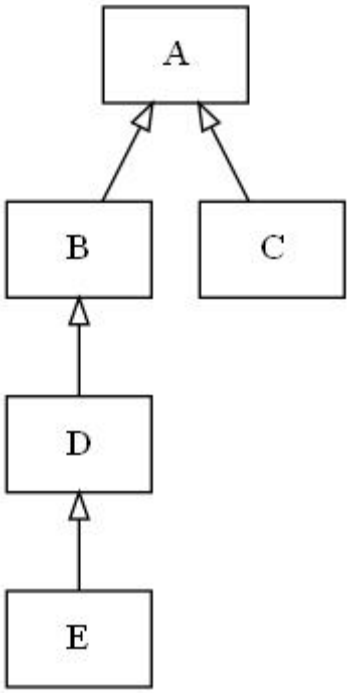
상속과 어트리뷰트 읽기

논리적으로, 어트리뷰트를 찾는 과정은 다음과 같다. 먼저, 로컬 __dict__를 확인한다. 찾지 못하면, 클래스의 __dict__를 확인한다. 클래스에서 찾지 못하면, __bases__를 통해 기본 클

래스들에서 확인한다. 그런데 여기에는 미묘한 구석이 있다.

단일 상속과 어트리뷰트 읽기
여러 계층에 걸쳐 상속이 이뤄지는 경우, 어트리뷰트를 찾기 위해 상속 트리를 차례대로 순회한다.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B): pass
class E(D): pass
```



단일 상속에서는 최상위 클래스로 가는 경로가 하나 밖에 없다. 첫 번째 일치하는 것에서 멈춘다.

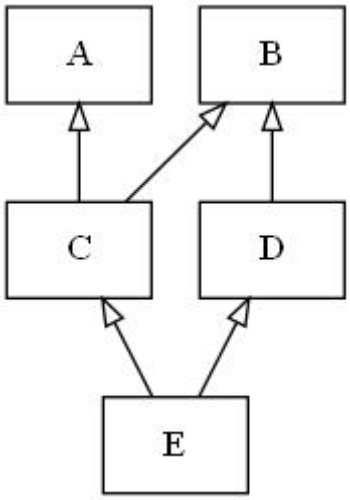
메서드 찾기 순서(MRO)
파이썬은 상속 사슬을 미리 계산해 클래스의 MRO 어트리뷰트에 저장한다. 그것을 직접 볼 수 있다.

```
>>> E.__mro__
(<class '__main__.E'>, <class '__main__.D'>,
 <class '__main__.B'>, <class '__main__.A'>,
 <type 'object'>)
>>>
```

이 사슬을 메서드 찾기 순서(Method Resolution Order)라 한다. 어트리뷰트를 찾기 위해 파이썬은 MRO를 순회한다. 첫 번째 일치하는 것이 선택된다.

다중 상속의 MRO
다중 상속에서는 최상위에 이르는 단일 경로가 없다. 예를 살펴보자.

```
class A: pass
class B: pass
class C(A, B): pass
class D(B): pass
class E(C, D): pass
```



어트리뷰트에 액세스할 때 무슨 일이 일어나는가?

```
e = E()
e.attr
```

어트리뷰트 검색 과정이 수행되는데, 그 순서는 어떻게 이뤄지는가? 그것이 문제다.

파이선에서는 클래스 순서를 정할 때 협동 다중 상속(cooperative multiple inheritance)이라는 규칙을 따른다.

항상 자식을 부모보다 먼저 확인한다.
부모가 둘 이상인 경우 항상 리스트된 순서대로 확인한다.
계층의 모든 클래스를 이 규칙에 따라 정렬함으로써 MRO를 계산한다.

```
>>> E.__mro__
(
  <class 'E'>,
```

```
  <class 'C'>,
  <class 'A'>,
  <class 'D'>,
  <class 'B'>,
  <class 'object'>)
>>>
```

이때 사용하는 알고리즘은 "C3 선형화(Linearization) 알고리즘"이라는 것으로, 세부적인 내용은 중요하지 않다. 집에 불이 났을 때 자식을 먼저 대피시킨 다음 부모가 대피하는 것처럼 클래스 계층이 같은 순서 규칙을 따른다는 점만 기억하면 된다.

이상한 코드 재사용(다중 상속이 관련됨)

다음 두 예제를 보자.

```
class Dog:
    def noise(self):
        return 'Bark'

    def chase(self):
        return 'Chasing!'

class LoudDog(Dog):
    def noise(self):
        # LoudBike와의 코드 공통성(아래)
        return super().noise().upper()
```

위 예제의 객체들은 아래 예제의 객체들과 아무 관련이 없다.

```
class Bike:
    def noise(self):
        return 'On Your Left'

    def pedal(self):
        return 'Pedaling!'

class LoudBike(Bike):
    def noise(self):
        # LoudDog과의 코드 공통성(위)
        return super().noise().upper()
```

그런데 LoudDog.noise()와 LoudBike.noise() 구현에는 공통적인 코드가 있다. 사실 코드가 똑같다. 소프트웨어 기술자라면 이런 코드에 끌리기 마련이다.

믹스인 패턴

믹스인(Mixin) 패턴은 코드 일부를 갖는 클래스다.

```
class Loud:
    def noise(self):
        return super().noise().upper()
```

이 클래스는 동떨어져 사용할 수 없다. 이것은 상속을 통해 다른 클래스들을 뒤섞는다.

```
class LoudDog(Loud, Dog):
    pass

class LoudBike(Loud, Bike):
    pass
```

noise 메서드를 한 번만 구현해, 서로 관계 없는 두 클래스에 재사용했다. 파이썬에서 다중 상속을 사용하는 주 용도가 이런 것이다.

super()를 사용하는 이유

메서드를 오버라이딩할 때는 항상 super()를 사용하라.

```
class Loud:
    def noise(self):
        return super().noise().upper()
```

super()는 MRO의 다음 클래스에 위임한다.

까다로운 부분은 당신이 그것을 알 수 없다는 점이다. 특히 다중 상속이 일어날 때는 그것이 무엇인지 알 수가 없다.

주의 사항

다중 상속은 강력한 도구다. 힘에는 책임이 따른다는 점을 명심하라. 프레임워크/라이브러리에서 컴포넌트의 구성과 관련된 고급 기능을 위해 그것을 종종 사용한다. 이제, 방금 본 것을 잊어라.

연습 문제

섹션 4에서, 보유 주식을 나타내는 Stock 클래스를 정의했다. 이 연습 문제에서는 그 클래스를 재사용한다. 인터프리터를 다시 시작해 인스턴스를 몇 개 만들자.

```
>>>                    ================================                    RESTART
================================
>>> from stock import Stock
>>> goog = Stock('GOOG',100,490.10)
>>> ibm  = Stock('IBM',50, 91.23)
>>>
```

연습 문제 5.1: 인스턴스 표현
상호작용적인 셸에서, 당신이 생성성 두 인스턴스의 내부 딕셔너리를 조사해 보라.

```
>>> goog.__dict__
... 출력을 보라 ...
>>> ibm.__dict__
... 출력을 보라 ...
>>>
```

연습 문제 5.2: 인스턴스 데이터 수정
위 인스턴스들 중 하나에 새 어트리뷰트를 설정해 보라.

```
>>> goog.date = '6/11/2007'
>>> goog.__dict__
... 출력을 보라 ...
>>> ibm.__dict__
... 출력을 보라 ...
>>>
```

위의 출력에서 goog 인스턴스는 date 속성을 갖는 반면 ibm 인스턴스는 그렇지 않은 것을 볼 수 있다. 파이썬은 속성에 아무 제한을 두지 않는다는 것에 유의하자. 예를 들어, 인스턴스의 어트리뷰트는 __init__() 메서드에서 셋업한 것에 국한되지 않는다.

어트리뷰트를 설정하는 대신, __dict__ 객체에 새 값을 직접 넣어라.

```
>>> goog.__dict__['time'] = '9:45am'
>>> goog.time
'9:45am'
>>>
```

여기서 인스턴스는 딕셔너리 위의 계층에 불과하다는 것을 알 수 있다. 참고: 보통은 이런 식으로 딕셔너리를 직접 조작하지 않는다. 항상 (.) 구문을 사용해 코드를 작성하자.

연습 문제 5.3: 클래스의 역할
클래스 정의를 만드는 정의는 해당 클래스의 모든 인스턴스에 의해 공유된다. 모든 인스턴스가 관련 클래스에 대한 링크를 갖는다.

```
>>> goog.__class__
... 출력을 살펴보라 ...
>>> ibm.__class__
... 출력을 살펴보라 ...
>>>
```

인스턴스에 대해 메서드를 호출해 보자.

```
>>> goog.cost()
49010.0
>>> ibm.cost()
4561.5
>>>
```
'cost'라는 이름은 goog.__dict__나 ibm.__dict__에 정의돼 있지 않다. 클래스 딕셔너리에서 제공한 것이다. 다음과 같이 해 보라.

```
>>> Stock.__dict__['cost']
... 출력을 살펴보라 ...
>>>
```
딕셔너리를 통해 cost() 메서드를 직접 호출해보자.

```
>>> Stock.__dict__['cost'](goog)
49010.0
>>> Stock.__dict__['cost'](ibm)
4561.5
>>>
```
클래스 정의에 정의된 함수를 호출한 것이다. self 인자가 인스턴스를 어떻게 얻는지도 유의하자.

Stock 클래스에 새로운 어트리뷰트를 추가해보자.

```
>>> Stock.foo = 42
>>>
```
새로 추가한 어트리뷰트가 이제 모든 인스턴스에 나타난다.

```
>>> goog.foo
42
>>> ibm.foo
42
>>>
```
그렇지만, 이는 인스턴스 딕셔너리에 속한 것은 아니다.

```
>>> goog.__dict__
... 출력을 살펴보면 'foo' 어트리뷰트가 없다 ...
>>>
```
인스턴스에서 foo 어트리뷰트에 액세스할 수 있는 이유는, 파이썬에서는 인스턴스 자체에서 뭔가를 찾을 수 없으면 항상 클래스 딕셔너리를 확인하기 때문이다.

참고: 연습 문제의 이 부분은 클래스 변수라고 알려진 것을 묘사한다. 예를 들어, 다음과 같은

클래스가 있다고 하자.

```
class Foo(object):
    a = 13                  # 클래스 변수
    def __init__(self,b):
        self.b = b          # 인스턴스 변수
```

이 클래스에서 변수 a는 클래스 자체의 몸체에 할당된 '클래스 변수'다. 생성되는 모든 인스턴스가 이것을 공유한다. 예:

```
>>> f = Foo(10)
>>> g = Foo(20)
>>> f.a          # 클래스 변수를 조사(두 인스턴스에 대해 같음)
13
>>> g.a
13
>>> f.b          # 인스턴스 변수를 조사(다름)
10
>>> g.b
20
>>> Foo.a = 42   # 클래스 변수의 값을 변경
>>> f.a
42
>>> g.a
42
>>>
```

연습 문제 5.4: 바운드 메서드

파이썬에서 메서드를 호출하는 것은 실제로는 두 단계로 이뤄지며 바운드 메서드라는 것이 관여한다. 예:

```
>>> s = goog.sell
>>> s
<bound method Stock.sell of Stock('GOOG', 100, 490.1)>
>>> s(25)
>>> goog.shares
75
>>>
```

바운드 메서드는 메서드를 호출하는 데 필요한 것을 모두 포함한다. 예를 들어, 메서드를 구현하는 함수의 레코드를 유지한다.

```
>>> s.__func__
<function sell at 0x10049af50>
```

```
>>>
```
Stock 딕셔너리에서 이것과 같은 값을 찾을 수 있다.

```
>>> Stock.__dict__['sell']
<function sell at 0x10049af50>
>>>
```
바운드 메서드는 인스턴스, 즉 self 인자도 기록한다.

```
>>> s.__self__
Stock('GOOG',75,490.1)
>>>
```
()를 사용해 함수를 호출할 때 모든 것이 합쳐진다. 예를 들어, s(25)를 호출하는 것은 실제로는 다음과 같은 일을 한다.

```
>>> s.__func__(s.__self__, 25)     # s(25)와 같음
>>> goog.shares
50
>>>
```
연습 문제 5.5: 상속

Stock으로부터 상속받는 새 클래스를 생성한다.

```
>>> class NewStock(Stock):
        def yow(self):
            print('Yow!')

>>> n = NewStock('ACME', 50, 123.45)
>>> n.cost()
6172.50
>>> n.yow()
Yow!
>>>
```
상속은 어트리뷰트를 위한 검색 프로세스를 확장함으로써 구현된다. __bases__ 어트리뷰트는 직계 부모의 튜플을 갖는다.

```
>>> NewStock.__bases__
(<class 'stock.Stock'>,)
>>>
```
__mro__ 어트리뷰트는 전체 부모의 튜플을 가지며, 그 순서에 따라 어트리뷰트를 검색한다.

```
>>> NewStock.__mro__
(<class '__main__.NewStock'>, <class 'stock.Stock'>, <class 'object'>)
```

```
>>>
```
위의 n 인스턴스의 cost() 메서드를 찾는 방법은 다음과 같다.

```
>>> for cls in n.__class__.__mro__:
        if 'cost' in cls.__dict__:
            break

>>> cls
<class '__main__.Stock'>
>>> cls.__dict__['cost']
<function cost at 0x101aed598>
>>>
```
마지막 편집일시 : 2020년 10월 3일 7:04

5.2 클래스와 캡슐화

클래스를 작성할 때는 내부적인 세부사항을 캡슐화하는 것이 일반적이다. 이 섹션에서는 프라이빗 변수와 프로퍼티(property)를 포함해, 파이썬 프로그래밍의 몇 가지 관례를 소개한다.

퍼블릭 vs 프라이빗

클래스의 주역할은 객체의 데이터와 내부 구현 세부사항을 캡슐화하는 것이다. 그러나, 외부 세계가 객체를 조작하는 데 사용할 퍼블릭(public) 인터페이스도 클래스에 정의해야 한다. 구현 세부사항과 퍼블릭 인터페이스를 구분하는 것이 중요하다.

문제점

파이썬에서는 클래스와 객체의 거의 모든 것이 오픈되어 있다.

객체의 내부를 쉽게 조사할 수 있고,
원하는 대로 바꿀 수 있다.
액세스 제어에 대한 강력한 개념은 없다(예: 프라이빗 클래스 멤버).
이는 내부 구현을 격리하고자 할 때 이슈가 된다.

파이썬의 캡슐화(Encapsulation)

파이썬은 의도된 사용법을 지시하는 프로그래밍 관례를 따른다. 이러한 관례는 명명(naming)에 기반을 둔다. 언어에서 규칙을 강요하기보다는 프로그래머가 자발적으로 준수하는 것이 일반적이다.

프라이빗 어트리뷰트

이름이 _로 시작하는 어트리뷰트는 프라이빗(private)으로 간주한다.

```
class Person(object):
    def __init__(self, name):
```

```
        self._name = 0
```
앞에서 언급한 것과 같이, 이것은 프로그래밍 스타일일 뿐이다. 여전히 그것에 액세스하고 변경할 수 있다.

```
>>> p = Person('Guido')
>>> p._name
'Guido'
>>> p._name = 'Dave'
>>>
```
_로 시작하는 이름은, 그것이 변수든 함수든 모듈명이든, 모두 내부 구현으로 간주하는 것이 일반적인 규칙이다. 그런 이름을 직접 사용하고 있다면 뭔가 잘못 하고 있는 것이다. 고수준 기능을 찾아라.

단순 어트리뷰트
다음 클래스를 생각해보자.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price
```
놀랍게도, 어떤 값에든지 어트리뷰트를 설정할 수 있다.

```
>>> s = Stock('IBM', 50, 91.1)
>>> s.shares = 100
>>> s.shares = "hundred"
>>> s.shares = [1, 0, 0]
>>>
```
이것을 보고 뭔가 추가적인 검사가 필요하다는 생각이 들었을지 모른다.

```
s.shares = '50'      # 문자열인 경우 TypeError를 일으키고 싶다
```
이것을 어떻게 할 수 있을까?

관리(Managed) 어트리뷰트
접근자(accessor) 메서드를 도입하는 방법이 있긴 하다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
    self.set_shares(shares)
    self.price = price
```

```
    # "get" 연산을 계층화하는 기능
    def get_shares(self):
        return self._shares

    # "set" 연산을 계층화하는 기능
    def set_shares(self, value):
        if not isinstance(value, int):
            raise TypeError('Expected an int')
        self._shares = value
```

이것 때문에 기존 코드가 깨지는 것이 너무 나쁘다. s.shares = 50이 s.set_shares(50)이 된다.

프로퍼티

더 나은 방법이 있다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price

    @property
    def shares(self):
        return self._shares

    @shares.setter
    def shares(self, value):
        if not isinstance(value, int):
            raise TypeError('Expected int')
        self._shares = value
```

이제 평범한 프로퍼티 액세스는 @property와 @shares.setter 하에서 getter와 setter 메서드를 트리거한다.

```
>>> s = Stock('IBM', 50, 91.1)
>>> s.shares          # @property를 트리거
50
>>> s.shares = 75     # @shares.setter를 트리거
>>>
```

이 패턴을 적용하면 소스 코드를 바꿀 필요가 없다. 새로운 setter도 클래스 내에 할당되면 내부 __init__() 메서드를 포함해 호출된다.

```
class Stock:
    def __init__(self, name, shares, price):
        ...
        # 이 할당은 아래 setter를 호출한다
        self.shares = shares
        ...

    ...
    @shares.setter
    def shares(self, value):
        if not isinstance(value, int):
            raise TypeError('Expected int')
        self._shares = value
```

프로퍼티와 프라이빗 이름이 혼동되곤 한다. 프로퍼티는 프라이빗 이름을 _shares 같이 내부적으로 사용하지만, 클래스의 나머지(프로퍼티가 아닌 것)는 shares와 같은 이름을 계속 사용할 수 있다.

프로퍼티는 또한 계산된 데이터 어트리뷰트에 유용하다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price

    @property
    def cost(self):
        return self.shares * self.price
    ...
```

이것은 추가적인 괄호를 버릴 수 있게 해주어, 그것이 실제로 메서드임을 숨긴다.

```
>>> s = Stock('GOOG', 100, 490.1)
>>> s.shares # 인스턴스 변수
100
>>> s.cost    # 계산된 변수
49010.0
>>>
```

통일된 액세스

마지막 예제는 객체에 좀 더 통일된 인터페이스를 넣는 방법을 보여준다. 이것을 하지 않는다면 객체는 사용하기 혼동될 수 있다.

```
>>> s = Stock('GOOG', 100, 490.1)
>>> a = s.cost() # 메서드
49010.0
>>> b = s.shares # 데이터 어트리뷰트
100
>>>
```
왜 cost에는 ()가 붙어야 하고 share는 그렇지 않은가? 프로퍼티는 이것을 고칠 수 있다.

데코레이터(Decorator) 구문
@ 구문을 "데코레이션(decoration)"이라고 부른다. 데코레이션은 바로 뒤따라오는 함수 정의에 적용되는 수정자를 지정한다.

```
...
@property
def cost(self):
    return self.shares * self.price
```
자세한 사항은 7장에서 다룬다.

__slots__ 어트리뷰트
어트리뷰트 이름의 집합을 제한할 수 있다.

```
class Stock:
    __slots__ = ('name','_shares','price')
    def __init__(self, name, shares, price):
        self.name = name
        ...
```
그외의 어트리뷰트에 대해서는 오류가 일어난다.

```
>>> s.price = 385.15
>>> s.prices = 410.2
Traceback (most recent call last):
File "<stdin>", line 1, in ?
AttributeError: 'Stock' object has no attribute 'prices'
```
이는 오류를 방지하고 객체 사용을 제한하지만, 실제로는 성능을 위해 사용되며 파이썬이 메모리를 더 효율적으로 사용하게 해준다.

캡슐화에 대한 최종 의견
프라이빗 어트리뷰트, 프로퍼티, 슬롯 등을 사용하지 마라. 이런 것들은 특수한 목적이 있으며, 다른 파이썬 코드를 읽다보면 눈에 띌 수 있다. 하지만 일상적인 코딩에는 거의 필요 없다.

연습 문제

연습 문제 5.6: 단순 프로퍼티

프로퍼티는 '계산된 어트리뷰트'를 객체에 추가하는 유용한 방법이다. stock.py에서 Stock 객체를 생성했다. 그 객체는 데이터의 종류에 따라 추출 방법이 일관적이지 않다.

```
>>> from stock import Stock
>>> s = Stock('GOOG', 100, 490.1)
>>> s.shares
100
>>> s.price
490.1
>>> s.cost()
49010.0
>>>
```

특히, cost는 메서드이므로 ()를 붙여야 한다.

만약 cost()를 프로퍼티로 바꿀 수 있다면 괄호를 제거할 수 있다. 다음과 같은 방법으로 cost 계산이 이뤄지게 Stock 클래스를 수정해보라.

```
>>> ================================ RESTART ================================
>>> from stock import Stock
>>> s = Stock('GOOG', 100, 490.1)
>>> s.cost
49010.0
>>>
```

cost가 프로퍼티로 정의되었으므로, s.cost()를 호출하면 작동하지 않을 것이다.

```
>>> s.cost()
... 실패 ...
>>>
```

이렇게 수정하면 앞의 pcost.py 프로그램이 깨진다. 문제가 생기지 않게 cost() 메서드의 ()를 제거하라.

연습 문제 5.7: 프로퍼티와 Setter

값이 프라이빗 어트리뷰트에 저장되고, 프로퍼티 함수의 쌍을 사용해 항상 정숫값이 설정됨을 보증하게 shares 어트리뷰트를 수정한다. 예상되는 행위의 예는 다음과 같다.

```
>>> ================================ RESTART ================================
```

```
>>> from stock import Stock
>>> s = Stock('GOOG',100,490.10)
>>> s.shares = 50
>>> s.shares = 'a lot'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: expected an integer
>>>
```

연습 문제 5.8: 슬롯 추가하기

Stock 클래스를 수정해 __slots__ 어트리뷰트를 추가한다. 그런 다음, 새로운 어트리뷰트를 추가할 수 없음을 확인한다.

```
>>> ================================ RESTART ================================
>>> from stock import Stock
>>> s = Stock('GOOG', 100, 490.10)
>>> s.name
'GOOG'
>>> s.blah = 42
... 어떤 일이 일어나는지 보라 ...
>>>
```

__slots__를 사용하면 파이썬은 객체의 내부 표현을 더 효율적으로 사용한다. s의 딕셔너리를 조사하려고 하면 무슨 일이 일어날까?

```
>>> s.__dict__
... 어떤 일이 일어나는지 보라 ...
>>>
```

데이터 구조로 사용되는 클래스의 최적화로 __slots__를 일반적으로 사용한다. 슬롯을 사용하면 프로그램이 메모리를 훨씬 적게 사용하고 더 빨리 실행된다. 하지만 그외의 클래스에서는 __slots__를 사용하지 않는 것이 좋다.

마지막 편집일시 : 2020년 10월 4일 12:17

---

6. 제너레이터

이터레이션(for 루프)은 파이썬의 가장 일반적인 프로그래밍 패턴이다. 프로그램에서 리스트를 다루고, 파일을 읽고, 데이터베이스에 질의하는 등의 여러 가지 일에 이터레이션이 자주 사용된다. 파이썬의 가장 큰 장점 중 하나가 바로 "제너레이터 함수"라는 형태로 이터레이션을 커스터마이즈하고 새롭게 정의할 수 있는 능력이다. 이 섹션은 이 주제를 다룬다. 끝으로, 실시간 스트리밍 데이터를 흥미로운 방식으로 처리하는 프로그램을 작성한다.

- 6.1 이터레이션 프로토콜

- 6.2 제너레이터를 사용해 이터레이션을 커스터마이즈하기
- 6.3 생산자/소비자 문제와 흐름
- 6.4 제너레이터 표현식

마지막 편집일시 : 2020년 7월 26일 12:26 오전

6.1 이터레이션 프로토콜
이 섹션은 이터레이션이 어떻게 작동하는지 살펴본다.

너도나도 이터레이션
온갖 객체들이 이터레이션을 지원한다.

```
a = 'hello'
for c in a: # a의 문자를 루핑
    ...

b = { 'name': 'Dave', 'password':'foo'}
for k in b: # 딕셔너리 키를 루핑
    ...

c = [1,2,3,4]
for i in c: # 리스트/튜플의 항목을 루핑
    ...

f = open('foo.txt')
for x in f: # 파일의 행을 루핑
    ...
```

이터레이션: 프로토콜
for 문을 생각해보자.

```
for x in obj:
    # 문장
```

내부적으로 어떻게 작동하는가?

```
_iter = obj.__iter__()          # 이터레이터 객체를 얻음
while True:
    try:
        x = _iter.__next__()  # 다음 항목을 얻음
    except StopIteration:      # 남은 항목이 없음
        break
    # 문장 ...
```

for 문에서 작동하는 모든 객체는 이러한 저수준 이터레이션 프로토콜을 구현한 것이다.

예제: 리스트에 대해 수작업으로 이터레이션.

```
>>> x = [1,2,3]
>>> it = x.__iter__()
>>> it
<listiterator object at 0x590b0>
>>> it.__next__()
1
>>> it.__next__()
2
>>> it.__next__()
```

```
3
>>> it.__next__()
Traceback (most recent call last):
File "<stdin>", line 1, in ? StopIteration
>>>
```

이터레이션 지원하기

이터레이션의 유용성을 이해한다면 스스로 작성한 객체도 이터레이션을 지원하게 하고 싶을 것이다. 예를 들어, 다음과 같이 커스텀 컨테이너를 만들 수 있다.

```
class Portfolio:
    def __init__(self):
        self.holdings = []

    def __iter__(self):
        return self.holdings.__iter__()
    ...

port = Portfolio()
for s in port:
    ...
```

연습 문제

연습 문제 6.1: 이터레이션 뜯어보기

다음 리스트를 생성한다.

```
a = [1,9,4,25,16]
```

이 리스트에 대해 수작업으로 이터레이션을 수행하라. __iter__()를 호출해 이터레이터를 얻은 다음, __next__() 메서드를 호출해 다음 번 원소를 얻는다.

```
>>> i = a.__iter__()
>>> i
<listiterator object at 0x64c10>
>>> i.__next__()
1
>>> i.__next__()
9
>>> i.__next__()
4
>>> i.__next__()
25
>>> i.__next__()
16
>>> i.__next__()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```

빌트인 함수 next()를 사용하면 이터레이터의 __next__() 메서드를 간편하게 호출할 수 있다. 파일을 가지고도 같은 일을 해 보자.

```
>>> f = open('Data/portfolio.csv')
>>> f.__iter__()     # 참고: 이것은 파일 자체를 반환함
<_io.TextIOWrapper name='Data/portfolio.csv' mode='r' encoding='UTF-8'>
>>> next(f)
'name,shares,price\n'
```

```
>>> next(f)
'"AA",100,32.20\n'
>>> next(f)
'"IBM",50,91.10\n'
>>>
```
파일의 끝에 도달할 때까지 next(f)를 계속 호출하라. 무슨 일이 일어나는지 관찰하라.

연습 문제 6.2: 이터레이션을 지원하기
당신이 직접 만든 객체가 이터레이션을 지원하게 하고 싶을 때가 있다. 기존 리스트나 다른 이터러블을 감싸는 객체를 만들었을 때가 특히 그렇다. 새 파일 portfolio.py에 다음 클래스를 정의하자.

```
# portfolio.py

class Portfolio:

    def __init__(self, holdings):
        self._holdings = holdings

    @property
    def total_cost(self):
        return sum([s.cost for s in self._holdings])

    def tabulate_shares(self):
        from collections import Counter
        total_shares = Counter()
        for s in self._holdings:
            total_shares[s.name] += s.shares
        return total_shares
```
이 클래스는 리스트 주위에 계층을 덧씌우되, total_cost 프로퍼티와 같은 추가 메서드를 갖는다. report.py의 read_portfolio() 함수를 수정해 다음과 같은 Portfolio 인스턴스를 생성하게 하자.

```
# report.py
...

import fileparse
from stock import Stock
from portfolio import Portfolio

def read_portfolio(filename):
    '''
    주식 포트폴리오 파일을 읽어 딕셔너리의 리스트를 생성.
    name, shares, price를 키로 사용.
    '''
    with open(filename) as file:
        portdicts = fileparse.parse_csv(file,
                                        select=['name','shares','price'],
                                        types=[str,int,float])

    portfolio = [ Stock(d['name'], d['shares'], d['price']) for d in portdicts ]
    return Portfolio(portfolio)
...
```
report.py 프로그램을 실행해 보라. Portfolio 인스턴스가 이터러블하지 않다는 사실로 인해

요란하게 실패하는 모습을 보게 된다.

```
>>> import report
>>> report.portfolio_report('Data/portfolio.csv', 'Data/prices.csv')
... 충돌한다 ...
```

Portfolio 클래스가 이터레이션을 지원하게 수정해서 문제를 해결해 보자.

```
class Portfolio:

    def __init__(self, holdings):
        self._holdings = holdings

    def __iter__(self):
        return self._holdings.__iter__()

    @property
    def total_cost(self):
        return sum([s.shares*s.price for s in self._holdings])

    def tabulate_shares(self):
        from collections import Counter
        total_shares = Counter()
        for s in self._holdings:
            total_shares[s.name] += s.shares
        return total_shares
```

이렇게 수정하고 나면 report.py 프로그램이 다시 잘 작동할 것이다. pcost.py 프로그램이 새로운 Portfolio 객체를 사용하게 수정하자. 이렇게 말이다.

```
# pcost.py

import report

def portfolio_cost(filename):
    '''
    포트폴리오 파일의 총 비용(주식 수 * 가격)을 계산
    '''
    portfolio = report.read_portfolio(filename)
    return portfolio.total_cost
...
```

잘 작동하는지 시험해 보자.

```
>>> import pcost
>>> pcost.portfolio_cost('Data/portfolio.csv')
44671.15
>>>
```

연습 문제 6.3: 더 적절한 컨테이너 만들기

컨테이너 클래스를 만들다 보면 이터레이션 외에도 넣고 싶은 기능이 더 있을 것이다. Portfolio 클래스를 다음과 같이 수정해 특수 메서드를 추가해 보자.

```
class Portfolio:
    def __init__(self, holdings):
        self._holdings = holdings

    def __iter__(self):
```

```
        return self._holdings.__iter__()

    def __len__(self):
        return len(self._holdings)

    def __getitem__(self, index):
        return self._holdings[index]

    def __contains__(self, name):
        return any([s.name == name for s in self._holdings])

    @property
    def total_cost(self):
        return sum([s.shares*s.price for s in self._holdings])

    def tabulate_shares(self):
        from collections import Counter
        total_shares = Counter()
        for s in self._holdings:
            total_shares[s.name] += s.shares
        return total_shares
```

새 클래스를 가지고 실험을 해 보자.

```
>>> import report
>>> portfolio = report.read_portfolio('Data/portfolio.csv')
>>> len(portfolio)
7
>>> portfolio[0]
Stock('AA', 100, 32.2)
>>> portfolio[1]
Stock('IBM', 50, 91.1)
>>> portfolio[0:3]
[Stock('AA', 100, 32.2), Stock('IBM', 50, 91.1), Stock('CAT', 150, 83.44)]
>>> 'IBM' in portfolio
True
>>> 'AAPL' in portfolio
False
>>>
```

이와 관련해 중요한 점을 관찰할 수 있다. 코드가 파이썬에서 공통적으로 사용되는 어휘를 말한다면 '파이썬다운(Pythonic)' 것으로 간주된다. 컨테이너 객체를 파이썬답게 만들려면 이터레이션, 인덱싱, containment, 그외 연산자를 지원하는 것이 중요하다.

마지막 편집일시 : 2020년 10월 3일 5:45

6.2 이터레이션을 커스터마이징

이 섹션은 제너레이터 함수를 사용해 이터레이션을 커스터마이즈하는 법을 살펴본다.

문제

커스텀 이터레이션 패턴을 만들고 싶다고 하자.

예(카운트다운):

```
>>> for x in countdown(10):
...     print(x, end=' ')
```

```
...
10 9 8 7 6 5 4 3 2 1
>>>
```
이것을 쉽게 할 수 있는 방법이 있다.

제너레이터
제너레이터는 이터레이션을 정의하는 함수다.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```
예:

```
>>> for x in countdown(10):
...    print(x, end=' ')
...
10 9 8 7 6 5 4 3 2 1
>>>
```
yield 문을 사용하는 함수는 제너레이터다.

제너레이터는 일반적인 함수와 다르게 작동한다. 제너레이터 함수를 호출하면 제너레이터 객체가 생성된다. 함수가 즉시 실행되는 것은 아니다.

```
def countdown(n):
    # print 문을 추가했다
    print('Counting down from', n)
    while n > 0:
        yield n
        n -= 1
>>> x = countdown(10)
# print 문이 없다!
>>> x
# x는 generator 객체다
<generator object at 0x58490>
>>>
```
이 함수는 __next__() 호출에 의해서만 실행된다.

```
>>> x = countdown(10)
>>> x
<generator object at 0x58490>
>>> x.__next__()
Counting down from 10
10
>>>
```
yield는 값을 생산하고, 함수 실행을 일시 정지한다. 다음번 __next__() 호출 시 함수가 재개된다.

```
>>> x.__next__()
9
>>> x.__next__()
8
```
제너레이터가 마지막으로 반환할 때, 이터레이션은 오류를 일으킨다.

```
>>> x.__next__()
1
>>> x.__next__()
Traceback (most recent call last):
File "<stdin>", line 1, in ? StopIteration
>>>
```
관찰: 제너레이터 함수는 리스트, 튜플, 딕셔너리, 파일 등에 사용하는 for 문과 똑같은 저수준 프로토콜을 구현한다.

연습 문제
연습 문제 6.4: 단순한 제너레이터
이터레이션을 커스터마이즈하고 싶을 때에는 제너레이터 함수를 떠올려라. 작성하기 쉽다---원하는 이터레이션 로직을 함수로 구현하고, yield 문으로 값을 내보내면 된다.

예를 들어, 다음 제너레이터는 파일에서 특정 부분 문자열이 일치하는 행을 검색한다.

```
>>> def filematch(filename, substr):
        with open(filename, 'r') as f:
            for line in f:
                if substr in line:
                    yield line

>>> for line in open('Data/portfolio.csv'):
        print(line, end='')

name,shares,price
"AA",100,32.20
"IBM",50,91.10
"CAT",150,83.44
"MSFT",200,51.23
"GE",95,40.37
"MSFT",50,65.10
"IBM",100,70.44
>>> for line in filematch('Data/portfolio.csv', 'IBM'):
        print(line, end='')

"IBM",50,91.10
"IBM",100,70.44
>>>
```
함수에서 커스터마이즈한 부분을 숨기고 for 루프에 먹이는 데 사용할 수 있다는 점에서 꽤 흥미롭다. 좀 더 특이한 사례를 살펴보자.

연습 문제 6.5: 스트리밍 데이터 소스를 모니터링
제너레이터는 로그 파일이라든지 주식 시장 피드와 같은 실시간 데이터 소스를 모니터링하는 흥미로운 방법이다. 이 개념을 탐색해보자. 시작하려면 다음 지침을 주의깊게 따르기 바란다.

Data/stocksim.py는 주식 시장 데이터를 시뮬레이션하는 프로그램이다. 이 프로그램은 출력으로서, Data/stocklog.csv 파일에 실시간 데이터를 지속적으로 기록한다. 별도의 명령 창에서 Data/ 디렉터리로 가서 이 프로그램을 실행한다.

```
bash % python3 stocksim.py
```
윈도우에서는 탐색기에서 stocksim.py를 더블클릭하면 실행된다. 이제 이 프로그램은 잊어버리자(실행되게 내버려둔다). 다른 창에서, 시뮬레이터에 의해 Data/stocklog.csv 파일이 기록되는 것을 살펴보자. 몇 초마다 새로운 텍스트 행이 파일에 추가되는 것을 볼 수 있을 것이

다. 다시 말하지만, 이 프로그램은 몇 시간이고 실행되게 내버려 둔다(걱정할 필요는 없다).

프로그램이 실행되면, 파일을 열고 파일의 끝에 새로운 출력이 있는지 감시(watch)하는 작은 프로그램을 작성하자. follow.py 파일에 다음과 같이 코드를 작성한다.

```
# follow.py
import os
import time

f = open('Data/stocklog.csv')
f.seek(0, os.SEEK_END)   # 파일 포인터를 파일의 끝으로부터 0 바이트로 이동

while True:
    line = f.readline()
    if line == '':
        time.sleep(0.1)   # 짧게 쉬었다가 재시도
        continue
    fields = line.split(',')
    name = fields[0].strip('"')
    price = float(fields[1])
    change = float(fields[4])
    if change < 0:
        print(f'{name:>10s} {price:>10.2f} {change:>10.2f}')
```

프로그램을 실행하면 실시간 주식 티커를 볼 수 있다. 이 코드는 유닉스의 tail -f 명령처럼 로그 파일을 감시한다.

참고: 파일에서 행을 읽는 일반적인 방법이 아니라는 점에서, 이 예의 readline() 메서드의 사용은 그리 일반적이지 않다(보통은 for 루프를 사용한다). 그렇지만 이 경우 파일의 끝에 새로운 데이터가 추가되는지 반복적으로 확인한다(readline()은 새 데이터 또는 빈 문자열을 반환한다).

https://wikidocs.net/84423

연습 문제 6.6: 제너레이터를 사용해 데이터를 생산
연습 문제 6.5의 코드를 보면, 코드의 첫 부분은 데이터의 행들을 생산하고, while 루프의 끝 문장은 데이터를 소비한다. 제너레이터 함수의 주요 기능은 모든 데이터 생산 코드를 재사용 가능한 함수로 이전하는 것이다.

제너레이터 함수 follow(filename)이 파일 읽기를 수행하도록 연습 문제 6.5의 코드를 수정하자. 다음 코드가 작동하게 하라.

```
>>> for line in follow('Data/stocklog.csv'):
        print(line, end='')

... 출력 행이 여기 만들어져야 한다 ...
```

주식 티커 코드를 다음과 같이 수정하자.

```
if __name__ == '__main__':
    for line in follow('Data/stocklog.csv'):
        fields = line.split(',')
        name = fields[0].strip('"')
        price = float(fields[1])
        change = float(fields[4])
        if change < 0:
```

```
            print(f'{name:>10s} {price:>10.2f} {change:>10.2f}')
```

연습 문제 6.7: 포트폴리오 감시하기
주식 데이터의 스트림을 감시해, 포트폴리오에 있는 주식의 정보만 티커를 프린트하도록 follow.py 프로그램을 수정하자. 예:

```
if __name__ == '__main__':
    import report

    portfolio = report.read_portfolio('Data/portfolio.csv')

    for line in follow('Data/stocklog.csv'):
        fields = line.split(',')
        name = fields[0].strip('"')
        price = float(fields[1])
        change = float(fields[4])
        if name in portfolio:
            print(f'{name:>10s} {price:>10.2f} {change:>10.2f}')
```

참고: 이 작업을 위해, Portfolio 클래스는 반드시 in 연산자를 지원해야 한다. 연습 문제 6.3을 참고해서 __contains__() 연산자를 구현하자.

논의
매우 강력한 것을 만들었다. 흥미로운 이터레이션 패턴(파일의 마지막 행을 읽기)을 그것의 작은 함수에 옮겼다. 이제 follow() 함수는 완전히 일반적인 유틸리티가 되었으며, 다른 프로그램에서도 사용할 수 있게 됐다. 예를 들어, 서버 로그, 디버깅 로그, 기타 그와 비슷한 데이터 소스를 감시할 수 있다. 훌륭하다.

마지막 편집일시 : 2021년 3월 7일 10:50

6.3 생산자, 소비자, 파이프라인
제너레이터는 생산자/소비자 문제와 데이터 흐름 파이프라인 같은 다양한 문제를 푸는 데 유용한 도구다. 이 섹션은 그에 대해 논의한다.

생산자-소비자 문제
제너레이터는 생산자-소비자(producer-consumer) 문제와 밀접한 연관이 있다.

```
# 생산자
def follow(f):
    ...
    while True:
        ...
        yield line          # 아래 `line`에 값을 생산
        ...

# 소비자
for line in follow(f):     # 위에서 `yield`한 값을 소비
    ...
```

yield가 생산한 값을 for 문에서 소비한다.

제너레이터 파이프라인
제너레이터의 이러한 측면을 사용해 처리 파이프라인(Unix 파이프 같은 것)을 설정할 수 있다.

생산자 → 처리 → 처리 → 소비자

처리 파이프는 초기 데이터 생산자, 중간 처리 단계, 최종 소비자로 이뤄진다.

생산자 → 처리 → 처리 → 소비자

```
def producer():
    ...
    yield item
    ...
```

생산자는 일반적으로 제너레이터다. 다른 시퀀스의 리스트가 될 수도 있다. yield는 파이프라인에 데이터를 공급한다.

생산자 → 처리 → 처리 → 소비자

```
def consumer(s):
    for item in s:
        ...
```

소비자는 for 루프다. 항목을 얻어 무언가를 한다.

생산자 → 처리 → 처리 → 소비자

```
def processing(s):
    for item in s:
        ...
        yield newitem
        ...
```

중간 처리 단계에서는 항목의 소비와 생산이 동시에 이뤄진다. 그것들은 데이터 스트림을 수정할 수도 있다. 그것들은 필터링(항목을 버림)을 할 수도 있다.

생산자 → 처리 → 처리 → 소비자

```
def producer():
    ...
    yield item            # yield한 item을 `processing`이 받음
    ...

def processing(s):
    for item in s:        # `producer`로부터 온 것
        ...
        yield newitem     # newitem을 yield
        ...

def consumer(s):
    for item in s:        # `processing`으로부터 온 것
        ...
```

파이프라인을 설정하는 코드

```
a = producer()
b = processing(a)
c = consumer(b)
```

데이터가 서로 다른 함수들을 통해 점진적으로 흐른다는 것을 알 수 있다.

연습 문제

이 연습 문제를 위해 stocksim.py 프로그램을 백그라운드에서 계속 실행해야 한다. 이전 연습 문제에서 작성한 follow() 함수를 사용한다.

연습 문제 6.8: 단순한 파이프라인 구성
파이프라인 개념의 실제 구현을 살펴보자. 다음 함수를 작성하라.

```
>>> def filematch(lines, substr):
        for line in lines:
            if substr in line:
                yield line

>>>
```

이 함수는 지난 연습 문제의 첫 번째 제너레이터 예제와 거의 같지만, 파일을 열지 않고 인자로 받은 행들의 시퀀스에 대해서만 연산을 한다. 다음을 시도해 보자.

```
>>> lines = follow('Data/stocklog.csv')
>>> ibm = filematch(lines, 'IBM')
>>> for line in ibm:
        print(line)

... 출력을 기다린다 ...
```

출력이 나타나는 데 시간이 걸릴 수도 있지만, 결국 IBM의 데이터를 포함하는 행이 나타난다.

연습 문제 6.9: 좀 더 복잡한 파이프라인 구성
파이프라인의 아이디어를 발전시켜서 더 많은 작업을 수행해 보자.

```
>>> from follow import follow
>>> import csv
>>> lines = follow('Data/stocklog.csv')
>>> rows = csv.reader(lines)
>>> for row in rows:
        print(row)

['BA', '98.35', '6/11/2007', '09:41.07', '0.16', '98.25', '98.35', '98.31', '158148']
['AA', '39.63', '6/11/2007', '09:41.07', '-0.03', '39.67', '39.63', '39.31', '270224']
['XOM', '82.45', '6/11/2007', '09:41.07', '-0.23', '82.68', '82.64', '82.41', '748062']
['PG', '62.95', '6/11/2007', '09:41.08', '-0.12', '62.80', '62.97', '62.61', '454327']
...
```

흥미롭다. follow() 함수의 출력이 csv.reader() 함수로 전달되어 행이 분할된 것을 볼 수 있다.

연습 문제 6.10: 더 큰 파이프라인 컴포넌트
파이프라인을 더 키워보자. 별도의 파일 ticker.py에 CSV 파일을 읽는 함수를 생성한다.

```
# ticker.py

from follow import follow
import csv

def parse_stock_data(lines):
    rows = csv.reader(lines)
    return rows

if __name__ == '__main__':
    lines = follow('Data/stocklog.csv')
    rows = parse_stock_data(lines)
```

```
    for row in rows:
        print(row)
```
특정 컬럼을 선택하는 함수를 새로 작성한다.

```
# ticker.py
...
def select_columns(rows, indices):
    for row in rows:
        yield [row[index] for index in indices]
...
def parse_stock_data(lines):
    rows = csv.reader(lines)
    rows = select_columns(rows, [0, 1, 4])
    return rows
```
프로그램을 다시 실행하자. 선택한 컬럼만 출력된다.

```
['BA', '98.35', '0.16']
['AA', '39.63', '-0.03']
['XOM', '82.45','-0.23']
['PG', '62.95', '-0.12']
...
```
자료형을 변환해 딕셔너리를 만드는 제너레이터 함수를 작성하자. 예:

```
# ticker.py
...

def convert_types(rows, types):
    for row in rows:
        yield [func(val) for func, val in zip(types, row)]

def make_dicts(rows, headers):
    for row in rows:
        yield dict(zip(headers, row))
...
def parse_stock_data(lines):
    rows = csv.reader(lines)
    rows = select_columns(rows, [0, 1, 4])
    rows = convert_types(rows, [str, float, float])
    rows = make_dicts(rows, ['name', 'price', 'change'])
    return rows
...
```
프로그램을 다시 실행하자. 다음과 같이 딕셔너리를 스트리밍하게 됐다.

```
{ 'name':'BA', 'price':98.35, 'change':0.16 }
{ 'name':'AA', 'price':39.63, 'change':-0.03 }
{ 'name':'XOM', 'price':82.45, 'change': -0.23 }
{ 'name':'PG', 'price':62.95, 'change':-0.12 }
...
```
연습 문제 6.11: 데이터를 필터링하기
데이터를 필터링하는 함수를 작성하자. 예:

```
# ticker.py
...
```

```
def filter_symbols(rows, names):
    for row in rows:
        if row['name'] in names:
            yield row
```
이것을 사용해 포트폴리오에 있는 주식만 나오게 해보자.

```
import report
portfolio = report.read_portfolio('Data/portfolio.csv')
rows = parse_stock_data(follow('Data/stocklog.csv'))
rows = filter_symbols(rows, portfolio)
for row in rows:
    print(row)
```
연습 문제 6.12: 모두 합치기
주어진 포트폴리오, 로그 파일, 표 형식대로 실시간 주식 티커를 생성하는 ticker(portfile, logfile, fmt) 함수를 ticker.py 프로그램에 작성한다. 예:

```
>>> from ticker import ticker
>>> ticker('Data/portfolio.csv', 'Data/stocklog.csv', 'txt')
      Name       Price      Change
---------- ---------- ----------
        GE       37.14       -0.18
      MSFT       29.96       -0.09
       CAT       78.03       -0.49
        AA       39.34       -0.32
...

>>> ticker('Data/portfolio.csv', 'Data/stocklog.csv', 'csv')
Name,Price,Change
IBM,102.79,-0.28
CAT,78.04,-0.48
AA,39.35,-0.31
CAT,78.05,-0.47
...
```
논의
배운 것: 다양한 제너레이터 함수를 생성할 수 있으며 그것들을 함께 엮어서 데이터 흐름 파이프라인 처리를 수행할 수 있다. 또한, 파이프라인의 단계들을 패키징하는 함수를 작성해 단일 함수로 호출할 수 있다(예: parse_stock_data() 함수).

마지막 편집일시 : 2020년 6월 29일 4:15

6.4 제너레이터 심화
이 섹션은 제너레이터 표현식과 itertools 모듈 같은 몇 가지 추가적인 제너레이터 관련 주제를 소개한다.

제너레이터 표현식
리스트 컴프리헨션의 제너레이터 버전.

```
>>> a = [1,2,3,4]
>>> b = (2*x for x in a)
>>> b
<generator object at 0x58760>
>>> for i in b:
...     print(i, end=' ')
...
```

```
2 4 6 8
>>>
```
리스트 컴프리헨션과의 차이점.

리스트를 생성하지 않는다.
유일한 용도는 이터레이션이다.
소비되고 나면 재사용할 수 없다.
일반적인 구문.

(<표현식> for i in s if <조건>)
함수 인자로도 사용할 수 있다.

sum(x*x for x in a)
모든 이터러블에 적용할 수 있다.

```
>>> a = [1,2,3,4]
>>> b = (x*x for x in a)
>>> c = (-x for x in b)
>>> for i in c:
...     print(i, end=' ')
...
-1 -4 -9 -16
>>>
```
제너레이터 표현식의 주용도는 시퀀스에서 계산을 수행하되 결과를 단 한 번만 사용하는 코드를 위한 것이다. 예를 들어, 파일에서 모든 주석을 제거한다.

```
f = open('somefile.txt')
lines = (line for line in f if not line.startswith('#'))
for line in lines:
    ...
f.close()
```
제너레이터를 사용하면 코드가 더 빨리 작동하며 메모리를 적게 사용한다. 스트림에 필터를 적용한 것과 같다.

제너레이터를 사용하는 이유
많은 프로그램은 이터레이션으로 깔끔하게 표현된다.
항목들의 컬렉션을 루핑하며 몇 가지 연산(검색, 교체, 수정 등)을 수행한다.
파이프라인을 처리하는 것은 더 넓은 범위의 데이터 처리 문제가 적용된다.
메모리를 더 효율적으로 사용한다.
필요할 때만 값을 생산한다.
거대한 리스트를 생성하는 것과 대비된다.
스트리밍 데이터를 다룰 수 있다
제너레이터는 코드 재사용을 촉진한다
코드에서 이터레이션과 이터레이션 사용을 분리한다.
흥미로운 이터레이션 함수의 도구모음을 만들고 믹스 앤 매치할 수 있다.
itertools 모듈
itertools는 이터레이터/제너레이터를 돕도록 설계된 다양한 함수가 있는 라이브러리 모듈이다.

```
itertools.chain(s1,s2)
itertools.count(n)
itertools.cycle(s)
itertools.dropwhile(predicate, s)
itertools.groupby(s)
```

itertools.ifilter(predicate, s)
itertools.imap(function, s1, ... sN)
itertools.repeat(s, n)
itertools.tee(s, ncopies)
itertools.izip(s1, ... , sN)
모든 함수는 데이터를 반복적으로 처리한다. 다양한 이터레이션 패턴을 구현한다.

자세한 정보는 2008년 파이콘의 시스템 프로그래머를 위한 제너레이터 트릭 튜토리얼을 참조하라.

연습 문제
지난 연습 문제에서, 로그 파일에 행을 기록하고 행들의 시퀀스로 파싱하는 코드를 작성했다. 이번 문제는 지난 문제와 이어진다. Data/stocksim.py가 계속 실행되는지 확인하자.

연습 문제 6.13: 제너레이터 표현식
제너레이터 표현식은 리스트 컴프리헨션의 제너레이터 버전이다. 예:

```
>>> nums = [1, 2, 3, 4, 5]
>>> squares = (x*x for x in nums)
>>> squares
<generator object <genexpr> at 0x109207e60>
>>> for n in squares:
...     print(n)
...
1
4
9
16
25
```
제너레이터 표현식은 리스트 컴프리헨션과 달리 단 한 번만 사용 가능하다. 따라서, 다른 for 루프를 시도하면 아무 것도 얻지 못할 것이다.

```
>>> for n in squares:
...     print(n)
...
>>>
```
연습 문제 6.14: 제너레이터 표현식을 함수 인자로 사용
제너레이터 표현식을 함수 인자로 사용하기도 한다. 처음에는 좀 이상해 보이겠지만, 다음과 같이 해 보라.

```
>>> nums = [1,2,3,4,5]
>>> sum([x*x for x in nums])     # 리스트 컴프리헨션
55
>>> sum(x*x for x in nums)       # 제너레이터 표현식
55
>>>
```
큰 리스트를 조작하는 경우, 제너레이터를 사용하는 버전이 메모리를 훨씬 적게 사용한다.

portfolio.py 파일에서 리스트 컴프리헨션을 사용하는 계산을 수행했다. 그것들을 제너레이터 표현식으로 바꿔보라.

연습 문제 6.15: 코드 단순화하기
제너레이터 표현식은 작은 제너레이터 함수를 대체할 수 있어 유용하다. 예를 들어, 다음과 같은 함수가 있다.

```
def filter_symbols(rows, names):
    for row in rows:
        if row['name'] in names:
            yield row
```
다음과 같이 작성할 수 있다.

```
rows = (row for row in rows if row['name'] in names)
```
제너레이터 표현식을 사용하도록 ticker.py 프로그램을 수정하자.

마지막 편집일시 : 2020년 10월 4일 12:41

---

7. 고급 주제

이 섹션에서는 고급 주제 가운데 일상적인 코딩에서 접할 수 있는 것을 살펴본다. 이러한 주제는 대부분 이전의 섹션에서 다룰 수도 있었지만, 설명이 너무 복잡해지는 것을 피하기 위해 미뤄둔 것이다.

이 섹션은 이러한 개념의 극히 기본적인 소개로서 다룬다는 점을 강조해야겠다. 자세한 내용은 자료를 찾아보라.

- 7.1 가변 인자 함수
- 7.2 익명 함수와 람다
- 7.3 함수를 반환하기(클로저)
- 7.4 함수 데코레이터
- 7.5 정적 및 클래스 메서드

마지막 편집일시 : 2020년 10월 2일 11:39 오후

7.1 가변 인자

이 섹션은 *args와 **kwargs 같은 함수 인자를 다룬다.

위치 가변 인자(*args)

임의의 개수의 인자를 받는 함수를 가리켜 가변 인자를 사용한다고 표현한다. 예:

```
def f(x, *args):
    ...
```
함수 호출.

```
f(1,2,3,4,5)
```
추가적인 인자를 튜플로 전달한다.

```
def f(x, *args):
    # x -> 1
    # args -> (2,3,4,5)
```
키워드 가변 인자(**kwargs)

함수는 임의의 개수의 키워드 인자도 받을 수 있다. 예:

```
def f(x, y, **kwargs):
    ...
```
함수 호출.

```
f(2, 3, flag=True, mode='fast', header='debug')
```
추가적인 키워드를 딕셔너리로 전달한다.

```
def f(x, y, **kwargs):
    # x -> 2
    # y -> 3
    # kwargs -> { 'flag': True, 'mode': 'fast', 'header': 'debug' }
```
두 가지를 혼합
함수는 임의의 개수의 가변 인자와 키워드 없는(non-keyword) 인자들을 받을 수 있다.

```
def f(*args, **kwargs):
    ...
```
함수 호출.

```
f(2, 3, flag=True, mode='fast', header='debug')
```
인자들은 위치 및 키워드 요소로 분할된다.

```
def f(*args, **kwargs):
    # args = (2, 3)
    # kwargs -> { 'flag': True, 'mode': 'fast', 'header': 'debug' }
    ...
```
이 함수는 위치 또는 키워드 인자들의 어떠한 조합이라도 받을 수 있다. 래퍼(wrapper)를 작성하거나 인자를 다른 함수에 전달하고자 할 때 사용하곤 한다.

튜플과 딕셔너리를 전달하기
튜플을 가변 인자로 확장할 수 있다.

```
numbers = (2,3,4)
f(1, *numbers)        # f(1,2,3,4)와 같음
```
마찬가지로 딕셔너리를 키워드 인자로 확장할 수 있다.

```
options = {
    'color' : 'red',
    'delimiter' : ',',
    'width' : 400
}
f(data, **options)
```

f(data, color='red', delimiter=',', width=400)와 같음
연습 문제
연습 문제 7.1: 가변 인자의 간단한 예
다음 함수를 정의해 보자.

```
>>> def avg(x,*more):
        return float(x+sum(more))/(1+len(more))

>>> avg(10,11)
10.5
>>> avg(3,4,5)
4.0
>>> avg(1,2,3,4,5,6)
3.5
>>>
```

*more 매개변수(parameter)가 추가적인 인자를 어떻게 수집하는지 살펴보라.

연습 문제 7.2: 튜플과 딕셔너리를 인자로 전달
데이터 파일을 읽어 다음과 같은 튜플을 얻었다고 하자.

```
>>> data = ('GOOG', 100, 490.1)
>>>
```

이 데이터로부터 Stock 객체를 생성하고자 한다. 그런데 data를 직접 전달하면 작동하지 않는다.

```
>>> from stock import Stock
>>> s = Stock(data)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: __init__() takes exactly 4 arguments (2 given)
>>>
```

그 대신 *data를 사용하면 간단히 해결할 수 있다. 다음과 같이 해 보라.

```
>>> s = Stock(*data)
>>> s
Stock('GOOG', 100, 490.1)
>>>
```

딕셔너리를 사용할 때는 **을 붙인다. 예:

```
>>> data = { 'name': 'GOOG', 'shares': 100, 'price': 490.1 }
>>> s = Stock(**data)
```

```
Stock('GOOG', 100, 490.1)
>>>
```

연습 문제 7.3: 인스턴스의 리스트를 생성

report.py 프로그램에서 다음과 같은 코드를 사용해 인스턴스의 리스트를 생성했다.

```
def read_portfolio(filename):
    '''
    주식 포트폴리오 파일을 읽어 딕셔너리의 리스트를 생성.
    name, shares, price를 키로 사용.
    '''
    with open(filename) as lines:
        portdicts = fileparse.parse_csv(lines,
                                        select=['name','shares','price'],
                                        types=[str,int,float])

    portfolio = [ Stock(d['name'], d['shares'], d['price'])
                  for d in portdicts ]
    return Portfolio(portfolio)
```

Stock(**d)를 사용해 코드를 단순화할 수 있다. 그렇게 바꿔보라.

연습 문제 7.4: 인자 통과

fileparse.parse_csv() 함수에는 파일 구분자를 바꾸거나 오류를 보고하는 옵션이 있다. 위의 read_portfolio() 함수에 그런 기능이 있으면 좋을 것이다. 다음과 같이 바꿔보자.

```
def read_portfolio(filename, **opts):
    '''
    주식 포트폴리오 파일을 읽어 딕셔너리의 리스트를 생성.
    name, shares, price를 키로 사용.
    '''
    with open(filename) as lines:
        portdicts = fileparse.parse_csv(lines,
                                        select=['name','shares','price'],
                                        types=[str,int,float],
                                        **opts)

    portfolio = [ Stock(**d) for d in portdicts ]
    return Portfolio(portfolio)
```

변경을 했으면, 오류가 있는 파일을 읽어보자.

```
>>> import report
>>> port = report.read_portfolio('Data/missing.csv')
```

```
Row 4: Couldn't convert ['MSFT', '', '51.23']
Row 4: Reason invalid literal for int() with base 10: ''
Row 7: Couldn't convert ['IBM', '', '70.44']
Row 7: Reason invalid literal for int() with base 10: ''
>>>
```
오류 메시지가 나오지 않게 해 보자.

```
>>> import report
>>> port = report.read_portfolio('Data/missing.csv', silence_errors=True)
>>>
```
마지막 편집일시 : 2020년 10월 3일 12:25

7.2 익명 함수와 람다

리스트 정렬 되돌아보기

리스트를 제자리에서(in-place) 정렬할 수 있다. sort 메서드를 사용한다.

```
s = [10,1,7,3]
s.sort() # s = [1,3,7,10]
```
역순으로 정렬할 수 있다.

```
s = [10,1,7,3]
s.sort(reverse=True) # s = [10,7,3,1]
```
간단해 보인다. 그런데, 딕셔너리를 정렬하려면 어떻게 해야 할까?

```
[{'name': 'AA', 'price': 32.2, 'shares': 100},
{'name': 'IBM', 'price': 91.1, 'shares': 50},
{'name': 'CAT', 'price': 83.44, 'shares': 150},
{'name': 'MSFT', 'price': 51.23, 'shares': 200},
{'name': 'GE', 'price': 40.37, 'shares': 95},
{'name': 'MSFT', 'price': 65.1, 'shares': 50},
{'name': 'IBM', 'price': 70.44, 'shares': 100}]
```
어떤 기준으로?

키(key) 함수를 사용해 정렬 기준을 안내할 수 있다. 키 함수는 딕셔너리를 받아서 정렬 기준값을 반환한다.

```
def stock_name(s):
    return s['name']

portfolio.sort(key=stock_name)
```
결과는 다음과 같다.

```
# 딕셔너리가 `name` 키를 기준으로 정렬됨
[
  {'name': 'AA', 'price': 32.2, 'shares': 100},
  {'name': 'CAT', 'price': 83.44, 'shares': 150},
  {'name': 'GE', 'price': 40.37, 'shares': 95},
  {'name': 'IBM', 'price': 91.1, 'shares': 50},
  {'name': 'IBM', 'price': 70.44, 'shares': 100},
  {'name': 'MSFT', 'price': 51.23, 'shares': 200},
  {'name': 'MSFT', 'price': 65.1, 'shares': 50}
]
```

콜백 함수(Callback Function)

앞의 예에서 키 함수는 콜백 함수의 일종이다. sort() 메서드는 당신이 제공한 함수를 콜백한다. 콜백 함수는 짧은 한 줄 짜리 함수로 작성해 단 하나의 오퍼레이션만을 위해 사용하는 경우가 많다. 프로그래머는 종종 이러한 추가적인 처리를 위한 지름길을 요구한다.

람다(Lambda): 익명 함수

함수를 작성하는 대신 람다를 사용하자. 앞의 정렬 예제를 가지고 해 본다.

```
portfolio.sort(key=lambda s: s['name'])
```

람다는 단일 표현식을 평가하는 이름 없는 함수를 생성한다. 위 코드는 함수로 만들었을 때보다 훨씬 짧다.

```
def stock_name(s):
    return s['name']

portfolio.sort(key=stock_name)

# 람다와 비교
portfolio.sort(key=lambda s: s['name'])
```

람다 사용하기

람다는 매우 제한적이다.

단일 표현식만 허용한다.

if, while 같은 문장은 허용하지 않는다.

sort()의 예와 같이 함수와 함께 사용하는 것이 일반적인 용도다.

연습 문제

주식 포트폴리오 데이터를 읽어 리스트로 변환하자.

```
>>> import report
>>> portfolio = list(report.read_portfolio('Data/portfolio.csv'))
>>> for s in portfolio:
```

```
        print(s)

Stock('AA', 100, 32.2)
Stock('IBM', 50, 91.1)
Stock('CAT', 150, 83.44)
Stock('MSFT', 200, 51.23)
Stock('GE', 95, 40.37)
Stock('MSFT', 50, 65.1)
Stock('IBM', 100, 70.44)
>>>
```

연습 문제 7.5: 필드 정렬하기
포트폴리오 데이터를 종목명을 기준으로 알파벳순으로 정렬해 보자.

```
>>> def stock_name(s):
        return s.name

>>> portfolio.sort(key=stock_name)
>>> for s in portfolio:
            print(s)

... 결과를 확인하라 ...
>>>
```

위의 stock_name() 함수는 portfolio 리스트의 단일 항목에서 종목명을 추출한다. sort()는 이 함수의 결과를 비교해 정렬한다.

연습 문제 7.6: 람다를 가지고 필드 정렬하기
lambda 표현식을 사용해 주식 수를 기준으로 포트폴리오를 정렬해보자.

```
>>> portfolio.sort(key=lambda s: s.shares)
>>> for s in portfolio:
        print(s)

... 결과를 확인하라 ...
>>>
```

주식 가격을 기준으로 포트폴리오를 정렬해보자.

```
>>> portfolio.sort(key=lambda s: s.price)
>>> for s in portfolio:
        print(s)

... 결과를 확인하라 ...
```

```
>>>
```
참고: lambda는 별도의 함수를 따로 정의할 필요 없이 sort()를 호출할 때 특수한 처리 함수를 정의할 수 있어 유용하다.

마지막 편집일시 : 2020년 10월 3일 6:02

7.3 함수를 반환하기

이 섹션은 함수를 사용해 다른 함수를 생성하는 아이디어를 소개한다.

도입

다음 함수를 생각해보자.

```
def add(x, y):
    def do_add():
        print('Adding', x, y)
        return x + y
    return do_add
```
add() 함수는 또다른 함수인 do_add를 반환한다.

```
>>> a = add(3,4)
>>> a
<function do_add at 0x6a670>
>>> a()
Adding 3 4
7
```
로컬 변수

외부 함수가 정의한 변수를 내부 함수가 어떻게 참조하는지 관찰하자.

```
def add(x, y):
    def do_add():
        # `x`와 `y`는 `add(x, y)`에 정의
        print('Adding', x, y)
        return x + y
    return do_add
```
좀 더 관찰해보면 이 변수들이 add()가 완료된 후에도 살아 있는 것을 볼 수 있다.

```
>>> a = add(3,4)
>>> a
<function do_add at 0x6a670>
>>> a()
Adding 3 4        # 이 변수들은 어디서 왔을까?
```

7
클로저
내부 함수를 결과로 반환할 때, 그 내부 함수를 클로저(closure)라고 부른다.

```
def add(x, y):
    # `do_add`는 클로저다
    def do_add():
        print('Adding', x, y)
        return x + y
    return do_add
```

필수 기능: 클로저는 나중에 함수가 올바로 작동하는 데 필요한 모든 변수의 값을 유지한다.
클로저를 함수와 그 함수가 의존하는 변숫값을 저장하는 환경이 합쳐진 것으로 생각할 수 있다.

클로저 사용하기
클로저는 파이썬의 중요 기능이다. 그런데 그 용법은 미묘한 데가 있다. 공통적인 응용은 다음과 같다.

콜백 함수에 사용.
지연된 평가.
데코레이터 함수(뒤에서 설명한다).
지연된 평가(Delayed Evaluation)
다음과 같은 함수를 생각해보자.

```
def after(seconds, func):
    time.sleep(seconds)
    func()
```

용례:

```
def greeting():
    print('Hello Guido')

after(30, greeting)
```

after는 제공된 함수를 실행한다... 나중에.

클로저는 추가적인 정보를 보유한다.

```
def add(x, y):
    def do_add():
        print(f'Adding {x} + {y} -> {x+y}')
    return do_add
```

```
def after(seconds, func):
    time.sleep(seconds)
    func()

after(30, add(2, 3))
# `do_add`는 레퍼런스 x -> 2와 y -> 3를 가짐
```

코드 반복

과도한 코드 반복을 피하는 기법으로서 클로저를 사용할 수도 있다. 코드를 작성하는 함수를 작성할 수 있다.

연습 문제

연습 문제 7.7: 반복을 피하기 위해 클로저 사용하기

클로저는 반복적인 코드를 생성하는 기능이 뛰어나다. 연습 문제 5.7에서 타입을 검사하는 프로퍼티를 정의한 코드를 작성했다.

```
class Stock:
    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price
    ...
    @property
    def shares(self):
        return self._shares

    @shares.setter
    def shares(self, value):
        if not isinstance(value, int):
            raise TypeError('Expected int')
        self._shares = value
    ...
```

코드를 일일이 타이핑하는 대신, 클로저를 사용해 자동으로 생성할 수 있다.

typedproperty.py 파일을 만들어 다음 코드를 넣자.

```
# typedproperty.py

def typedproperty(name, expected_type):
    private_name = '_' + name
    @property
```

```
    def prop(self):
        return getattr(self, private_name)

    @prop.setter
    def prop(self, value):
        if not isinstance(value, expected_type):
            raise TypeError(f'Expected {expected_type}')
        setattr(self, private_name, value)

    return prop
```

이제 다음과 같은 클래스를 정의해보자.

```
from typedproperty import typedproperty

class Stock:
    name = typedproperty('name', str)
    shares = typedproperty('shares', int)
    price = typedproperty('price', float)

    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price
```

인스턴스를 생성해 타입 검사가 잘 되는지 확인해 보자.

```
>>> s = Stock('IBM', 50, 91.1)
>>> s.name
'IBM'
>>> s.shares = '100'
... TypeError가 발생한다 ...
>>>
```

연습 문제 7.8: 함수 호출을 단순화히기

위의 예에서 typedproperty('shares', int)와 같이 타이핑하여 호출하기가 귀찮았을 것이다. 반복적으로 하려면 더 불편하다. typedproperty.py 파일에 다음과 같은 정의를 추가하자.

```
String = lambda name: typedproperty(name, str)
Integer = lambda name: typedproperty(name, int)
Float = lambda name: typedproperty(name, float)
```

이제 이 함수들을 사용하도록 Stock 클래스를 재작성한다.

```
class Stock:
```

```
    name = String('name')
    shares = Integer('shares')
    price = Float('price')

    def __init__(self, name, shares, price):
        self.name = name
        self.shares = shares
        self.price = price
```
아, 좀 더 낫다. 이와 같이 클로저와 lambda를 사용해 코드를 단순화하고 거슬리는 반복을 제거할 수 있다. 이렇게 하는 쪽이 좋을 때가 많다.

연습 문제 7.9: 실전에 응용
stock.py의 Stock 클래스를 재작성해, 타입 있는 프로퍼티를 사용하게 하라.

마지막 편집일시 : 2020년 10월 4일 12:28

7.4 함수 데코레이터
이 섹션은 데코레이터의 개념을 소개한다. 이것은 고급 주제이므로 이 코스에서는 겉핥기로 다룬다.

로깅 예제
다음 함수를 생각해보자.

```
def add(x, y):
    return x + y
```
이 함수에 로깅을 추가해보자.

```
def add(x, y):
    print('Calling add')
    return x + y
```
이제 두 번째 함수에도 로깅을 추가한다.

```
def sub(x, y):
    print('Calling sub')
    return x - y
```
관찰
관찰: 코드가 반복된다.

같은 코드를 복사해서 프로그램을 작성하는 것은 성가신 일이다. 작성하기도 번거롭고 유지보수도 힘들다. 특히 작동을 변경하고 싶을 때 문제가 된다(예: 로깅 작동을 바꾸기).

로깅을 만드는 코드
함수에 로깅을 추가해주는 함수를 만들 수 있을 것이다. 래퍼(wrapper) 말이다.

```
def logged(func):
    def wrapper(*args, **kwargs):
        print('Calling', func.__name__)
        return func(*args, **kwargs)
    return wrapper
```

이제 사용해보자.

```
def add(x, y):
    return x + y

logged_add = logged(add)
```

logged가 반환한 함수를 호출할 때 무슨 일이 일어나는가?

```
logged_add(3, 4)        # 로깅 메시지가 나타난다
```

이 예는 래퍼 함수(wrapper function)를 생성하는 과정을 보여준다.

래퍼는 다른 함수를 감싸는 함수다. 약간의 처리를 추가하지만, 그외에는 원래 함수와 똑같이 작동한다.

```
>>> logged_add(3, 4)
Calling add    # 부가적인 출력이 래퍼에 의해 추가됨
7
>>>
```

참고: logged() 함수는 래퍼를 생성해, 그것을 결과로 반환한다.

데코레이터
파이썬에서는 함수 주변에 래퍼를 두는 일이 매우 흔하다. 그래서 아예 전용 구문이 있다.

```
def add(x, y):
    return x + y
add = logged(add)

# 특수한 구문
@logged
def add(x, y):
    return x + y
```

이러한 특수 구문은 앞에서 했던 것과 똑같은 일을 한다. 데코레이터는 새로운 구문이다.
이를 가리켜 함수를 데코레이트한다고(decorate)한다.

부연 설명
여기서는 간단히 시연했지만, 데코레이터와 관련해 미묘한 세부사항이 많다. 클래스에서 데코레이터를 사용하는 경우를 예로 들 수 있다. 또는 함수에 데코레이터를 여러 개 붙이는 경우를 생각해 볼 수 있다. 그렇기는 해도, 앞의 예는 데코레이터를 어떻게 사용하는지 잘 나타낸다. 광범위한 함수 정의에 반복적으로 나타나는 코드에 대응하는 것이 일반적인 용도다. 데코레이터를 이용하면 그러한 코드를 한 곳에 모아둘 수 있다.

연습 문제
연습 문제 7.10: 타이밍을 위해 데코레이터 사용하기
어떤 함수를 정의하면, 그 이름과 모듈이 __name__과 __module__ 어트리뷰트에 저장된다. 예:

```
>>> def add(x,y):
      return x+y

>>> add.__name__
'add'
>>> add.__module__
'__main__'
>>>
```

timethis.py 파일에서, 함수를 수행하는 데 얼마나 걸리는지 프린트하는 추가적인 논리 계층으로 감싸는 데코레이터 함수 timethis(func)를 작성하자. 이를 위해 다음과 같은 타이밍 호출로 함수를 감싼다.

```
start = time.time()
r = func(*args,**kwargs)
end = time.time()
print('%s.%s: %f' % (func.__module__, func.__name__, end-start))
```

데코레이터가 다음과 같이 작동하게 하라.

```
>>> from timethis import timethis
>>> @timethis
def countdown(n):
        while n > 0:
              n -= 1

>>> countdown(10000000)
__main__.countdown : 0.076562
>>>
```

논의: 이 @timethis 데코레이터는 어느 함수 정의의 앞에든지 올 수 있다. 따라서,

데코레이터를 성능 튜닝을 위한 진단 도구로 사용할 수 있다.

마지막 편집일시 : 2021년 3월 5일 4:32

7.5 데코레이트된 메서드
이 섹션은 메서드 정의와 함께 사용할 수 있는 빌트인 데코레이터 몇 가지를 논의한다.

미리 정의된 데코레이터
클래스 정의에 특수한 메서드 유형을 지정하는 데 사용할 수 있는 데코레이터가 미리 정의돼있다

```
class Foo:
    def bar(self,a):
        ...

    @staticmethod
    def spam(a):
        ...

    @classmethod
    def grok(cls,a):
        ...

    @property
    def name(self):
        ...
```
하나씩 살펴보자.

정적 메서드
@staticmethod는 정적(static) 클래스 메서드(C++/Java에서 유래)를 정의하는 데 사용한다. 정적 메서드는 클래스에 속한 함수이지만 인스턴스에 대해 연산을 하지 않는다.

```
class Foo(object):
    @staticmethod
    def bar(x):
        print('x =', x)

>>> Foo.bar(2)
x=2
>>>
```
클래스를 위한 내부 지원 코드를 구현하는 데 정적 메서드를 종종 사용한다. 생성한

인스턴스를 관리하는 코드를 예로 들 수 있다(메모리 관리, 시스템 자원, 지속성, 잠금 등). 특정 설계 패턴에서도 사용한다(여기서는 다루지 않는다).

클래스 메서드
@classmethod는 클래스 메서드를 정의하는 데 사용한다. 클래스 메서드는 인스턴스 대신 class 객체를 첫 번째 매개변수로 받는다.

```
class Foo:
    def bar(self):
        print(self)

    @classmethod
    def spam(cls):
        print(cls)

>>> f = Foo()
>>> f.bar()
<__main__.Foo object at 0x971690>    # 인스턴스 `f`
>>> Foo.spam()
<class '__main__.Foo'>                # `Foo` 클래스
>>>
```

생성자(constructor)의 대안으로 클래스 메서드를 종종 사용한다.

```
class Date:
    def __init__(self,year,month,day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def today(cls):
        # 클래스가 어떻게 인자로 전달되는지 보라
        tm = time.localtime()
        # 새 인스턴스를 생성하는 데 사용됨
        return cls(tm.tm_year, tm.tm_mon, tm.tm_mday)

d = Date.today()
```

클래스 메서드는 상속과 같이 까다로운 문제를 해결한다.

```
class Date:
    ...
```

```
    @classmethod
    def today(cls):
        # 올바른 클래스를 얻음(예: `NewDate`)
        tm = time.localtime()
        return cls(tm.tm_year, tm.tm_mon, tm.tm_mday)

class NewDate(Date):
    ...

d = NewDate.today()
```

연습 문제

연습 문제 7.11: 클래스 메서드 실습

report.py와 portfolio.py 파일에서 Portfolio 객체의 생성이 다소 애매하다. 예를 들어, report.py 프로그램에는 다음과 같은 코드가 있다.

```
def read_portfolio(filename, **opts):
    '''
    주식 포트폴리오 파일을 읽어 딕셔너리의 리스트를 생성.
    name, shares, price를 키로 사용.
    '''
    with open(filename) as lines:
        portdicts = fileparse.parse_csv(lines,
                                        select=['name','shares','price'],
                                        types=[str,int,float],
                                        **opts)

    portfolio = [ Stock(**d) for d in portdicts ]
    return Portfolio(portfolio)
```

그리고 portfolio.py 파일은 Portfolio()를 정의하는데, 초기화가 약간 이상하다.

```
class Portfolio:
    def __init__(self, holdings):
        self.holdings = holdings
    ...
```

솔직히 코드가 여기저기 흩어져 있어 책임 관계가 헷갈린다. Portfolio 클래스에 Stock 인스턴스의 리스트를 둘 것이라면, 클래스를 좀 더 명확하게 바꾸는 것이 좋겠다. 이렇게 말이다.

```
# portfolio.py

import stock
```

```
class Portfolio:
    def __init__(self):
        self.holdings = []

    def append(self, holding):
        if not isinstance(holding, stock.Stock):
            raise TypeError('Expected a Stock instance')
        self.holdings.append(holding)
    ...
```

CSV 파일에서 포트폴리오를 읽고 싶다면, 그것을 위한 클래스 메서드를 만들어야 할 것이다.

```
# portfolio.py

import fileparse
import stock

class Portfolio:
    def __init__(self):
        self.holdings = []

    def append(self, holding):
        if not isinstance(holding, stock.Stock):
            raise TypeError('Expected a Stock instance')
        self.holdings.append(holding)

    @classmethod
    def from_csv(cls, lines, **opts):
        self = cls()
        portdicts = fileparse.parse_csv(lines,
                                        select=['name','shares','price'],
                                        types=[str,int,float],
                                        **opts)

        for d in portdicts:
            self.append(stock.Stock(**d))

        return self
```

이 새로운 포트폴리오 클래스를 사용하기 위해, 이제 다음과 같은 코드를 작성할 수 있다.

```
>>> from portfolio import Portfolio
```

```
>>> with open('Data/portfolio.csv') as lines:
...     port = Portfolio.from_csv(lines)
...
>>>
```

Portfolio 클래스에 이러한 변경을 반영하고 클래스 메서드를 사용하게 report.py 코드를 수정하라.

마지막 편집일시 : 2020년 8월 6일 9:40

---

8. 테스팅과 디버깅

이 절은 테스팅, 로깅, 디버깅과 관련한 새로운 기초 주제를 소개한다.

- 8.1 테스팅
- 8.2 로깅, 오류 처리 및 진단
- 8.3 디버깅

마지막 편집일시 : 2020년 6월 27일 7:00

8.1 테스팅

테스팅 □, 디버깅 □

파이썬은 동적 특성으로 인해 애플리케이션 대부분에 있어 테스팅이 매우 중요하다. 컴파일러로는 버그를 찾을 수 없다. 버그를 찾는 유일한 방법은 코드를 실행해 모든 기능을 점검하는 것이다.

assert 문

assert 문은 프로그램의 내부 점검이다. 표현식이 참이 아니면 AssertionError 예외가 발생한다.

assert 문의 구문은 다음과 같다.

assert <표현식> [, '진단 메시지']

예:

```
assert isinstance(10, int), 'Expected int'
```

사용자 입력(예: 웹 양식 같은 것을 통해 입력한 데이터)을 검사하는 데 assert를 사용하면 안 된다. 내부적인 점검과 불변 조건(항상 참인 조건)을 검사하는 것이 목적이다.

계약 프로그래밍(Contract Programming)

assert를 아낌없이 활용하는 소프트웨어 설계 접근법으로, 계약에 의한 설계(Design By Contract)라고도 한다. 소프트웨어 설계자가 소프트웨어 구성요소의 정확한 인터페이스 사양을 정의해야 한다고 규정한다.

예를 들어, 함수의 모든 입력에 assert를 넣을 수 있다.

```
def add(x, y):
    assert isinstance(x, int), 'Expected int'
    assert isinstance(y, int), 'Expected int'
    return x + y
```

입력값 검사는 적절한 인자를 사용하지 않은 호출자를 즉시 잡아낸다.

```
>>> add(2, 3)
5
>>> add('2', '3')
Traceback (most recent call last):
...
AssertionError: Expected int
>>>
```

인라인 테스트

assert를 가지고 단순한 테스트를 할 수도 있다.

```
def add(x, y):
    return x + y

assert add(2,2) == 4
```

이런 식으로 테스트 코드를 같은 모듈에 포함한다.

장점: 코드가 명백히 망가진 경우 임포트 시도 자체가 실패한다.

테스트를 제대로 하려면 이 방식만으로는 부족하다. 이것은 심각한 오류를 찾아내는 기본 점검(smoke test)에 불과하다. 이 함수는 모든 예제에서 작동하는가? 그렇지 않다면 뭔가 잘못된 것이 분명하다.

unittest 모듈

다음과 같은 코드가 있다고 하자.

```
# simple.py

def add(x, y):
    return x + y
```

이제 이것을 테스트한다고 하자. 다음과 같이 테스트 파일을 따로 생성한다.

```
# test_simple.py
```

```
import simple
import unittest
```
그런 다음, 테스트 클래스를 정의한다.

```
# test_simple.py

import simple
import unittest

# unittest.TestCase로부터 상속함에 유의
class TestAdd(unittest.TestCase):
    ...
```
테스트 클래스는 반드시 unittest.TestCase로부터 상속해야 한다.

테스트 클래스에 테스트 메서드를 정의한다.

```
# test_simple.py

import simple
import unittest

# unittest.TestCase로부터 상속함에 유의
class TestAdd(unittest.TestCase):
    def test_simple(self):
        # 단순한 정수 인자를 가지고 테스트
        r = simple.add(2, 2)
        self.assertEqual(r, 5)
    def test_str(self):
        # 문자열을 가지고 테스트
        r = simple.add('hello', 'world')
        self.assertEqual(r, 'helloworld')
```
*중요: 메서드 이름이 test로 시작해야 한다.

unittest 사용하기
unittest 모듈에 몇 가지 검사 함수가 기본으로 들어있다. 그것들은 각기 다른 일을 한다.

```
# 표현이 참인지 검사
self.assertTrue(표현)

# x == y를 검사
self.assertEqual(x,y)
```

```
# x != y를 검사
self.assertNotEqual(x,y)

# x와 y가 비슷한지 검사
self.assertAlmostEqual(x,y,자릿수)

# callable(인자1,인자2,...)이 예외를 일으키는지 검사
self.assertRaises(예외, callable, 인자1, 인자2, ...)
```

이것이 다가 아니다. 모듈에 다른 검사가 더 있다.

unittest 실행하기

테스트를 실행하기 위해 코드를 스크립트로 바꾸자.

```
# test_simple.py

...

if __name__ == '__main__':
    unittest.main()
```

그 다음에 파이썬으로 테스트 파일을 실행한다.

```
bash % python3 test_simple.py
F.
======================================================
FAIL: test_simple (__main__.TestAdd)
------------------------------------------------------
Traceback (most recent call last):
  File "testsimple.py", line 8, in test_simple
    self.assertEqual(r, 5)
AssertionError: 4 != 5
------------------------------------------------------
Ran 2 tests in 0.000s
FAILED (failures=1)
```

부연 설명

효과적인 테스팅은 예술에 가까우며, 대규모 애플리케이션에서는 꽤 복잡해질 수 있다.

unittest 모듈은 테스트 실행과 결과 수집 등 테스팅의 여러 측면에 유용한 옵션이 아주 많다. 자세한 사항은 문서를 참조하라.

서드 파티 테스트 도구

unittest 모듈은 파이썬에 빌트인되어 있으므로 어디서나 사용할 수 있다는 장점이 있다. 그렇지만, 너무 장황하다고 생각하는 프로그래머가 많다. unittest 대신 pytest도 많이 사용된다. pytest를 사용하면 테스팅 파일이 다음과 같이 단순해진다.

```
# test_simple.py
import simple

def test_simple():
    assert simple.add(2,2) == 4

def test_str():
    assert simple.add('hello','world') == 'helloworld'
```

테스트를 실행하려면 명령행에서 python -m pytest와 같이 타이핑하면 된다. 모든 테스트를 찾아서 실행해준다.

pytest 예제가 많이 있지만 여기서는 다루지 않는다. 사용하기 쉬우므로 한번 시도해보라.

연습 문제

이 연습 문제에서는 파이썬 unittest 모듈의 기본 작동 원리를 탐구한다.

앞의 연습 문제에서 Stock 클래스가 있는 stock.py를 작성했다. 이 연습 문제는 타입 있는 프로퍼티와 관련된 연습 문제 7.9를 풀었다고 가정한다. 만약 어떤 이유로든 작동하지 않는다면 Solutions/7_9의 해답 코드를 복사해서 사용해도 된다.

연습 문제 8.1: 유닛 테스트 작성하기

Stock 클래스에 대한 단위 테스트를 test_stock.py 파일에 따로 작성한다. 인스턴스 생성을 검사하는 코드는 다음과 같다.

```
# test_stock.py

import unittest
import stock

class TestStock(unittest.TestCase):
    def test_create(self):
        s = stock.Stock('GOOG', 100, 490.1)
        self.assertEqual(s.name, 'GOOG')
        self.assertEqual(s.shares, 100)
        self.assertEqual(s.price, 490.1)

if __name__ == '__main__':
```

```
    unittest.main()
```
단위 테스트를 실행하자. 다음과 같이 출력된다.

```
.
----------------------------------------------------------------------
Ran 1 tests in 0.000s

OK
```
이것이 잘 작동하면, 다음 사항을 점검하는 단위 테스트를 추가로 작성한다.

s.cost 프로퍼티가 올바른 값 (49010.0)을 반환하는지 확인한다.
s.sell() 메서드가 올바로 작동하는지 확인한다. s.shares 값이 감소해야 한다.
s.shares 어트리뷰트에 정수가 아닌 값이 설정되지 않음을 확인한다.
끝으로, 예외가 일어나는지 확인해야 한다. 다음과 같은 코드를 가지고 하면 쉽다.

```
class TestStock(unittest.TestCase):
    ...
    def test_bad_shares(self):
        s = stock.Stock('GOOG', 100, 490.1)
        with self.assertRaises(TypeError):
            s.shares = '100'
```
마지막 편집일시 : 2020년 10월 3일 1:02

8.2 로깅
이 섹션에서는 logging 모듈을 간단히 소개한다.

logging 모듈
logging 모듈은 진단 정보를 기록하기 위한 표준 라이브러리 모듈이다. 정교한 기능이 많은, 매우 큰 모듈이기도 하다. 유용한 몇 가지 예를 살펴보자.

예외 되돌아보기
연습 문제에서 다음과 같은 parse() 함수를 작성했다.

```
# fileparse.py
def parse(f, types=None, names=None, delimiter=None):
    records = []
    for line in f:
        line = line.strip()
        if not line: continue
        try:
            records.append(split(line,types,names,delimiter))
```

```
        except ValueError as e:
            print("Couldn't parse :", line)
            print("Reason :", e)
    return records
```

try-except 문에 집중하자. except 블록에서 무슨 일을 해야 할까?

경고 메시지를 출력해야 하나?

```
try:
    records.append(split(line,types,names,delimiter))
except ValueError as e:
    print("Couldn't parse :", line)
    print("Reason :", e)
```

아니면 조용히 무시해야 하나?

```
try:
    records.append(split(line,types,names,delimiter))
except ValueError as e:
    pass
```

두 가지 행위를 모두 원할 수 있으므로(사용자가 선택) 어느 해결책도 만족스럽지 않을 것이다.

logging 모듈 사용하기

logging 모듈을 사용해 해결할 수 있다.

```
# fileparse.py
import logging
log = logging.getLogger(__name__)

def parse(f,types=None,names=None,delimiter=None):
    ...
    try:
        records.append(split(line,types,names,delimiter))
    except ValueError as e:
        log.warning("Couldn't parse : %s", line)
        log.debug("Reason : %s", e)
```

경고 메시지 또는 특수한 Logger 객체를 발행하도록 코드를 수정한다.

logging.getLogger(__name__)으로 생성한 것이다.

로깅 기초

로거 객체를 생성한다.

log = logging.getLogger(name) # name은 문자열이다
로그 메시지를 발행한다.

log.critical(message [, args])
log.error(message [, args])
log.warning(message [, args])
log.info(message [, args])
log.debug(message [, args])
각 메시지는 심각도(severity) 수준이 각기 다르다.

이것들은 모두 포매팅된 로그 메시지를 생성한다. 메시지를 생성하기 위해 args를 % 연산자와 함께 사용한다.

logmsg = message % args # 로그에 기록됨
로깅 설정
로깅 작동을 각각 설정할 수 있다.

```
# main.py

...

if __name__ == '__main__':
    import logging
    logging.basicConfig(
        filename  = 'app.log',       # Log output file
        level     = logging.INFO,    # Output level
    )
```

일반적으로, 프로그램이 시작할 때 한 번만 설정한다. 설정은 logging을 호출하는 코드와 분리된다.

부연 설명
로깅은 설정할 수 있는 폭이 넓다. 출력 파일, 수준, 메시지 포맷 등 모든 것을 설정할 수 있다. 그렇지만, logging을 사용하는 코드는 그런 것에 신경쓰지 않아도 된다.

연습 문제
연습 문제 8.2: 모듈에 logging 추가하기
fileparse.py에는 잘못된 입력에 의해 발생하는 예외와 관련해 몇 가지 오류 처리가 있다. 다음과 같다.

```
# fileparse.py
```

```
import csv

def parse_csv(lines, select=None, types=None, has_headers=True, delimiter=',',
silence_errors=False):
    '''
    CSV 파일을 파싱 및 형변환하여 레코드의 리스트를 생성.
    '''
    if select and not has_headers:
        raise RuntimeError('select requires column headers')

    rows = csv.reader(lines, delimiter=delimiter)

    # 파일 헤더가 있으면 읽음
    headers = next(rows) if has_headers else []

    # 특정 컬럼을 선택한 경우, 필터링을 위한 인덱스를 만들고 출력 컬럼을 설정
    if select:
        indices = [ headers.index(colname) for colname in select ]
        headers = select

    records = []
    for rowno, row in enumerate(rows, 1):
        if not row:      # 데이터가 없는 행을 건너뜀
            continue

        # 특정 컬럼 인덱스가 선택되었으면 그것을 고름
        if select:
            row = [ row[index] for index in indices]

        # 행에 형변환을 적용
        if types:
            try:
                row = [func(val) for func, val in zip(types, row)]
            except ValueError as e:
                if not silence_errors:
                    print(f"Row {rowno}: Couldn't convert {row}")
                    print(f"Row {rowno}: Reason {e}")
                continue

        # 튜플의 딕셔너리를 만듦
        if headers:
```

```
            record = dict(zip(headers, row))
        else:
            record = tuple(row)
        records.append(record)

    return records
```

진단 메시지를 발행하는 print 문에 유의하라. 이 print를 logging 오퍼레이션으로 교체하는 것은 간단한 편이다. 코드를 다음과 같이 바꾸자.

```
# fileparse.py
import csv
import logging
log = logging.getLogger(__name__)

def parse_csv(lines, select=None, types=None, has_headers=True, delimiter=',',
silence_errors=False):
    '''
    CSV 파일을 파싱 및 형변환하여 레코드의 리스트를 생성.
    '''
    if select and not has_headers:
        raise RuntimeError('select requires column headers')

    rows = csv.reader(lines, delimiter=delimiter)

    # 파일 헤더가 있으면 읽음
    headers = next(rows) if has_headers else []

    # 특정 컬럼을 선택한 경우, 필터링을 위한 인덱스를 만들고 출력 컬럼을 설정
    if select:
        indices = [ headers.index(colname) for colname in select ]
        headers = select

    records = []
    for rowno, row in enumerate(rows, 1):
        if not row:      # 데이터가 없는 행을 건너뜀
            continue

        # 특정 컬럼 인덱스가 선택되었으면 그것을 고름
        if select:
            row = [ row[index] for index in indices]
```

```
        # 행에 형변환을 적용
        if types:
            try:
                row = [func(val) for func, val in zip(types, row)]
            except ValueError as e:
                if not silence_errors:
                    log.warning("Row %d: Couldn't convert %s", rowno, row)
                    log.debug("Row %d: Reason %s", rowno, e)
                continue

        # 튜플의 딕셔너리를 만듦
        if headers:
            record = dict(zip(headers, row))
        else:
            record = tuple(row)
        records.append(record)

    return records
```

이렇게 변경했으므로, 잘못된 데이터를 입력해보자.

```
>>> import report
>>> a = report.read_portfolio('Data/missing.csv')
Row 4: Bad row: ['MSFT', '', '51.23']
Row 7: Bad row: ['IBM', '', '70.44']
>>>
```

기본값으로는 경고(WARNING) 수준 이상의 로깅 메시지만 출력된다. 출력은 단순한 print 문처럼 보인다. 그렇지만 logging 모듈을 설정하는 경우 로깅 수준, 모듈 등의 추가적인 정보가 제공된다. 다음과 같이 타이핑하여 확인해보자.

```
>>> import logging
>>> logging.basicConfig()
>>> a = report.read_portfolio('Data/missing.csv')
WARNING:fileparse:Row 4: Bad row: ['MSFT', '', '51.23']
WARNING:fileparse:Row 7: Bad row: ['IBM', '', '70.44']
>>>
```

log.debug()의 출력은 나타나지 않는다. 디버그(DEBUG) 메시지도 나타나도록 수준을 변경해보자.

```
>>> logging.getLogger('fileparse').level = logging.DEBUG
>>> a = report.read_portfolio('Data/missing.csv')
WARNING:fileparse:Row 4: Bad row: ['MSFT', '', '51.23']
```

```
DEBUG:fileparse:Row 4: Reason: invalid literal for int() with base 10: ''
WARNING:fileparse:Row 7: Bad row: ['IBM', '', '70.44']
DEBUG:fileparse:Row 7: Reason: invalid literal for int() with base 10: ''
>>>
```

심각(CRITICAL) 메시지만 나타나게 해 보자.

```
>>> logging.getLogger('fileparse').level=logging.CRITICAL
>>> a = report.read_portfolio('Data/missing.csv')
>>>
```

연습 문제 8.3: 프로그램에 로깅을 추가하기

애플리케이션에 로깅을 추가하려면 메인 모듈에서 로깅 모듈을 초기화하는 메커니즘이 필요하다. 다음과 같은 셋업 코드를 포함하는 방식으로 할 수 있다.

```
# 이 파일은 logging 모듈의 기본 구성을 설정한다.
# 이 설정을 변경함으로써 로깅 출력을 원하는 대로 조정할 수 있다.
import logging
logging.basicConfig(
    filename = 'app.log',              # 로그 파일의 이름(생략하면 stderr을 사용)
    filemode = 'w',                    # 파일 모드('a': 추가)
    level    = logging.WARNING,        # 로깅 수준(DEBUG, INFO, WARNING, ERROR,
CRITICAL)
)
```

당신의 프로그램의 시작 단계에 이것을 두어야 한다. 예를 들어, report.py 프로그램의 어디에 두어야 할까?

마지막 편집일시 : 2020년 7월 30일 5:38

8.3 디버깅

프로그램이 충돌했다...

```
bash % python3 blah.py
Traceback (most recent call last):
  File "blah.py", line 13, in ?
    foo()
  File "blah.py", line 10, in foo
    bar()
  File "blah.py", line 7, in bar
    spam()
  File "blah.py", 4, in spam
    line x.append(3)
AttributeError: 'int' object has no attribute 'append'
```

어떡하지?!

트레이스백 읽기
마지막 행이 문제를 일으킨 주범이다.

```
bash % python3 blah.py
Traceback (most recent call last):
  File "blah.py", line 13, in ?
    foo()
  File "blah.py", line 10, in foo
    bar()
  File "blah.py", line 7, in bar
    spam()
  File "blah.py", 4, in spam
    line x.append(3)
# 충돌 원인
AttributeError: 'int' object has no attribute 'append'
```

그렇지만 읽고 이해하기가 쉽지만은 않다.

전문가의 조언: 트레이스백 전체를 구글에 붙여넣어 검색하라.

REPL 사용하기
-i 옵션을 사용해 스크립트를 사용할 때 파이썬이 살아있도록 한다.

```
bash % python3 -i blah.py
Traceback (most recent call last):
  File "blah.py", line 13, in ?
    foo()
  File "blah.py", line 10, in foo
    bar()
  File "blah.py", line 7, in bar
    spam()
  File "blah.py", 4, in spam
    line x.append(3)
AttributeError: 'int' object has no attribute 'append'
>>>
```

이것은 인터프리터 상태를 유지한다. 충돌 이후에 여기저기 찔러볼 수 있다. 변숫값과 기타 상태를 확인한다.

프린트를 사용한 디버깅
print() 디버깅은 꽤 일반적이다.

조언: repr()을 사용하라.

```
def spam(x):
    print('DEBUG:', repr(x))
    ...
```

repr()은 값의 정확한 표현(representation)을 나타낸다. 출력을 예쁘게 프린트하는 것과는 다르다.

```
>>> from decimal import Decimal
>>> x = Decimal('3.4')
# `repr` 없이
>>> print(x)
3.4
# `repr` 사용
>>> print(repr(x))
Decimal('3.4')
>>>
```

파이썬 디버거

이 프로그램에서 디버거를 수작업으로 실행할 수 있다.

```
def some_function():
    ...
    breakpoint()        # 디버거 진입(파이썬 3.7 이상)
    ...
```

이것은 breakpoint() 호출에서 디버거를 시작한다.

이전 버전에서는 방법이 약간 다르다. 다른 디버깅 안내서에서 다음과 같이 설명하는 것을 볼 수 있을 것이다.

```
import pdb
...
pdb.set_trace()         # `breakpoint()` 대신 이것을 사용하라
...
```

디버거에서 실행하기

전체 프로그램을 디버거에서 실행할 수도 있다.

```
bash % python3 -m pdb someprogram.py
```

이렇게 하면 첫 번째 문장 이전에 디버거에 자동으로 진입한다. 중단점(breakpoint)을 설정하고 구성을 바꿔볼 수 있다.

일반적인 디버거 명령:

```
(Pdb) help              # 도움말
(Pdb) w(here)           # 스택 트레이스(stack trace)를 프린트
(Pdb) d(own)            # 한 스택 수준 아래로 이동
(Pdb) u(p)              # 한 스택 수준 위로 이동
(Pdb) b(reak) loc       # 중단점 설정
(Pdb) s(tep)            # 한 인스트럭션(instruction)을 실행
(Pdb) c(ontinue)        # 계속 실행
(Pdb) l(ist)            # 소스 코드 보기
(Pdb) a(rgs)            # 현재 함수의 인자를 프린트
(Pdb) !statement        # 문장(statement)을 실행
```

중단점 위치는 다음 중 하나다.

```
(Pdb) b 45              # 현재 파일의 45행
(Pdb) b file.py:34      # file.py의 34행
(Pdb) b foo             # 현재 파일의 foo() 함수
(Pdb) b module.foo      # module의 foo()
```

연습 문제

연습 문제 8.4: 버그? 무슨 버그?

작동한다. 출시해라!

마지막 편집일시 : 2020년 10월 3일 5:40

==========

9. 패키지

이 과정의 마지막 섹션은 코드를 패키지 구조로 조직화하는 방법에 대한 몇 가지 세부 사항을 알아본다. 서드 파티 패키지를 설치하는 방법과 여러분이 작성한 코드를 다른 사람들을 위해 제공하는 방법도 논의한다.

패키징은 계속 진화하며, 파이썬 개발에 있어 너무 복잡한 주제다. 특정 도구에 초점을 맞추기보다는, 코드를 제공하거나 의존성을 관리함에 있어 어떤 도구를 사용하더라도 적용할 수 있는 일반적인 코드 조직화 원칙에 집중한다.

- 9.1 패키지
- 9.2 서드 파티 모듈
- 9.3 코드를 다른 사람에게 주기

마지막 편집일시 : 2020년 7월 26일 12:27

9.1 패키지

더 큰 프로그램을 작성한다면, 독립적인 파일의 거대한 모음을 최상위 수준에서 조직화하는 것은 바람직하지 않다. 이 섹션은 패키지의 개념을 소개한다.

모듈(Module)

모든 파이썬 소스 파일은 모듈이다.

```
# foo.py
def grok(a):
    ...
def spam(b):
    ...
```

import 문은 모듈을 적재하고 실행한다.

```
# program.py
import foo

a = foo.grok(2)
b = foo.spam('Hello')
...
```

패키지 vs 모듈

더 큰 코드 모음에서는 모듈을 패키지로 조직화하는 것이 일반적이다.

```
# 이것을
pcost.py
report.py
fileparse.py

# 이렇게
porty/
    __init__.py
    pcost.py
    report.py
    fileparse.py
```

적당한 이름을 골라 최상위 디렉터리를 만들어라. 위의 예에서는 porty로 정했다(이름을 잘 고르는 것이 가장 중요한 첫걸음이다).

__init__.py 파일을 디렉터리에 추가한다. 디렉터리가 비어 있을 것이다.

소스 파일을 디렉터리에 넣어라.

패키지 사용하기

패키지는 임포트를 위한 네임스페이스 역할을 한다.

즉, 여러 수준의 임포트가 존재한다.

```
import porty.report
port = porty.report.read_portfolio('port.csv')
```
import 문에는 여러 변형이 있다.

```
from porty import report
port = report.read_portfolio('portfolio.csv')

from porty.report import read_portfolio
port = read_portfolio('portfolio.csv')
```
두 가지 문제점
이러한 접근은 두 가지 문제가 있다.

같은 패키지에서 여러 파일을 임포트하면 문제가 생긴다.
패키지에 있는 메인 스크립트가 깨진다.
따라서 모든 것이 깨진다. 그렇지만 그외에는 작동한다.

문제: 임포트
같은 패키지에서 파일들을 임포트하려면 임포트에 패키지 이름을 포함해야 한다. 구조를 기억하자.

```
porty/
    __init__.py
    pcost.py
    report.py
    fileparse.py
```
수정한 임포트 예.

```
# report.py
from porty import fileparse

def read_portfolio(filename):
    return fileparse.parse_csv(...)
```
모든 임포트는 상대가 아닌 절대 임포트다.

```
# report.py
import fileparse     # 깨진다. fileparse을 찾지 못한다.

...
```
상대 임포트
패키지 이름을 직접 사용하는 대신, 현재 패키지를 .로 가리킬 수 있다.

```
# report.py
from . import fileparse

def read_portfolio(filename):
    return fileparse.parse_csv(...)
```
구문:

```
from . import modname
```
이렇게 하면 패키지 이름을 바꾸기 쉽다.

문제점: 메인 스크립트
패키지 서브모듈을 메인 스크립트로서 실행하면 깨진다.

```
bash $ python porty/pcost.py # 깨짐
...
```
이유: 파이썬을 단일 파일에서 실행하는데, 파이썬은 나머지 패키지 구조를 올바로 볼 수 없기 때문이다(sys.path가 잘못됨).

모든 임포트가 깨진다. 이것을 해결하려면 프로그램을 실행할 때 -m 옵션을 사용한다.

```
bash $ python -m porty.pcost # 작동함
...
```
__init__.py 파일
이 파일의 주목적은 여러 모듈을 한데 모으는 것이다.

예: 함수들을 결속시키기

```
# porty/__init__.py
from .pcost import portfolio_cost
from .report import portfolio_report
```
이렇게 하면 임포트할 때 이름이 최상위 수준에 나타난다.

```
from porty import portfolio_cost
portfolio_cost('portfolio.csv')
```
다중 수준 임포트를 사용하는 대신.

```
from porty import pcost
pcost.portfolio_cost('portfolio.csv')
```
스크립트를 위한 솔루션
이제 패키지에서 스크립트를 실행하려면 -m package.module을 사용해야 한다.

```
bash % python3 -m porty.pcost portfolio.csv
```
또다른 방법도 있다. 새로운 최상위 수준 스크립트를 작성하는 것이다.

```
#!/usr/bin/env python3
# pcost.py
import porty.pcost
import sys
porty.pcost.main(sys.argv)
```
이 스크립트는 패키지 바깥에 있다. 예를 들어, 디렉터리 구조를 보자.

```
pcost.py        # 최상위 수준 스크립트
porty/          # 패키지 디렉터리
    __init__.py
    pcost.py
    ...
```
애플리케이션 구조

코드와 파일의 구조화는 애플리케이션 유지보수성의 핵심이다.

파이썬에서 모든 문제에 맞는 단 한 가지 방식이 있지는 않다. 그렇지만 다음과 같은 구조는 여러 가지 문제에 적합하다.

```
porty-app/
  README.txt
  script.py          # SCRIPT
  porty/
    # 라이브러리 코드
    __init__.py
    pcost.py
    report.py
    fileparse.py
```
최상위 수준의 porty-app은 문서, 최상위 수준 스크립트, 예제 등 모든 것을 담는다.

재차 말하지만, 최상위 수준 스크립트는 코드 패키지 외부에 두어야 한다. 한 단계 높이자.

```
#!/usr/bin/env python3
# porty-app/script.py
import sys
import porty

porty.report.main(sys.argv)
```
연습 문제

이제 디렉터리에는 프로그램 여러 개가 있다.

```
pcost.py            # 포트폴리오 비용 계산
report.py           # 보고서 작성
ticker.py           # 실시간 주식 시세 생성
```

그외의 기능을 지원하는 모듈도 여러 개 있다.

```
stock.py            # Stock 클래스
portfolio.py        # Portfolio 클래스
fileparse.py        # CSV 파싱
tableformat.py      # 표 작성
follow.py           # 로그 파일 추적
typedproperty.py    # 타입 있는 클래스 프로퍼티
```

연습 문제에서는 코드를 정리해 공통 패키지에 넣는다.

연습 문제 9.1: 단순한 패키지 만들기

porty/라는 디렉터리를 만들고 위의 파이썬 파일들을 모두 집어넣어라. 그 디렉터리에 빈 __init__.py 파일도 만들어라. 디렉터리에 다음과 같이 파일이 있어야 한다.

```
porty/
    __init__.py
    fileparse.py
    follow.py
    pcost.py
    portfolio.py
    report.py
    stock.py
    tableformat.py
    ticker.py
    typedproperty.py
```

디렉터리에 있는 __pycache__는 삭제한다. 앞서 프리컴파일한 파이썬 모듈이 들어 있다. 지우고 새로 시작하자.

패키지 모듈을 임포트해보자.

```
>>> import porty.report
>>> import porty.pcost
>>> import porty.ticker
```

임포트에 실패한다면 해당 파일의 모듈 임포트에 패키지 상대 임포트를 포함하게 수정한다. 예를 들어, import fileparse는 다음과 같이 바꾼다.

```
# report.py
from . import fileparse
...
```
from fileparse import parse_csv 같은 문장이 보이면, 다음과 같이 변경한다.

```
# report.py
from .fileparse import parse_csv
...
```
연습 문제 9.2: 애플리케이션 디렉터리 만들기
코드를 모두 '패키지'에 넣기만 한다고 애플리케이션이 되는 것은 아니다. 지원 파일, 문서, 스크립트, 그외의 것들이 필요하다. 이러한 파일들은 위에서 만든 porty/ 디렉터리의 바깥에 있어야 한다.

porty-app이라는 디렉터리를 새로 만들자. 연습 문제 9.1에서 생성한 porty 디렉터리를 그 디렉터리로 옮긴다. Data/portfolio.csv와 Data/prices.csv 테스트 파일을 이 디렉터리로 복사한다. 당신에 대한 정보를 담은 README.txt 파일도 추가로 작성한다. 이제 코드가 다음과 같이 되어 있을 것이다.

```
porty-app/
    portfolio.csv
    prices.csv
    README.txt
    porty/
        __init__.py
        fileparse.py
        follow.py
        pcost.py
        portfolio.py
        report.py
        stock.py
        tableformat.py
        ticker.py
        typedproperty.py
```
코드를 실행하려면 최상위 porty-app/ 디렉터리에서 작업해야 한다. 터미널에서 다음과 같이 실행한다.

```
shell % cd porty-app
shell % python3
>>> import porty.report
>>>
```
앞에서 만든 스크립트들을 메인 프로그램으로서 실행해보자.

```
shell % cd porty-app
shell % python3 -m porty.report portfolio.csv prices.csv txt
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84

shell %
```

연습 문제 9.3: 최상위 스크립트

python -m 명령을 사용하는 것은 좀 이상할 때가 있다. 패키지의 특성을 다루는 최상위 수준 스크립트를 작성하는 것이 좋다. 위의 보고서를 생성하는 print-report.py 스크립트를 작성한다.

```
#!/usr/bin/env python3
# print-report.py
import sys
from porty.report import main
main(sys.argv)
```

이 스크립트를 최상위 수준의 porty-app/ 디렉터리에 넣는다. 해당 위치에서 실행해야 한다.

```
shell % cd porty-app
shell % python3 print-report.py portfolio.csv prices.csv txt
      Name      Shares       Price      Change
---------- ---------- ---------- ----------
        AA         100        9.22      -22.98
       IBM          50      106.28       15.18
       CAT         150       35.46      -47.98
      MSFT         200       20.89      -30.34
        GE          95       13.48      -26.89
      MSFT          50       20.89      -44.21
       IBM         100      106.28       35.84

shell %
```

최종 코드는 다음과 같은 구조가 된다.

```
porty-app/
    portfolio.csv
    prices.csv
    print-report.py
    README.txt
    porty/
        __init__.py
        fileparse.py
        follow.py
        pcost.py
        portfolio.py
        report.py
        stock.py
        tableformat.py
        ticker.py
        typedproperty.py
```

마지막 편집일시 : 2020년 10월 3일 7:09

9.2 서드 파티 모듈

파이썬에는 빌트인 모듈의 방대한 라이브러리가 있다(전자제품에 배터리를 끼워 파는 것에 비유해 batteries included라고 표현한다).

게다가 서드 파티 모듈도 아주 많다. 파이썬 패키지 인덱스(Python Package Index, PyPi)를 둘러보라. 아니면 특정 주제를 구글에서 검색해도 된다.

서드 파티 의존성을 어떻게 다룰 것인지가 파이썬에서 끊임없이 진화하는 주제다. 이 섹션에서는 기본 원리를 소개하는 선에서 다룬다.

모듈 검색 경로

sys.path는 모든 디렉터리의 목록을 포함하는 디렉터리로, import 문에서 이것을 확인하다. 한번 살펴보자.

```
>>> import sys
>>> sys.path
... 결과를 살펴보라 ...
>>>
```

만약 어떤 것을 임포트하려고 했는데 그것을 이 디렉터리 중 어디에서도 찾을 수 없으면 ImportError 예외가 발생한다.

표준 라이브러리 모듈

파이썬의 표준 라이브러리에 속한 모듈은 보통 /usr/local/lib/python3.6과 같은 곳에 있다.

간단한 테스트를 통해 확인해 볼 수 있다.

```
>>> import re
>>> re
<module 're' from '/usr/local/lib/python3.6/re.py'>
>>>
```

REPL에서 모듈을 찾아보기만 해도 디버깅에 도움이 되는 정보를 얻을 수 있다. 이것은 파일의 위치를 보여준다.

서드 파티 모듈

서드 파티 모듈은 일반적으로 전용 site-packages 디렉터리에 있다. 앞에서 한 것과 같은 방법으로 찾아볼 수 있다.

```
>>> import numpy
<module 'numpy' from '/usr/local/lib/python3.6/site-packages/numpy/__init__.py'>
>>>
```

import와 관련해 예상대로 작동하지 않는 이유를 찾으려고 할 때, 모듈을 들여다보는 것은 좋은 디버깅 방법이다.

모듈 설치하기

서드 파티 모듈을 설치하는 가장 일반적인 방법은 pip를 사용하는 것이다. 예:

```
bash % python3 -m pip install packagename
```

이 명령은 패키지를 다운로드해 site-packages 디렉터리에 설치한다.

문제점

당신이 직접 제어하지 않는 파이썬을 사용 중일 수 있다.

회사 승인 설치

OS에 포함된 파이썬 버전을 사용하고 있는 경우.

당신은 컴퓨터에 글로벌 패키지를 설치할 권한이 없을 수 있다.

다른 의존성이 있을 수 있다.

가상 환경(Virtual Environment)

패키지 설치 이슈의 일반적인 해법은 자신을 위한 "가상 환경"을 생성하는 것이다. 여기에 '왕도'는 없으며, 여러 가지 도구와 기법이 경쟁한다. 하지만, 표준 파이썬 설치를 사용한다면 다음의 방법을 시도할 수 있다.

```
bash % python -m venv mypython
bash %
```

잠시 기다리면 mypython이라는 디렉터리가 새로 만들어진다. 이것을 당신만의 작은 파이썬 설치본이라고 생각할 수 있다. 그 디렉터리 안에서 bin/ 디렉터리(유닉스) 또는 Scripts/ 디렉터리(윈도우)를 찾을 수 있다. 거기 있는 activate 스크립트를 실행하면, 해당 파이썬

버전이 '활성화(activate)'되어, 셸의 디폴트 python 명령이 된다. 예:

```
bash % source mypython/bin/activate
(mypython) bash %
```

이곳에 파이썬 패키지를 설치할 수 있다. 예:

```
(mypython) bash % python -m pip install pandas
...
```

여러 패키지를 실험하고 사용해보고 싶을 때 가상 환경을 사용하면 좋다. 한편, 개발하는 애플리케이션에 특정 패키지 의존성이 필요한 경우는 약간 다른 문제가 있다.

애플리케이션에서 서드 파티 의존성을 다루기
만약 당신이 작성한 애플리케이션이 특정 서드 파티에 의존성이 있으면, 당신의 코드와 의존성을 포함하는 환경을 생성하고 보존하는 데 어려움이 있다. 안타깝게도, 파이썬에서 이 영역은 매우 혼란스럽고 빈번한 변화가 있다. 지금도 계속 진화하고 있다.

정보를 제공하더라도 금세 구식이 되어버리므로, 파이썬 패키징 사용자 안내서(Python Packaging User Guide)를 참조하기를 권한다.

연습 문제
연습 문제 9.4 : 가상 환경에 설치
가상 환경을 만들어, 위에서 설명한 판다스 설치 과정을 재현해 보라.

마지막 편집일시 : 2020년 7월 26일 12:27

9.3 배포
여러분이 작성한 코드를 동료라든지 다른 누군가에게 주고 싶을 수도 있다. 그렇게 하는 가장 기본적인 방법을 알아보자. 자세한 정보는 파이썬 패키징 사용자 안내서(Python Packaging User Guide)를 참조하라.

setup.py 파일 생성
프로젝트 디렉터리 최상위에 setup.py 파일을 추가한다.

```
# setup.py
import setuptools

setuptools.setup(
    name="porty",
    version="0.0.1",
    author="Your Name",
    author_email="you@example.com",
```

```
    description="Practical Python Code",
    packages=setuptools.find_packages(),
)
```

MANIFEST.in 생성

프로젝트와 연관된 추가 파일이 있다면 MANIFEST.in 파일에 지정한다. 예:

```
# MANIFEST.in
include *.csv
```

MANIFEST.in 파일을 setup.py와 같은 디렉터리에 둔다.

소스 배포판 생성

코드 배포판을 생성하려면 setup.py 파일을 사용한다. 예:

```
bash % python setup.py sdist
```

이것은 dist/ 디렉터리에 .tar.gz 또는 .zip 파일을 생성한다. 그 파일을 다른 사람에게 줄 수 있다.

당신의 코드를 설치

당신이 작성한 파이썬 코드를 다른 사람들이 pip를 사용해 설치하게 할 수 있다. 이전 단계에서 생성한 파일을 제공하면 된다. 예:

```
bash % python -m pip install porty-0.0.1.tar.gz
```

부연 설명

위에서 설명한 단계는 다른 사람에게 줄 수 있는 파이썬 코드의 패키지를 생성하는 최소한의 기초만 다뤘다. 현실에서는 서드 파티 의존성, 외부 코드(예: C/C++) 포함 여부 등으로 인해 훨씬 복잡하다. 그런 일들을 다루는 것은 이 코스의 범위를 벗어난다. 우리는 아주 작은 첫걸음만 딛었다.

연습 문제

연습 문제 9.5: 패키지 만들기

연습 문제 9.3에서 만든 porty-app/ 코드를 가지고 이 섹션에서 설명한 단계를 실습해보라. setup.py 파일과 MANIFEST.in 파일을 최상위 디렉터리에 추가하라. python setup.py sdist를 실행해 소스 배포 파일을 생성하라.

끝으로, 당신이 만든 패키지를 파이썬 가상 환경에 설치해보라.

마지막 편집일시 : 2020년 7월 26일 12:27

끝!

이 과정을 모두 마쳤다. 시간과 노력을 기울인 것에 감사한다. 앞으로의 파이썬 해킹이 더욱

재미있고 생산적이 되기를 바란다!
언제든 의견을 고맙게 받겠다. 나를 https://dabeaz.com이나 트위터 @dabeaz에서 찾을 수 있다. - David Beazley.

마지막 편집일시 : 2020년 6월 25일 1:52

==========

A. Practical Python Programming - 강의 노트
저자: David Beazley

개요
이 문서는 나의 "Practical Python" 코스의 내용을 가르침에 있어 학습목표, 대상자, 주의점을 포함해 일반적인 참고 사항과 조언을 제공한다.

이 지침은 기업 트레이닝 환경의 전형적인 3일 교육 코스에서 사람들을 가르치는 사람들을 위한 것이다. 자신만의 코스를 만들어 가르치고자 하는 경우에도 이 문서가 도움이 될 것이다.

대상자 및 일반적인 접근
이 코스는 프로그래밍 경험이 있는 사람들을 위한 "파이썬 입문" 코스로 기획했다. "프로그래밍 첫걸음" 코스를 듣는 사람들을 가르치도록 설계한 것이 아니다.

필자의 경험상, 파이썬 코스를 들으러 오는 일반적인 수강생은 높은 수준의 소프트웨어 기술자 혹은 프로그래머가 아닌 경우가 많다. 기술자, 과학자, 웹 프로그래머, 경험이 적은 개발자가 뒤섞여 있을 가능성이 높다. 수강생의 배경은 매우 다양하다. C, C++, 자바를 다뤄본 학생이 있을 수도 있고, PHP와 HTML을 알 수도 있고, MATLAB 같은 도구를 다뤄봤을 수도 있다. 선수 조건을 명확히 하려 노력했음에도 불구하고 전통적인 "프로그래밍" 경험이 전혀 없는 수강생이 있을 수도 있다.

이를 염두에 두고, 이 코스는 데이터를 조작하는 일반적인 문제(특히 주식 시장의 데이터)를 통해 파이썬을 가르친다. 이 영역을 선택한 이유는 단순하면서도 수강생의 배경에 관계 없이 누구나 알기 때문이다. 예를 들어, 프로그래밍 기술이 부족한 학생도 스프레드시트(예: Excel)를 사용하는 것과 같이 일반적인 것은 알 것이다. 그러므로, 수강생이 헤매고 있을 때 "튜플은 스프레드시트의 데이터 열 같은 것"이라든지 "리스트 컴프리헨션(list comprehension)은 스프레드시트 컬럼에 연산을 적용한 결과를 다른 컬럼에 넣는 것"이라고 설명할 수 있다. 핵심 개념은 난해한 "컴퓨터 과학" 문제에 얽매이지 않고(예: "피보나치 수열을 계산해봅시다") 실세계에 발붙이는 것이다.

이 문제 영역은 다른 프로그래밍 주제를 소개할 때도 효과적이다. 예를 들어, 과학자/공학자는 데이터 분석이나 플로팅에 관심이 있을 것이다. 따라서, 그들에게는 matplotlib을 사용해 플롯을 작성하는 법을 보여줘라. 웹 프로그래머는 주식 시장 데이터를 웹페이지에 표시하는 법을 알고 싶을 것이다. 그러면 템플릿 엔진을 설명하면 된다. 시스템

관리자는 로그 파일을 다루는 법을 알고 싶을 것이다. 그러면 실세계의 주식 데이터 로그 파일을 가지고 수업을 하라. 소프트웨어 기술자는 설계에 관심이 있을 것이다. 그렇다면 주식 데이터를 객체에 캡슐화하는 방법이라든지 프로그램의 확장성을 높이는 법을 설명하라(예: 이 프로그램의 출력을 열 가지 테이블 포맷으로 표현하는 법). 아이디어를 내 보라.

발표 지침
발표 슬라이드(노트)는 이야기 구조로 되어 있으며 수강생이 연습을 할 때 참고할 수 있게 제공한다. 모든 슬라이드의 모든 항목을 다루려 애쓸 필요 없다. 수강생이 읽을 수 있으며 코딩하다가 다시 볼 시간이 있을 것으로 가정한다. 나는 슬라이드를 빠르게 훑으면서 상호작용적인 짧은 예를 보여주는 편이다. 시연을 하면서 슬라이드를 통째로 건너 뛰기도 한다. 모든 슬라이드를 다루지 않아도 된다. 몇 가지만 골라 인터프리터에서 직접 시연하라. 규칙: 특별히 까다롭지 않은 한, 슬라이드 당 1분을 넘기지 마라. 솔직히 말해서, 슬라이드 대부분은 넘기고 필요하다고 생각되는 것만 라이브 데모 위주로 강의를 해도 된다 나는 종종 그렇게 한다.

코스 연습 문제
이 코스에는 약 130 개의 연습 문제가 있다. If you do every single exercise and give students time to think and code, it will likely take them about 10-12 hours. In practice, you will probably find that students require more time on certain exercises. I have some notes about this below.

You should repeatedly emphasize to students that solution code is available and that it is okay to look at it and copy it--especially due to time requirements.

Prior to teaching the course, I would strongly advise that you go through and work every single course exercise so that there are no surprises.

During course delivery, I usually work every single exercise from scratch, without looking at the solution, on my computer while the students also work. For this, I strongly advise you to have a printed copy of the exercises on hand that you can look at without having to pull it up on the computer screen (which is being projected). Near the end of the exercise time period, I will start discussing my solution code, emphasizes different bits on the screen and talking about them. If there are any potential problems with the solution (including design considerations), I'll also talk about it. Emphasize to students that they may want to look at/copy solution code before going forward.

Section 1: Introduction
The major goal of this section is to get people started with the environment. This includes using the interactive shell and editing/run short programs. By the end of the section, students should be able to write short scripts that read data files and

perform small calculations. They will know about numbers, strings, lists, and files. There will also be some exposure to functions, exceptions, and modules, but a lot of details will be missing.

The first part of this course is often the longest because students are new to the tools and may have various problems getting things to work. It is absolutely critical that you go around the room and make sure that everyone can edit, run, and debug simple programs. Make sure Python is installed correctly. Make sure they have the course exercises downloaded. Make sure the internet works. Fix anything else that comes up.

Timing: I aim to finish section 1 around lunch on the first day.

Section 2 : Working with Data
This section is probably the most important in the course. It covers the basics of data representation and manipulation including tuples, lists, dicts, and sets.

Section 2.2 the most important. Give students as much time as they need to get exercises working within reason. Depending on audience, the exercises might last 45 minutes. In the middle of this exercise, I will often move forward to Section 2.3 (formatted printing) and give students more time to keep working. Together, Sections 2.2/2.3 might take an hour or more.

Section 2.4 has people explore the use of enumerate(), and zip(). I consider these functions essential so don't skimp on it.

Section 2.5 introduces the collections module. There is a LOT that could be said about collections, but it won't be fully appreciated by students at this time. Approach this more from the standpoint of "here's this cool module you should look at later. Here are a few cool examples."

Section 2.6 introduces list comprehensions which are an important feature for processing list data. Emphasize to students that list comprehensions are very similar to things like SQL database queries. At the end of this exercise, I often do an interactive demo involving something more advanced. Maybe do a list comprehension and plot some data with matplotlib. Also an opportunity to introduce Jupyter if you're so inclined.

Section 2.7 is the most sophisticated exercise. 그것은 파이썬의 일급(first-class) 데이터 사용과 관련이 있으며, 원하는 모든 종류의 객체를 리스트 같은 자료 구조에 담을 수 있다는 사실과 관련이 있다. The exercises are related to parsing columns of data in

CSV files and concepts are later reused in Section 3.2.

Timing: Ideally, you want to be done with section 2 on the first day. However, it is common to finish with section 2.5 or 2.6. So, don't panic if you feel that you're a bit behind.

3. 프로그램 조직화

The main goal of this section is to introduce more details about functions and to encourage students to use them. The section builds from functions into modules and script writing.

Section 3.1 is about going from simple “scripting” to functions. 수강생이 조직화되지 않은 “스크립트”를 작성하지 않게 하라. 코드는 함수를 사용해 모듈화해야 한다. 조직화된 코드는 이해하기 쉽고, 나중에 변경하기도 쉬우며, 실행 속도도 좀 더 빠르다. 함수는 좋다.

섹션 3.2의 연습 문제는 전체 코스에서 가장 고급일 것이다. 수강생은 컬럼 기반의 데이터를 파싱하는 범용 유틸리티 함수를 작성한다. 리스트 컴프리헨션과 함수의 리스트를 많이 사용한다(예: 함수를 일급 객체로서 사용). 사람들에게 코드의 각 단계가 어떻게 작동하는지 상세하게 안내할 필요가 있을 것이다. The payoff is huge however---you can show people a short general purpose function that does something amazingly powerful and which would be virtually impossible to write in C, C++, or Java without having a LOT of very complicated code. There are a lot of possible design/discussion avenues for this code. Use your imagination.

Section 3.3 adds error handling to the function created in Section 3.2 This is a good time to talk about exception handling generally. Definitely talk about the dangers of catching all exceptions. This might be a good time to talk about the “Errors should never pass silently” item on the “Zen of Python.”

*Note: Before Exercise 3.4, make sure students get fully working versions of report.py, pcost.py, and fileparse.py. Copy from Solutions folder if needed *

Section 3.4 Introduces module imports. The file written in Section 3.2-3.3 is used to simplify code in Section 3.1. Be aware that you may need to help students fix issues with IDLE, sys.path, and other assorted settings related to import.

Section 3.5 talks about __main__ and script writing. There's a bit about command line arguments. You might be inclined to discuss a module like argparse. However, be warned that doing so opens up a quagmire. It's usually better to just mention it and move on.

Section 3.6 opens up a discussion about design more generally in Python. Is it better to write code that's more flexible vs code that's hardwired to only work with filenames? This is the first place where you make a code change and have to refactor existing code.

Going forward from here, most of the exercises make small changes to code that's already been written.

4. 클래스와 객체

This section is about very basic object oriented programming. In general, it is not safe to assume that people have much background in OO. So, before starting this, I usually generally describe the OO "style" and how it's data and methods bundled together. Do some examples with strings and lists to illustrate that they are "objects" and that the methods (invoked via .) do things with the object. Emphasize how the methods are attached to the object itself. For example, you do items.append(x), you don't call a separate function append(items, x).

Section 4.1 introduces the class statement and shows people how to make a basic object. Really, this just introduces classes as another way to define a simple data structure--relating back to using tuples and dicts for this purpose in section 2.

Section 4.2 is about inheritance and how you use to create extensible programs. This set of exercises is probably the most significant in terms of OO programming and OO design. Give students a lot of time to work on it (30-45 minutes). Depending on interest, you can spend a LOT of time discussing aspects of OO. For example, different design patterns, inheritance hierarchies, abstract base classes, etc.

Section 4.3 does a few experiments with special methods. I wouldn't spend too much time fooling around with this. Special methods come up a bit later in Exercise 6.1 and elsewhere.

Timing: This is usually the end of the 2nd day.

5. Inside Objects

This section takes students behind the scenes of the object system and how it's built using dictionaries, how instances and classes are tied together, and how inheritance works. However, most important part of this section is probably the material about encapsulation (private attributes, properties, slots, etc.)

Section 5.1 just peels back the covers and has students observe and play with the

underlying dicts of instances and classes.

Section 5.2 is about hiding attributes behind get/set functions and using properties. I usually emphasize that these techniques are commonly used in libraries and frameworks--especially in situations where more control over what a user is allowed to do is desired.

An astute Python master will notice that I do not talk about advanced topics such as descriptors, or attribute access methods (__getattr__, __setattr__) at all. I have found, through experience, that this is just too much mental overload for students taking the intro course. Everyone's head is already on the verge of exploding at this point and if you go talk about how something like descriptors work, you'll lose them for the rest of the day, if not the rest of the course. Save it for an "Advanced Python" course.

If you're looking at the clock thinking "There's no way I'm going to finish this course", you can skip section 5 entirely.

6. 제너레이터

The main purpose of this section is to introduce generators as a way to define custom iteration and to use them for various problems related to data handling. The course exercises have students analyze streaming data in the form of stock updates being written to a log file.

There are two big ideas to emphasize. First, generators can be used to write code based on incremental processing. This can be very useful for things like streaming data or huge datasets that are too large to fit into memory all at once. The second idea is that you can chain generators/iterators together to create processing pipelines (kind of like Unix pipes). Again, this can be a really powerful way to process and think about streams, large datasets, etc.

Some omissions: Although the iteration protocol is described, the notes don't go into detail about creating iterable objects (i.e., classes with __iter__() and next()). In practice, I've found that it's not necessary to do this so often (generators are often better/easier). So, in the interest of time, I've made a conscious decision to omit it. Also not included are extended generators (coroutines) or uses of generators for concurrency (tasklets, etc.). That's better covered in advanced courses.

7. 고급 주제

Basically this section is an assortment of more advanced topics that could have

been covered earlier, but weren't for various reasons related to course flow and content of the course exercises. If you must know, I used to present this material earlier in the course, but found that students were already overloaded with enough information. Coming back to it later seems to work better---especially since by this point, everyone is much more familiar with working in Python and starting to get the hang of it.

Topics include variadic function arguments (args, *kwargs), lambda, closures, and decorators. Discussion of decorators is only a tiny hint of what's possible with metaprogramming. Feel free to say more about what's possible, but I'd probably stay out of metaclasses! Lately, I have been demoing "numba" as an example of a more interesting decorator.

If you're pressed for time, most of section 7 can be skipped or heavily compressed (you could skip exercises for instance).

8. Testing and Debugging
The main purpose of this section is just to introduce various tools and techniques related to testing, debugging, and software development. Show everyone the unittest module. Introduce the logging module. Discuss assertions and the idea of "contracts." Show people the debugger and profiler. Most of this is self-explanatory.

9. 패키지(Package)
At this point, students have written an assortment of files (pcost.py, report.py, fileparse.py, tableformat.py, stock.py, portfolio.py, follow.py, etc.). Two main goals in this section. First, put all of the code into a Python package structure. This is only a gentle introduction to that, but they'll move the files into a directory and everything will break. They'll need to fix their import statements (package relative imports) and maybe fiddle with an __init__.py file. Second goal, write a simple setup.py file that they can use to package up the code and give it away to someone. That's it. End of the course.

마지막 편집일시 : 2020년 7월 3일 2:33